Dissertation presented to the Instituto Tecnológico de Aeronáutica, in partial fulfillment of the requirements for the degree of Master of Science in the Graduate Program of Electronic and Computer Engineering, Field of Computer Science .

Luiz Carlos Giacomossi Junior

COOPERATIVE DECISION-MAKING FOR DRONE SWARMS IN SIMULATED ENVIRONMENTS

Dissertation approved in its final version by signatories below:

Prof. Dr. Marcos R. O. A. Maximo

Advisor

Eng. Dr. José Fernando Basso Brancalion

Co-advisor

Prof. Dr. Emília Villani

Pro-Rector of Graduate Courses

Campo Montenegro
São José dos Campos, SP - Brazil
2023

Cataloging-in Publication Data
Documentation and Information Division

Giacomossi Junior, Luiz Carlos
Cooperative Decision-Making for Drone Swarms in Simulated Environments / Luiz Carlos Giacomossi Junior.
São José dos Campos, 2023.
169f.

Dissertation of Master of Science – Course of Electronic and Computer Engineering. Area of Computer Science – Instituto Tecnológico de Aeronáutica, 2023. Advisor: Prof. Dr. Marcos R. O. A. Maximo. Co-advisor: Eng. Dr. José Fernando Basso Brancalion.

1. Drones Swarms. 2. Decision-Making. 3. Loyal Wingman. I. Instituto Tecnológico de Aeronáutica. II. Title.

BIBLIOGRAPHIC REFERENCE

GIACOMOSSI JUNIOR, Luiz Carlos. **Cooperative Decision-Making for Drone Swarms in Simulated Environments**. 2023. 169f. Dissertation of Master of Science – Instituto Tecnológico de Aeronáutica, São José dos Campos.

CESSION OF RIGHTS

AUTHOR'S NAME: Luiz Carlos Giacomossi Junior
PUBLICATION TITLE: Cooperative Decision-Making for Drone Swarms in Simulated Environments.
PUBLICATION KIND/YEAR: Dissertation / 2023

It is granted to Instituto Tecnológico de Aeronáutica permission to reproduce copies of this dissertation and to only loan or to sell copies for academic and scientific purposes. The author reserves other publication rights and no part of this dissertation can be reproduced without the authorization of the author.

Luiz Carlos Giacomossi Junior
Avenida Rocha Pombo, 1733
83005-280 – São José dos Pinhais–PR

COOPERATIVE DECISION-MAKING FOR DRONE SWARMS IN SIMULATED ENVIRONMENTS

Luiz Carlos Giacomossi Junior

Thesis Committee Composition:

Prof. Dr.	José Maria Parente de Oliveira	President	-	ITA
Prof. Dr.	Marcos R. O. A. Maximo	Advisor	-	ITA
Eng. Dr.	José Fernando Basso Brancalion	Co-advisor	-	EMBRAER
Profa. Dr.	Ana Carolina Lorena	Internal Member	-	ITA
Prof. Dr.	Jaime Shinsuke Ide	External Member	-	Yale Lockheed Martin

ITA

I dedicate this thesis to all the people who accompanied me on this academic journey, especially to my family and friends, for their unconditional love, encouragement, and support over the years. I also thank the professors and colleagues who inspired and challenged me to grow academically. This achievement is ours, and I share it with you with all my heart.

Acknowledgments

I would like to express my sincere gratitude to Professor Marcos Maximo and Dr. José Fernando Brancalion, my advisors, for their unwavering support and guidance throughout my research journey. Their expert advice and valuable feedback have been instrumental in shaping this work and have significantly contributed to its quality. Their commitment to my research has been inspiring, and I have learned so much from their knowledge and expertise. I am truly fortunate to have had them as my mentors, and I will always be grateful for their contributions to my academic growth.

Equally important, I want to convey my profound appreciation to Embraer for providing me with a scholarship that has been instrumental in enabling me to pursue my academic goals. This scholarship has not only eased my financial burden but has also provided me with the necessary resources and support to complete my dissertation. I am deeply grateful to Embraer's commitment to higher education and for investing in the future of talented students. This opportunity has been life-changing, and I will always be thankful for the generosity and support that Embraer has shown me.

Furthermore, I am deeply grateful to Professors Peter Funk and Rickard Sohlberg from Mälardalen University, members of the Artificial Intelligence and Intelligent Systems Group, in Västerås-Sweden for providing me with the invaluable opportunity to contribute to their research project. Working alongside them has been an honor and a privilege, and I have learned so much from their expertise, guidance, and mentorship. Their dedication, enthusiasm, and support have been instrumental in my academic growth, and I will always cherish this experience.

Also, I would like to acknowledge the love and support of my parents, Luiz Carlos Giacomossi and Alessandra Brenneisen Giacomossi, who have been my constant source of strength and motivation. Without their unwavering support and love, this accomplishment would not have been possible.

Finally, my special thanks go to my colleagues and friends Jorge Ricardo Jr., Nils Sundelius, Jonathan de Oliveira and Angelo Caregnato-Neto, who have offered their encouragement, support, and valuable assistance and insights throughout the project.

"No problem can withstand the assault of sustained thinking."
— Voltaire

Resumo

O enxame de Veículos Aéreos Não Tripulados (VANT), também denominado enxame de *drones*, tem sido objeto de estudo de diversas pesquisas devido ao seu potencial para aprimorar aplicações como monitoramento, vigilância, missões de busca e resgate e missões de defesa. Com vários *drones* voando simultaneamente, o desafio é aumentar seu nível de automação e inteligência, evitando colisões, reduzindo o nível de comunicação com essas entidades e melhorando a organização estratégica para realizar uma tarefa específica. Com a aplicação de técnicas de tomada de decisão, é possível automatizar muitas das decisões inteligentes dos VANTs, deixando que os comandantes humanos se concentrem nas decisões de alto nível em cenários modernos, proporcionando uma vantagem tática crucial. Neste contexto, se estudam duas aplicações, sendo a primeira um cenário de defesa e a segunda um cenário civil de busca e resgate de pessoas. A capacidade de vários sistemas de aeronaves tripuladas e não tripuladas de engajar e desativar cooperativamente uma ameaça aérea desempenha um papel decisivo nos cenários de guerra modernos. Neste contexto, aplicamos métodos-chave para habilitar a chamada capacidade de engajamento cooperativo de ameaças entre uma equipe de agentes, *e.g.*, um enxame de *drones*, com capacidades de combate e comunicação. Em particular, esta pesquisa combina técnicas de tomada de decisão e controle baseadas em IA para um enxame de *drones* leais para coordenar ações de defesa eficientes de maneira cooperativa e autônoma. Nesse sentido, aplicamos esses conceitos em um cenário de defesa modelado para analisar o conceito de *loyal wingman*, que é um interessante teste para tomada de decisão cooperativa e técnicas de controle de baixo nível. Os métodos investigados foram implementados inicialmente em um simulador 2D e posteriormente em simulador 3D de alta fidelidade para fornecer uma aplicação e avaliação de estratégias de comportamento e métodos de controle. Neste trabalho também se estuda a aplicação de exames em um cenário de busca e resgate de pessoas, onde se propõe uma solução para coordenar um enxame utilizando uma inteligência autônoma e distribuída entre *drones* para uma aplicação cooperativa de busca de pessoas com base em um mapa discreto compartilhado. Os resultados mostraram uma eficácia do enxame usando métodos de tomada de decisão distribuídos, indicando ser uma abordagem eficaz para resolver o problema.

Abstract

The swarm of Unmanned Aerial Vehicles (UAV), also called swarm of drones, has been the subject of several research studies due to its potential to modernize applications such as monitoring, surveillance, search and rescue, and defense missions. With several drones flying simultaneously, the challenge is to increase their level of automation and intelligence, avoiding collisions, reducing the level of communication with these entities and improving the strategic organization to carry out a specific task. With the application of decision-making techniques, we can automate many of the intelligent decisions of UAVs, leaving human commanders to focus on high-level decisions in modern scenarios, providing a crucial tactical advantage. In this context, two applications are studied, the first being a defense scenario and the second a civilian search and rescue scenario. The ability of multiple manned and unmanned aircraft systems to cooperatively engage and disable an aerial threat plays a decisive role in modern warfare scenarios. In this context, we apply key methods to enable the so-called cooperative threat engagement capability among a team of agents, *e.g.*, a swarm of drones, with combat and communication capabilities. In particular, this research combines AI-based decision-making and control techniques to a swarm of loyal drones to coordinate efficient defense actions cooperatively and autonomously. In this sense, we apply these concepts in a modeled defense scenario to analyze the *Loyal Wingman* concept, which is an interesting test for cooperative decision making and low-level control techniques. The investigated methods were initially implemented in a 2D simulator and later in a high fidelity 3D simulator to provide an application and evaluation of behavior strategies and control methods. This work also studies the application of tests in a scenario of search and rescue of people, where a solution is proposed to coordinate a swarm using an autonomous and distributed intelligence between drones for a cooperative application of search of people based on a shared discrete map. The results showed a swarm effectiveness using distributed decision-making methods, indicating that this is an effective approach to solve the problem.

List of Figures

FIGURE 1.1 – Defense Scenario where a Manned-Unmanned Team (MUM-T) is protecting a leading drone and critical infrastructure from aerial threats 26

FIGURE 1.2 – Search and Rescue Scenario where a drone swarm is searching for a person in distress in a forest. 27

FIGURE 2.1 – Example of a Finite State Machine (FSM). 42

FIGURE 2.2 – Example of generic Behavior-Tree (BT). 44

FIGURE 2.3 – Example of a Behavior tree (BT). 45

FIGURE 2.4 – Example of a swarm formation executing the lawn mower search pattern. The dashed lines illustrate the paths. 58

FIGURE 2.5 – One-dimensional normal bivariate function. 65

FIGURE 2.6 – Interaction of potential fields between drones and obstacles. 66

FIGURE 2.7 – Diagram for a simplified flight control system for a 2D drone. 69

FIGURE 3.1 – Scenario of interest where LW UAVs, highlighted in blue, supported by ground assets, escort a leader UAV and engage kamikaze threats in order to defend a protected area. 76

FIGURE 3.2 – Problem breakdown indicating the main software layers. 79

FIGURE 3.3 – Problem I definition diagram. 82

FIGURE 3.4 – Decision-Making module for a kamikaze UAV using FSM. 84

FIGURE 3.5 – Behavior tree for the loyal-wingman UAV. The blue-circled behaviors represent movement behaviors, and the red-circled behaviors depicting offensive behaviors. 88

FIGURE 3.6 – Join formation sequence diagram. 89

FIGURE 3.7 – Altruistic and Selfish behavior sequence. 90

FIGURE 3.8 – Diagram of LW-Kamikaze task allocation problem. 91

FIGURE 3.9 – Uniform-circular formation pattern. 92

FIGURE 3.10 –Simulator view, we see the MUM-T being attacked by a kamikaze swarm. The protected area is seen on the upper-right corner. A LW is seen up-close. 94

FIGURE 3.11 –MUM-T under attack of a kamikaze swarm, leader at the center and Loyal Wingmen in formation. LW in yellow destroying a kamikaze (vaporizer gun). LW marked in red is slowing down a kamikaze (freezing gun). 95

FIGURE 3.12 –Schematic representation of a non-planar fully actuated octocopter. To enhance safety during takeoff, landing, and general operation, a protective cage is depicted as a blue box surrounding the drone. This cage serves the purpose of safeguarding the rotors and providing overall protection to the system. 96

FIGURE 3.13 –Diagram of the computational implementation. 97

FIGURE 3.14 –Data flow diagram. 98

FIGURE 3.15 –Class diagram of the components that implement the high-level decision-making module. 103

FIGURE 3.16 –full view of a running simulation using fully-actuated octacopters. Blue Dome represents the protected area.The Black Drone represents the manned-drone, which is operated by a human pilot; White Drones symbolize the loyal wingman drones, which operate in a collaborative manner. Red Drone represents a kamikaze threat, signifying a potential danger. 104

FIGURE 3.17 –full view of a running simulation, we can see the decision-making visualizer and 3D simulator in the upper right part of the figure. The position of each agent can be seen in the legend at the top left. Each drone have a unique color, and have a legend that presents its current behavior, a velocity vector, the ammunition of its weapons and the availability, and a line indicating the agent's destination waypoint. Note that both visualizers represent the same situation from 2D and 3D viewpoints. 104

FIGURE 3.18 –Alternative visualization of the scenario involving under-actuated UAVs. The simulation showcases a fleet of drones in full motion, with each drone distinguished by its distinctive color scheme. The red drones symbolize the Kamikazes. The white drones symbolize the loyal wingmen. Meanwhile, the green drone represents the leader. 105

FIGURE 3.19 –ROS 2 network diagram of the communication between software layers. 108

FIGURE 3.20 –Setup Sequence diagram. 111

FIGURE 3.21 –Parameters description. 114

FIGURE 3.22 –Group attacking – cooperative work to neutralize threats. 116

FIGURE 3.23 –Sacrifice attack to preserve MUM-T as last resort tool. 116

FIGURE 3.24 –Cooperative work to neutralize threat. 117

FIGURE 3.25 –Loyal Wingman specializations according to regions in formation . . 117

FIGURE 3.26 –Demonstration sequence of engagement to a Kamikaze threat, in red. The leader is seen in the center, in green, and four Loyal Wingman in blue. 119

FIGURE 3.27 –Demonstration of a cooperative neutralization sequence. In this simulation, the leader is in seen in black. The LW1 are represented in the blue circle, LW2 in the yellow circle and the Kamikazes in red. 120

FIGURE 3.28 –Demonstration of the sacrifice neutralization sequence. In this simulation, the leader is in seen in black. The LW are represented in white and the Kamikazes in red. 121

FIGURE 3.29 –Freezing attack being used as a neutralization method. In this simulation, the leader is seen in black. The LW1 in white and the Kamikazes K1 and K2 in red. 121

FIGURE 3.30 –Monte Carlo simulation results. ST stands for survival time and KD stands for Kamikazes destroyed. 122

FIGURE 3.31 –Overall simulation iteration time. Legend: • iteration time, red line is the iteration time average, and block adopted sampling time. . . . 123

FIGURE 4.1 – Search and Rescue scenario, a swarm of drones, highlighted within the red circle, is deployed. The environment is populated with obstacles, symbolized as trees, while the primary objective is to locate and rescue a lost person, represented as the target. 125

FIGURE 4.2 – Example of a probability map. Green, Yellow and Red cells represent low, medium and high probability regions, respectively. No flight zones are illustrated as Blue cells. The star represents the object of interest. 128

FIGURE 4.3 – Finite State Machine that controls the drone's behavior. 131

FIGURE 4.4 – Drones avoiding collision during simulation execution. 132

FIGURE 4.5 – To provide clear visual indication, we represent visited cells in green and not visited cells in red. 133

FIGURE 4.6 – The mission commences with the deployment of UAVs, which are clearly identifiable as they are marked in yellow and distributed across the map. In the designated yellow square, we observe a drone actively selecting an unvisited cell among the eight neighboring cells to continue the search process. 133

FIGURE 4.7 – Implementation of the unvisited cell search. We can see a drone in a position where it is surrounded by visited cells. In this case, the drone will randomly select a cell to proceed, illustrated by the yellow arrows. 134

FIGURE 4.8 – Drone stuck in obstacle looking for the nearest drone. 135

FIGURE 4.9 – Mission closure phase, target is located by one UAV in the swarm. The drone then shares the location on the person and all drones navigate towards the individual. 135

FIGURE 4.10 –Target is located and a swarm is formed around the missing person. 136

FIGURE 4.11 –Example of search process using three drones. Green, yellow, and red cells represent low, medium, and high probability regions, respectively. No flight zones are illustrated as blue cells. The yellow circle with a person represents the object of interest. Each line with a different color represents the path taken by a different drone. Gray circle represents the start point. 138

FIGURE 4.12 –Drone performing the search in the 8-connected cells. Red, Yellow and Blue cells represent the high, medium and low probabilities respectively. In Gray, cells that were already visited. 139

FIGURE 4.13 –Drone performs the search using the Bayes algorithm and updates the probabilities associated with each cell as it visits them. These probabilities correspond to the numbers assigned to each cell and are updated at every new cell the drone explores. Note that these probabilities are multiplied by 100. 139

FIGURE 4.14 –2D Simulation view of the search mission in using Bayesian probabilities and HC search. 140

FIGURE 4.15 –High-level overview of the components and interactions within the simulation environment. 141

FIGURE 4.16 –Mission completion time by the number of drones used. 143

FIGURE 4.17 –Local minima observed in each tested scenarios. 144

FIGURE 4.18 –Drone performing the search in the 8-connected cells. Red, Yellow and Blue cells represent the high, medium and low probabilities respectively. In Gray, cells that were already visited. The numbers represent the probability of each cell. 146

FIGURE 5.1 – Search and rescue scenario adapted and performed in the py-bullet 3D environment. The cells' colors represent the probability of a subregion of the map. Green refers to low probability cells, Yellow to medium probability, and Red cells to high probability cells. The duck represents the missing individual. 154

FIGURE 5.2 – End of the search, with the individual being found and all drones near their location. The gray cells represent the discarded areas where the search was conducted and the individual was not identified. Individual represented by the rubber duck on the blue cell. . . 155

List of Tables

TABLE 2.1 – Node types and return logic. 44

TABLE 3.1 – Behavior description for a Kamikaze drone. 85
TABLE 3.2 – Behavior description for an LW UAV in the 3D environment. 86
TABLE 3.3 – Use Case description for a Loyal Wingman UAV. 87
TABLE 3.4 – Table of commands. 98
TABLE 3.5 – Weapons and PSO parameters. 113
TABLE 3.6 – Experimental Results. 114

TABLE 4.1 – Proposed heuristics for Swarm Intelligence. 130
TABLE 4.2 – Experiments description. 142
TABLE 4.3 – Results of the experiments. 145

List of Abbreviations and Acronyms

AI	Artificial Intelligence
ACE	Air Combat Evolution program
AVO	Acceleration Velocity Obstacles method
BT	Behavior Tree
BS	Bayesian Search Theory
CEC	Cooperative Engagement Capability
CMA-ES	Covariance Matrix Adaptation Evolution Strategy
CTEDS	Cooperative Threat Engagement with Drone Swarms
DARPA	Defense Advanced Research Projects Agency
DQN	deep Q network
ED	Enemies Destroyed
FCS	Flight Control System
FSM	Finite State Machine
GCS	Ground Control System
GPS	Global Positioning System
HC	Hill Climbing
HL	High-Level Decision-Making
ITA	Aeronautics Institute of Technology
KD	Kamikazes Destroyed
LL	Low-Level Control
LMS	Lawn Mower Search
LW	Loyal Wingman
LWS	LW survived
MAV	Multirotor Aerial Vehicle
MDU	Mälardalen University
MUM-T	Manned-Unmanned Team
PA	Protected Area
PID	proportional–integral–derivative
PF	Potential Field

PSO	Particle Swarm Optimization
RL	Reinforcement Learning
ROS	Robot Operating System
SA	Situational Awareness
SAR	Search and Rescue
SMC	Sliding Mode Control
SoS	Systems-of-Systems
ST	Survival Time
UAV	Unmanned Aerial Vehicle
UCAV	Unmanned combat aerial vehicle

List of Symbols

M	Binary matrix
$M_{i,j}$	Element in row i and column j
C	Cost matrix
S	Set of zeros on the alternating path
$\mathbf{v}_i^{t+1}$	The updated velocity of particle i at time $t+1$
w	The inertia weight for the PSO
$\mathbf{v}_i^t$	Current velocity of a particle in the PSO
c_1	Cognitive coefficient
c_2	Social acceleration coefficient
$\mathbf{p}_i$	Particle's i personal best
$\mathbf{p}_g$	Swarm's global best
$\mathbf{r}_1$	Random vector – uniform distribution between 0 and 1
$\mathbf{r}_2$	Random vector – uniform distribution between 0 and 1
$\mathbf{x}_i^t$	The current position of particle i at time step t
$\mathbf{x}_i^{t+1}$	The updated position of particle i
$\mathbf{x}_i^t$	The current position of particle i
$\mathbf{v}_i^{t+1}$	The updated velocity of particle i at a time step
$P(A\|B)$	The posterior probability of event A given data B
$P(A)$	The prior probability of event A
$P(B)$	The probability of observing the data B
$P(B\|A)$	The likelihood of observing the data B given event A
$P(B \cap A)$	the probability of B and A
$\mathbf{F_r}$	Repulsive force vector
d	Distance to the obstacles
Q	Repulsion coefficient
$\mathbf{p}$	The position vector of the agent in the environment
$Nobs$	The quantity of obstacles in the environment
$U(\mathbf{p})$	The potential energy function
$\nabla U(\mathbf{p})$	The gradient of the potential energy function
$\overrightarrow{d}_{i,k}$	The distance between the drones i and an obstacle k

$\mathbf{p}_r$	Position reference
$\mathbf{p}$	Current position
$\boldsymbol{f}$	force command signal
ψ_r	Yaw angle reference
ψ	Current yaw angle
$\dot{\psi}$	Angular velocity
K_p	P+V Controller position proportional coefficient
K_v	P+V Controller velocity proportional coefficient
$\mathbf{v}$	Current velocity of the drone
K_y	Yaw angle's proportional coefficient
K_{yv}	Yaw rate's proportional coefficient
`num_loyal_wingman`	Number of Loyal Wingman present in a simulation
`cartridges_vaporizing`	Vaporizer gun ammunition
`cartridges_freezing`	Freezing gun ammunition
`freezing_cooldown`	Cooldown time for the freezing gun
`vaporizer_cooldown`	Cooldown time for the vaporizer gun
`freezing_range`	Range of the Freezing gun
`vaporizer_range`	Range of the Vaporizer gun
`time_frozen`	Time span that the Kamikaze is frozen
`freezing_hit_prob`	Hitting probability for the Freezing gun
`vaporizer_hit_prob`	Hitting probability for the Freezing gun
`max_num_of_kamikaze`	Amount of Kamikazes present in the simulated scenario
`max_detection_range`	Ground-based radars's Maximum detection distance
n	Total number of autonomous agents in the scenario
$\mathcal{I}$	Set containing the IDs of all the agents
$\mathcal{I}_m$	Set containing the IDs of the MUM-T members
$\mathcal{I}_k$	Set containing the IDs of the Kamikaze agents
$\mathcal{A}$	Set of possible actions for an agent
a	Discrete action
$\mathcal{S}$	Set containing the state of an agent
$\mathbf{s}$	State containing the current state of an agent
$\mathcal{W}$	Set of workers
$\mathcal{J}$	Set of tasks
j_i	Task i
$\mathcal{C}$	Set of costs
w_i	Worker i
$\mathcal{W}$	Set of Workers
$\mathbf{C}$	Cost matrix
$c_{i,j}$	The element of the ith row and jth column of $\mathbf{C}$

$\mathbf{L}$	Permutation matrix
$\mathbf{R}$	Permutation matrix
`Leader`	Boolean representing the leader
$\mathcal{K}$	Set of behaviors for the kamikaze agent
$\mathtt{t}_1$	Idle state time
$\mathtt{t}_1$	Time to select closest LW to attack
`PA`	Boolean representing the protected area
`LW Swarm Destroyed`	Boolean representing the destruction of the MUM-T
$\mathtt{p}_1$	Probability to attack the Leader
$\mathtt{p}_2$	Probability to attack the Loyal Wingman
$\mathtt{p}_3$	Probability to attack the protected area (PA)
$\mathtt{p}_4$	Probability to attack PA or Leader
$d(\mathbf{p}, \mathbf{q})$	The distance of the loyal wingman $\mathbf{p}$ to a threat $\mathbf{q}$
A_v	Availability of LW's weapons
P	Penalty cost
d_a	Engagement range radius
r_f	Formation radius
n_f	Quantity of Loyal Wingman in formation
$\mathbf{r}_g^{\ell/g}$	The position vector of the leader
$\mathbf{D}_3(\cdot)$	The rotation matrix around the $\vec{z}_g$ axis
α	Formation's step angle
ϕ	Formation's offset angle
$\mathbf{e}_1$	Vector pointing in the direction of the first axis
$\check{\mathbf{r}}_g^{j/g}(t)$	The target position for the agent j in GCS
$\mathbf{r}_g^{j/g}(t)$	Current position of the Kamikaze agent j in GCS
$\mathbf{r}_g^{\ell/g}(t)$	Current position of the leader ℓ agent in GCS
$\check{\mathbf{r}}_g^{i/l}$	The target position for agent i relative to the point l
$\mathbf{e}_3$	Unit vector in the direction of the z-axis in the GCS
d_t	Distance maintained from the target during the chase
$\mathbf{r}_g^{k/\ell}(t)$	Relative position of the target Kamikaze k agent with respect to the reference point ℓ
ED	Amount of kamikazes destroyed
LWS	Amount of Loyal Wingman survivors
ST	Duration of an iteration
$Fitness$	Optimization reward function
`distance_engament`	Distance for the lw engage a kamikaze
`distance_formation`	Formation radius
R_{person}	Range of detection of a missing person
$h(w, l)$	A cell in the search map

p	The probability of the target being in a cell
f	the probability of finding the target in the cell
p'	The posterior probability of a searched cell
r'	The posterior probability of the other cells
r	The individual probability of all other cells

Contents

1 Introduction 24
1.1 Objectives 24
1.2 Motivation 25
1.3 Literature Review 27
1.3.1 Drone Swarms Challenges 29
1.3.2 Benefits and Limitations of drone swarms applications 30
1.3.3 Loyal Wingman Application 30
1.3.4 Search and Rescue Application 33
1.4 Contributions 35
1.5 Text Organization 37

2 Background 39
2.1 Decision-making in Robotics 40
2.2 AI Decision-making Techniques 41
2.2.1 Finite State-Machines for Decision-Making 42
2.2.2 Behavior Trees 43
2.3 Cooperation in Robotics 49
2.4 Optimization Algorithms 51
2.4.1 Particle Swarm Optimization (PSO) 52
2.4.2 The Kuhn-Munkres Algorithm 54
2.5 Search Algorithms 56
2.5.1 Lawn Mower Search (LMS) 57
2.5.2 Hill Climbing (HC) Search 59

2.5.3 Bayesian Search (BS) Theory . . . 60
2.6 Collision Avoidance . . . 63
2.6.1 Potential Field (PF) . . . 64
2.6.2 Acceleration-Velocity-Obstacles (AVO) . . . 66
2.7 Position and Attitude Controllers . . . 68
2.7.1 Sliding Mode Control (SMC) . . . 70
2.8 Tools for UAV Simulation Development . . . 71
2.8.1 Python and Pygame library . . . 71
2.8.2 Robot Operating System (ROS) and ROS2 . . . 73

3 Loyal Wingman Application . . . 75
3.1 Loyal Wingman Scenario . . . 75
3.1.1 Mission . . . 77
3.1.2 Description of the entities . . . 77
3.2 Problem Breakdown . . . 79
3.2.1 Low-Level Control . . . 80
3.3 Problem Definition . . . 81
3.3.1 LW-Kamikaze Assignment Problem . . . 82
3.4 Decision-Making Development . . . 83
3.4.1 Decision-Making for the Kamikaze UAV . . . 84
3.4.2 Decision-Making for the Loyal Wingman UAV . . . 85
3.5 Drone Swarm Simulation . . . 93
3.5.1 High-Level 2D Prototype . . . 93
3.5.2 High-Fidelity 3D Simulation . . . 95
3.5.3 Communication Interface . . . 103
3.5.4 Messages definition . . . 107
3.5.5 Simulation Setup . . . 111
3.6 Results and Discussion . . . 112
3.6.1 Metrics of Interest . . . 112
3.6.2 High-Level Prototype Experiments and Results . . . 113

3.6.3 High-Fidelity 3D Simulator Experiments and Results 118

4 Search and Rescue Application 124

4.1 Search and Rescue Scenario 124

4.1.1 Mission 125

4.1.2 Description of the entities 126

4.2 Problem Breakdown 127

4.2.1 Problem Definition 129

4.3 Search Solution Development 129

4.3.1 Decision-Making for the Search and Rescue Drone Swarm 130

4.3.2 Bayesian Search for Search and Rescue 136

4.3.3 Search Process using a Probability Map 138

4.4 UAV Search Swarm Simulation 140

4.5 Results and Discussions 141

4.5.1 Experiments of the Decision-Making 142

4.5.2 Experiments with the Map of Probabilities 144

5 Conclusion 148

5.1 General Conclusions 148

5.2 Loyal Wingman Application 149

5.3 Search and Rescue Application 152

5.4 Similarities Between Contexts 154

Bibliography 157

1 Introduction

In this chapter, we provide an introduction to the context addressed in this dissertation. We commence by outlining the objectives of the work, followed by an exploration of the motivation and a comprehensive literature review on drone swarms and their selected applications. Lastly, we present the contributions made by this project.

1.1 Objectives

The main objective is to investigate decision-making methods for swarms of drones in two distinct scenarios: defense scenario where we apply Loyal Wingmen drones and a civil application for search and rescue of a missing person. The specific objectives are:

1. Define the scenario and capabilities required for a cooperative air engagement system with a swarm of heterogeneous drones, focusing on the context of Loyal Wingman UAVs and their kamikaze adversaries in simulated environments.

2. Develop a decision-making module prototype that implements distributed autonomous decision-making techniques for the cooperative engagement system.

3. Collaborate with Embraer and other ITA researchers to develop a simulation of a swarm of loyal wingmen drones in a high-fidelity simulated system.

4. Collaborate with Embraer and other ITA researchers to ensure consistency between the high-level decision-making and the low-level system, that includes cooperative path planning and guidance, as well as the definition of the interface between the high-level and low-level systems.

5. Define the intended behavior of the system and select the appropriate artificial intelligence techniques and technologies to implement it effectively.

6. Implement simulation environments that incorporates the agents and cooperative engagement capacity, along with the autonomous decision-making system.

7. Develop intelligent search strategies for search and rescue missions using drone swarms.

8. Develop a high-level decision-making simulation with simplified dynamics, but with easy and quick prototyping for the development and testing of search and rescue algorithms for drones in a 2D environment.

9. Evaluate the performance of the system through simulations, using relevant evaluation metrics to measure the effectiveness of the cooperative engagement techniques implemented.

10. Analyze and document the results of the evaluation, identifying areas of improvement and potential future directions for research and development.

1.2 Motivation

The study of decision-making in drone swarms presents a compelling and multifaceted motivation. As unmanned aerial vehicles continue to proliferate in various industries and applications, the collective behavior of drone swarms becomes an increasingly important area of research. These swarms possess the potential to perform complex tasks, such as search and rescue operations, patrol monitoring, and even military operations, with greater efficiency and versatility compared to individual drones. Understanding the decision-making processes within these swarms is crucial for optimizing their performance, enhancing their autonomy, and ensuring safe and reliable operation. By unraveling the intricate mechanisms by which drone swarms make decisions, we can unlock new insights into swarm intelligence, human-swarm interaction, and the principles that govern emergent behavior in complex systems. Ultimately, this knowledge can propel the development of advanced swarm technologies, revolutionizing industries and transforming the way we approach critical challenges in the modern world. The study of decision-making in drone swarms is essential for enhancing effectiveness and efficiency in civil and military applications, such as search and rescue missions and the deployment of loyal wingman drones, an emerging type of combat drone which is being developed by the aeronautical industry.

This work aims to investigate the utilization of Loyal Wingman (LW) drones equipped with combat capabilities in engaging and disabling aerial explosive threats through cooperative efforts. Specifically, we evaluate a defense scenario where a Manned-Unmanned Team (MUM-T) comprises Loyal Wingmen UAVs tasked with protecting a leader unmanned aerial vehicle (UAV) and critical infrastructure from aerial threats, illustrated in Figure 1.1. Furthermore, the investigation presented in this dissertation offers an analysis of the developed scenario within this context.

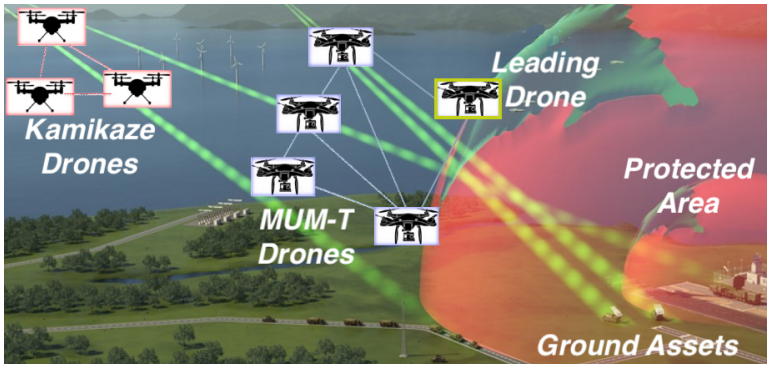

FIGURE 1.1 – Defense Scenario where a Manned-Unmanned Team (MUM-T) is protecting a leading drone and critical infrastructure from aerial threats

In our research, the vehicles are modeled as fully actuated UAVs, capable of independent attitude and position tracking (Brescianini; D'Andrea, 2016; SANTOS; BEZERRA, 2022). To address the design challenges arising from this scenario, we propose a problem breakdown that focuses on both high-level (HL) decision-making tasks and low-level (LL) control aspects, enabling effective communication, coordination, and collaboration between the leader and Loyal Wingmen in a defense scenario. The LL control aspect addresses the robustness requirements of UAV flight control systems against disturbances and uncertainties by designing a joint geometric attitude-position control law using a multi-input first-order sliding mode strategy. Additionally, to incorporate collision avoidance capabilities for the UAVs, we utilize the reciprocal acceleration velocity obstacles (AVO) method (BERG *et al.*, 2011a).

Furthermore, the autonomous behavior of the MAVs is designed using state-of-the-art artificial intelligence (AI) techniques, employing finite-state machines (FSMs) and behavior trees (BTs) decision-making algorithms (IOVINO *et al.*, 2022a) to select and execute modular behaviors for both the LW and the aerial threats. An emphasis is placed on ensuring cohesive teamwork among the agents, thereby promoting effective cooperation within the MUM-T. To achieve this, we employ the classic Kuhn-Munkres task allocation algorithm (KUHN, 1955) to distribute tasks between the MUM-T members.

The Cooperative Engagement Capability (CEC) is a systems-of-systems (SoS) capability that has been extended to include aerial platforms, such as drones. In this project, our focus is on investigating distributed autonomous decision-making applied to a swarm of heterogeneous drones within the CEC concept. By leveraging recent advances in robotics, we aim to enhance the coordination and collaborative actions of multiple drones to accomplish specific tasks. This includes the implementation of cooperative engagement strategies. Through this research, we seek to further enhance the capabilities of CEC by integrating it with autonomous drone systems, enabling a more efficient and effective response to complex missions and enhancing overall mission success.

Furthermore, this dissertation includes the work developed during the internship carried out at Mälardalen University (MDU), which focused on the search and rescue scenario with drone swarms. The project was carried out in conjunction with Embraer and ITA researchers who are working on the low-level aspects of the project (flight control methods) in the Brazilian-Swedish collaboration project entitled CTEDS - "Cooperative Threat Engagement with Drone Swarms."

Moreover, this work focuses on utilizing drone swarms for search and rescue applications. The integration of drone technology in search and rescue missions represents a groundbreaking advancement in emergency response capabilities, as it harnesses the collective power and efficiency of autonomous aerial systems. By employing drone swarms, search and rescue operations can revolutionize the way we respond to emergencies and save lives. By integrating advanced swarm intelligence algorithms to autonomous drones, this research aims to optimize search and rescue missions, enhance situational awareness, and ultimately save lives. In Figure 1.2 we describe the scenario of study where there is a swarm of drones searching for a person in distress in a forest region. To tackle this problem, we utilize an interdisciplinary approach combining robotics and AI-Based decision-making. This work aims for a future where drone swarms act as indispensable allies in crisis management, rapidly locating and aiding individuals in distress, and providing invaluable support to first responders on the ground.

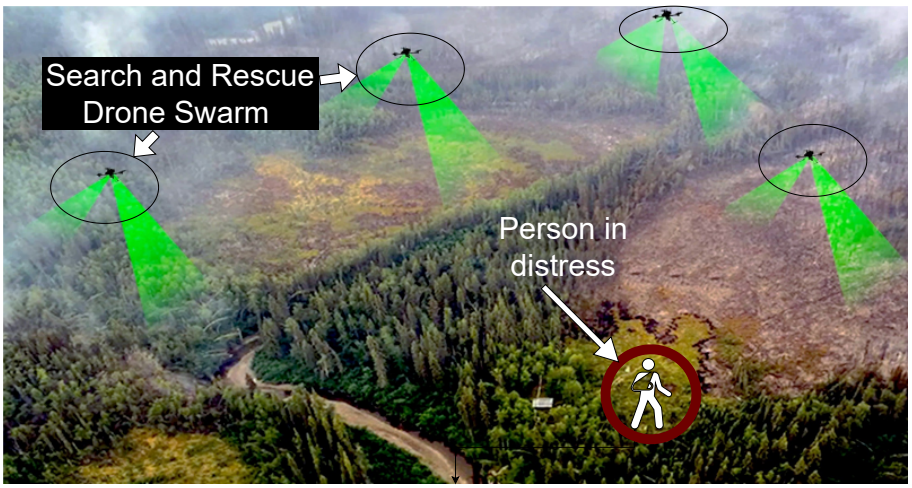

FIGURE 1.2 – Search and Rescue Scenario where a drone swarm is searching for a person in distress in a forest.

1.3 Literature Review

Swarms are a natural phenomenon observed in many animal groups, where they demonstrate remarkable coordination and cohesion through simple rules. Drone swarm intelligence is a rapidly emerging field that focuses on the development of sophisticated

algorithms and technologies to enable groups of drones to work together in a coordinated and efficient manner. Swarms of drones have a wide range of potential applications, ranging from surveillance and search-and-rescue operations to precision agriculture and military missions. The key to successful drone swarm intelligence lies in developing algorithms that can enable the drones to communicate and collaborate with one another in real-time, adapting to changing conditions and making decisions based on shared information (CHEN *et al.*, 2020). As the technology continues to evolve, the potential for drone swarm intelligence to revolutionize a wide range of industries is significant, offering new opportunities for improved efficiency, safety, and productivity. Despite the potential benefits of drone swarms, there are legitimate concerns regarding their ethical and legal implications. Privacy, security, and regulation are areas that require careful consideration before deploying drone swarms (FINN; WRIGHT, 2012; BOYLE, 2018). For instance, drone swarms could be used to conduct surveillance on individuals without their consent, potentially violating their privacy rights. Additionally, if these systems are hacked or compromised, they could pose a threat to public safety and security.

In recent years, the concept of swarms has been applied to Unmanned Aerial Vehicles (UAVs), drawing significant interest from researchers. Compared to traditional single drone applications, drone swarms can collaboratively complete complex tasks with higher efficiency and reduced cost, especially in harsh environments. Achieving the highest level of autonomy in UAV swarms is a significant challenge, with communication among the drones that make up the swarm being a crucial aspect. UAV swarm autonomy is the ability to perform a coordinated task among multiple UAVs without human intervention, making communication critical for their success. Brambilla *et al.* (2013) defined a swarm as a group of homogeneous entities working cooperatively towards a common goal, and this definition applies well to UAV swarm applications. Chen *et al.* (2020) discuss several challenges in realizing this approach, and communication is a vital aspect of UAV swarm networks (CAMPION *et al.*, 2019). Swarm cooperation is the ultimate goal for UAV swarms, and achieving it requires robust communication mechanisms and coordination protocols (HUANG; MESSINA, 2007).

The control and autonomy of a UAV swarm are crucial for the success of a mission, especially when dealing with a large number of drones. Similar to animal behaviors, a UAV swarm must cooperate to achieve a common goal. Despite the various inherent challenges, the possible applications of UAV swarms are numerous, with surveillance being a prominent one. Applications that use cameras or other types of remote sensing equipment are common, including detection and identification of anomalies (DANTAS *et al.*, 2022), traffic monitoring (BISIO *et al.*, 2022), and search and rescue operations (SKOROBOGATOV *et al.*, 2020). Another important application of UAV swarms is the search for objects of interest, which is also addressed in this work.

1.3.1 Drone Swarms Challenges

Controlling and managing UAV swarms entail numerous challenges that need to be resolved for their reliable and effective operation (MICHAEL *et al.*, 2016). One of the most significant challenge in controlling and managing UAV swarms is communication. To achieve a common goal, reliable communication among the drones is vital to share information and coordinate their actions (HONG; KIM, 2012). Coordinating the movements of multiple drones in a swarm is another complex task, especially in dynamic and uncertain environments where synchronization is crucial to avoid collisions and maximize effectiveness (GROCHOLSKY *et al.*, 2006).

UAV swarms must navigate effectively in challenging environments where GPS signals may be disrupted or unavailable, such as urban areas or disaster zones. To enable accurate and reliable navigation, sophisticated sensors and algorithms are required (NEIRA; TARDOS, 2001). Achieving high levels of autonomy is also a significant challenge, as UAV swarms need to adapt to changing conditions and make decisions based on the data collected from various sensors without human intervention (BALDWIN *et al.*, 2018). Including decisions related to flight paths, landing, and avoiding obstacles.

Power management is essential for the longevity and effectiveness of UAV swarms since they require significant power to operate. Optimizing power usage and recharging the drones' batteries in the field are key challenges (ROBINSON *et al.*, 2006). Additionally, as UAV swarms become more prevalent, the risk of cyber-attacks and other forms of interference increases. Ensuring the security and integrity of the swarm's communication and control systems is a critical challenge (MICHAEL *et al.*, 2016).

Airspace management faces significant challenges with the growing use of drones in various applications (MERKERT; BUSHELL, 2020). The safe and efficient integration of drones into traditional airspace is crucial to avoid collisions and ensure the safety of all parties involved. Aviation authorities and regulators are developing specific regulations for drone use, establishing altitude restrictions, no-fly zones, and communication requirements. Additionally, advanced air traffic management systems are being adapted to accommodate both manned and unmanned aircraft, enabling harmonious coexistence in the airspace. As drone technology continues to evolve and its usage becomes more widespread, airspace management will play a crucial role in ensuring safety and optimizing the efficiency of this integration.

These challenges are just a few of the many factors that must be considered when controlling and managing UAV swarms. Addressing these challenges necessitates innovative solutions and ongoing research and development efforts (PURSEL *et al.*, 2018).

1.3.2 Benefits and Limitations of drone swarms applications

UAV swarms offer a number of benefits and limitations when used in a wide variety of applications. One benefit of using a swarm of UAVs is increased efficiency as they can perform tasks more quickly and efficiently than single drones or human operators (CHEN *et al.*, 2020). Additionally, using a swarm of small UAVs can be more cost-effective than deploying a single large UAV or a team of human operators (HUTCHINGS *et al.*, 2019). UAV swarms can also be used in hazardous environments where human intervention may be dangerous (ZHAO *et al.*, 2021). Another benefit is the ability of UAV swarms to cover larger areas and gather more data than a single drone or human operator (HUTCHINGS *et al.*, 2019). UAV swarms are also flexible and can adapt to changing environments, making them suitable for a wide range of applications (ZHAO *et al.*, 2021).

However, there are also several limitations associated with the use of UAV swarms. Communication issues, such as signal interference and bandwidth limitations, can affect the coordination and effectiveness of UAV swarms (CHEN *et al.*, 2020). Smaller UAVs used in a swarm often have limited payload capabilities, which can restrict the types of tasks they can perform (ZHAO *et al.*, 2021). The limited battery life of small UAVs can also restrict their operational time, as they may need to return to the base station for recharging (HUTCHINGS *et al.*, 2019). Additionally, controlling and managing a swarm of UAVs requires sophisticated algorithms and software, making it a complex task that may require specialized expertise (CHEN *et al.*, 2020). Finally, the use of UAV swarms is subject to various legal and regulatory restrictions that can limit their deployment in certain situations or locations (ZHAO *et al.*, 2021).

It is important to consider these benefits and limitations when deciding whether to use UAV swarms for specific applications. While UAV swarms have the potential to provide significant benefits, the limitations should not be overlooked, and they should be addressed to maximize the effectiveness of the swarm.

1.3.3 Loyal Wingman Application

The cooperative engagement capability (CEC) is an emerging systems-of-systems capability in which multiple systems coordinate their actions through a network to improve their abilities to effectively perform a given task (APL-TEAM, 1995). The term was coined in the 90s in the Johns Hopkins University Applied Physics Laboratory in an effort to enhance the United States Navy's defense capability. The project aimed to improve the situational awareness of a fleet by sharing tactical information among data link participants and speeding up the decision process regarding the interception, engaging, and neutralization of a flying threat, *e.g.*, a cruising missile or aircraft. With the recent ad-

vances in aerial robotics, the CEC concept can be extended to unmanned flying platforms to promote group behavior towards the achievement of a common purpose via a shared communication network. In this context, drone swarms are of particular interest since they enable the use of several low-cost and even disposable platforms in modern defense or surveillance applications.

Situational awareness in aircraft systems is paramount for safe and efficient flight operations (ENDSLEY; CONNORS, 2008). It refers to a pilot's ability to gather, process, and comprehend information about the aircraft's environment, performance, and current state. This includes monitoring critical parameters such as altitude, airspeed, engine performance, weather conditions, and traffic proximity. Pilots rely on advanced avionics systems, including radar, GPS, and data-link technologies, to maintain an accurate and real-time understanding of their surroundings. Maintaining situational awareness allows pilots to make informed decisions, anticipate potential hazards, and react promptly to changing circumstances, ensuring the safety of passengers, crew, and the aircraft itself throughout the flight.

In particular, manned-unmanned teaming (MUM-T) is an essential component of the future unmanned aircraft systems' operational environment. It extends the CEC concept by promoting the seamless integration of unmanned platforms, endowed with some level of autonomy, with manned ones. The unmanned platforms aim to perform given tasks under the command of the manned ones. This integration combines the inherent strengths of both platforms to achieve synergy not seen in single-agent systems, thus enabling a higher degree of effectiveness for the team of networked agents. In this context, the Defense Advanced Research Projects Agency's (DARPA) Air Combat Evolution (ACE) program [1] aims to revolutionize air-to-air combat by developing and advancing autonomous systems. With the ultimate objective of conducting a full-scale flight exercise involving actual aircraft, ACE strives to push the boundaries of technological innovation for a new era of aerial warfare that combines human expertise with the precision and efficiency of autonomous systems.

Specifically, the MUM-T context includes the concept of the Loyal Wingman (LW), which is an unmanned aerial vehicle (UAV) under the tactical command of a high-level manned or remotely controlled leader (HUMPHREYS *et al.*, 2015). Recently, we have witnessed a rapid emergence of multirotor aerial vehicles (MAVs) (SANTOSO *et al.*, 2016) in either research and industrial applications where aircraft size, weight, and cost play a critical role in their operation, effectiveness, or feasibility (KUMAR; MICHAEL, 2012). Therefore, we argue that low-cost experimental MAVs are a suitable alternative to investigate the loyal wingman concept in MUM-T scenarios.

[1] https://www.darpa.mil/program/air-combat-evolution

The Loyal Wingman drones are rapidly gaining attention in the military and defense industries (HOWARD, 2020). Boeing is currently developing a Loyal Wingman named MQ-28 (BOEING, Accessed: 2023), which is an unmanned combat aerial vehicle (UCAV) designed to operate alongside manned aircraft, providing support and expanding the capabilities of manned platforms. These drones are designed to fly alongside manned aircraft and operate in coordination with them, providing a range of capabilities such as reconnaissance, surveillance, target acquisition, and electronic warfare (TUCKER, 2020). Unlike traditional UAVs that are controlled remotely by a human operator, loyal wingman drones are capable of autonomous decision-making and can operate independently or in collaboration with other drones in a swarm (EXCELLENCE, 2021). The development of loyal wingman drones is seen as a major breakthrough in the evolution of airpower, providing military forces with increased flexibility, agility, and strategic advantage on the battlefield (TUCKER, 2020).

In the context of UCAVs, Fu *et al.* (2014) explores the potential of intelligent technologies for air-combat UAVs, highlighting research opportunities. It addresses challenges in decision-making methods and identifies trends in this field. The study examines two primary control modes: ground control by human experts and AI-based control. Furthermore, the authors reference the lack of autonomy as the biggest obstacle for UAV to perform combat tasks.

Focusing on AI-based control for combat UAVs, the work by Pope *et al.* (2021) showcased the use of AI to autonomously pilot an F-16 fighter jet in simulated air-to-air combat. They employed a hierarchical deep reinforcement learning (RL) approach to train an agent capable of one-vs-one within visual range air combat, also known as a dogfight, in a high-fidelity simulation environment. Their solution outperformed human expert pilots, securing a second-place rank in the DARPA AlphaDogfight Trials. Similarly, the paper by Yang *et al.* (2020) proposes an autonomous maneuver decision model for UAVs in short-range air combat using RL. Simulation results demonstrate the effectiveness of the model based on deep Q network (DQN) in achieving autonomous decision-making for UAVs in air combat.

Furthermore, Xing *et al.* (2019) presents a study on swarm versus swarm UAV combat, proposing a self-organized offense-defense decision-making algorithm, utilizing a distributed architecture, treating each UAV as an agent that exchanges information with neighbors to solve local decision problems. The decision-making process involves target allocation using a distributed consensus-based auction algorithm and swarm motion decision based on social forces. Simulation experiments validate the algorithm's effectiveness.

In the context of AI-based automomous UCAVs, this research aims to explore the essential attributes that define the decision-making capabilities of LW drones, with the objective of paving the way for their future deployment in diverse military operations

(HOWARD, 2020; EXCELLENCE, 2021). Through an analysis of their characteristics in a gamified defense scenario, this study aims to make a significant contribution to the their practical utilization. To achieve this, the research will investigate the application of LW drones by simulating the dynamics of two adversarial drone swarms within a virtual environment. By undertaking this investigation, valuable insights can be gained to further advance the potential applications of LW drones in real-world scenarios.

1.3.4 Search and Rescue Application

Drone swarms are gaining popularity in various industries, including search and rescue operations (ZHAO *et al.*, 2021). Drone swarms consist of multiple drones working together in a coordinated and collaborative manner to achieve a specific objective (CHEN *et al.*, 2020). In search and rescue operations, drone swarms can be used to cover large areas quickly and efficiently, allowing rescue teams to locate and aid those in need of assistance (HUTCHINGS *et al.*, 2019). The use of drone swarms in search and rescue operations can significantly reduce the time and cost involved in traditional search and rescue missions, while also improving the safety of rescue teams (ZHAO *et al.*, 2021). However, the deployment of drone swarms in search and rescue operations is not without its challenges, including regulatory hurdles, technical limitations, and ethical concerns (ZHAO *et al.*, 2021). This work will examine the use of drone swarms in search and rescue applications, exploring the benefits and challenges of their use and offering a new approach for the usage of decision making techniques for drone swarms in this critical field.

Given a large number of drones, the control and autonomy of the UAV swarm are crucial for the success of a mission, i.e., similar to the animal behaviors, a UAV swarm has to cooperate to achieve a goal. Despite the various inherent challenges, the possible applications of a UAV swarm are numerous. Applications that use cameras or other types of remote sensing equipment are the most prominent, including detection and identification of anomalies for surveillance (DANTAS *et al.*, 2022), traffic monitoring, and search and rescue (SKOROBOGATOV *et al.*, 2020). Another important application is the search for objects of interest, which we address in this work.

Mission planning on an online level requires continuous update when conditions in the environment occur. Planning is one of the biggest challenges in artificial intelligence (AI) and building intelligent systems able to plan effectively often requires a hybrid approach. There is no single algorithm that can perform perfect planning and the challenge is to develop planning approaches that are as efficient as possible for a specific task. As humans we may first make a coarse grain plan that we start with, even if we do not have a full picture of the task and challenges ahead. On the top level the task for the coarse-grained plan may have conflicting goals, e.g., to minimise the risk of problems, find the target

as fast as possible and use as little resources as possible. We humans perform online planning by reevaluating and adjusting our plans as the situation changes and we collect more information and may even have backup plans if the main plan does not work as expected.

Search patterns utilizing a UAV swarm often rely on heuristic approaches due to the unattainability of an optimal solution (AUNON P. CRUZ, 2018). In this study, we present a simulator that enables the implementation and testing of the series of proposed search heuristics. The dynamics of the autonomous UAVs are simplified and controlled through a P+V control law, which guides the vehicles' movements. Additionally, a potential field model is employed to prevent collisions, and a cooperative and autonomous decision-making framework is developed for the drones.

In this context, we propose the utilization of UAV swarms for search and rescue missions aimed at locating and rescuing distressed individuals. Our work makes a significant contribution by introducing a novel approach that enables the effective use of drone swarms in such missions, particularly when utilizing a discrete map of the target area. Additionally, we explore a variant of the map that incorporates probabilistic information provided beforehand about the region. To achieve this objective, we employ a set of heuristics inspired by the research conducted by Sundelius N. (2023). Furthermore, we develop sophisticated simulations capable of emulating search missions involving drones in both 2D and 3D virtual environments. These simulations serve as valuable tools for evaluating and refining our approach.

Overall, our research endeavors to enhance the efficacy and efficiency of search and rescue operations by leveraging the collective capabilities of UAV swarms and incorporating innovative strategies based on heuristic principles. This approach involves the development of a novel swarm intelligence model, which also provides a heuristic to avoid the frequent problem of local minima in drone swarm applications that use the potential fields as a method to avoid collisions. Our proposed method is based on distributed decision-making to provide autonomy to UAVs, combined with the development of a collective intelligence module for the coordination of the swarm, with a shared discrete map search strategy for the search mission of a missing target (e.g., an individual). Please note that in this application, direct combat engagement with other drones is explicitly excluded. When a person is identified, the patrolling drones will immediately alert the commander of the mission. The commander, in turn, will promptly assess the situation and take appropriate measures to successfully conclude the mission.

1.4 Contributions

First of all, it is worth mentioning that this text is based on the following published papers:

1. Ricardo Jr., J. A.; **Giacomossi, L.**; João F. S. Trentin; Brancalion, J. FB; Maximo, M.R.O.A ; and Santos, D. A., "Cooperative Threat Engagement with Heterogeneous Drone Swarms." **IEEE Access**, 2023.

2. **Giacomossi, L.**; Sundelius, N. ; Brancalion, J. FB; Maximo, M.R.O.A ; Funk, P., and Sohlberg, R., "Cooperative Search and Rescue with Drone Swarm" **IAI2023 - 7th International Congress and Workshop on Industrial AI and eMaintenance**, Luleå, Sweden, 2023.

3. **Giacomossi, L.**; Ricardo Jr, J. A. ; Brancalion, J. FB; Maximo, M.R.O.A ; and Santos, D. A., "Cooperative Threat Engagement with Heterogeneous Drone Swarms." **33th Congress of the International Council of the Aeronautical Sciences (ICAS)**, Stockholm, Sweden, 2022.

4. **Giacomossi, L.**; Schwanz Dias, S; Brancalion , J.F. and Maximo, M.R.O.A., "Cooperative and Decentralized Decision-Making for Loyal Wingman UAVs," **2021 IEEE Latin American Robotics Symposium (LARS)**, 2021.

5. **Giacomossi, L.**; F. Souza; R. Gomes Cortes; H. M. Montecinos Cortez; C. Ferreira; C. A. C. Marcondes; D. S. Loubach; E. F. Sbruzzi; F. A. N. Verri; J. C. Marques; L. A. Pereira; M. R. O. A. Maximo; V. V. Curtis, "Autonomous and Collective Intelligence for UAV Swarm in Target Search Scenario," **2021 IEEE Latin American Robotics Symposium (LARS)** 2021.

During the development of the project present in this dissertation, the following papers related to drone applications were also produced by the author:

1. L. F. Pimentel; **L. Giacomossi Jr.**; J. C. De Matos; V. V. Curtis and F. A. N. Verri, "Detecção de Drones Intrusos" Translation: "Detection of Intruder Drones" . **SIGE XXIII - Simpósio de Aplicações Operacionais em Áreas de Defesa**, São José dos Campos, ITA - 2021.

2. **L. Giacomossi**, A. Caregnato-Neto and M. R. O. A. Maximo, "Optimization of Force and Torque Bounds for the Flight Control System of a Quadcopter using PSO," **2022 IEEE Latin American Robotics Symposium (LARS)**, Nov 2022.

To the best of our knowledge, the main contributions of this work are:

1) Introduces a novel defense scenario and simulation environment utilizing Loyal-wingman UAVs. This concept was initially proposed in the two-dimensional prototype described in Giacomossi *et al.* (2021a) and further extended in Giacomossi *et al.* (2022b), Ricardo *et al.* (2023), where a three-dimensional environment with realistic flight dynamics is taken into account, providing a more realistic and immersive simulation platform.

2) The research models an AI-based decision-making for the Kamikaze UAVs present in the defense scenario. Specifically, a simple yet effective approach is employed to make them autonomous was developed in Giacomossi *et al.* (2021a) and extended in Giacomossi *et al.* (2022b), Ricardo *et al.* (2023), utilizing Finite State Machines (FSMs) to capture the decision-making process of these UAVs.

3) The work addresses the challenge of distributed and cooperative AI-based decision-making. By employing the behavior tree technique, the research successfully models the decision-making process for the loyal wingman UAVs, developed in the prototype (GIACOMOSSI *et al.*, 2021a) and extended in Giacomossi *et al.* (2022b), Ricardo *et al.* (2023). This approach promotes collaborative decision-making among the UAVs, enhancing their collective intelligence and overall system performance.

4) The study suggests a model of software architecture required for implementing these agents in a simulated environment. The proposed architecture outlines the necessary components and structures essential for these agents.

5) A strategy of cooperation is proposed, which utilizes task allocation to effectively distribute tasks among the UAVs (RICARDO *et al.*, 2023). This strategy plays a crucial role in neutralizing kamikaze threats, as it optimizes resource allocation and facilitates efficient coordination among the UAVs.

6) The research introduces a simple and effective uniform-circular formation for the manned-unmanned team. This formation enhances the team's maneuverability and coordination, resulting in improved performance and mission success.

Regarding the Search and Rescue (SAR) application, the study offers significant contributions, including:

1) The proposed work presents a novel search and rescue scenario and simulation environment utilizing a swarm of UAVs, as outlined in Giacomossi *et al.* (2021b). This concept has been further extended and refined in subsequent studies such as Giacomossi *et al.* (2023) and Sundelius N. (2023). The research specifically focuses on a two-dimensional environment that has been purposefully developed for this project. This unique environment allows for comprehensive exploration and testing of the proposed swarm-based search and rescue system.

2) Modeling of AI-based decision-making using Finite State Machines (FSMs) for the search UAVs. The AI component is divided into two groups (GIACOMOSSI *et al.*, 2021b): autonomous and collective. The autonomous AI pertains to individual UAVs, enabling them to make independent decisions based on their observations and environmental cues. On the other hand, the collective AI focuses on the coordination and collaboration of the search swarm as a whole. This division allows efficient search strategies, maximizing the overall search and rescue mission success.

3) The development of an open-source swarm simulator (GIACOMOSSI *et al.*, 2021b) and adapted in Giacomossi *et al.* (2023). This simulator is designed to be adaptable for various scenarios, accommodating any number of drones and obstacles. Key features of the simulator include a drone model, obstacle representation, collision avoidance mechanisms, and a discretized map that can be randomly generated. By providing an open-source solution, this simulator not only enables researchers and practitioners to replicate and validate the proposed swarm-based search and rescue system.

4) The simulation environment was purposefully designed for streamlined and rapid prototyping, primarily in response to the project's computational constraints. Consequently, it was crafted to simplify the development of decision-making solutions for search and rescue scenarios, particularly when dealing with hardware limitations. Our environment is significantly more computationally efficient than many popular drone simulation frameworks.

5) The study in Giacomossi *et al.* (2023) presents a novel approach by incorporating a probability map that assigns probabilities to each region's cells, indicating the likelihood of finding a missing person. To enhance the search efficiency, a comprehensive set of heuristics was developed, combining the Bayesian search algorithm, Hill Climbing algorithm, and the probability map of the discretized region. By combining the strengths of these techniques, the study aims to optimize search efforts and increase the chances of successfully locating missing individuals.

1.5 Text Organization

This document is organized as follows:

- The Chapter 2 provides the theoretical background related to decision-making and the main techniques used in this work. In this chapter, Section 2.1 introduces the context of decision-making in robotic systems, followed by Section 2.2 which delves into the selected artificial intelligence techniques for decision-making. Section 2.3 introduces the important concept of cooperation in robotics. Additionally, Section

2.4 covers the background of optimization algorithms, while Section 2.5 presents the selected search algorithms used in this work, including the Lawn Mower algorithm, Hill Climbing, and Bayesian Search. In Section 2.6, we discuss the concept of collision avoidance and the main techniques selected for this work. Finally, Section 2.8 presents the main techniques utilized for the development of the simulators.

- In Chapter 3, we address the defense application using the Loyal Wingman UAVs. First, in Section 3.1, a scenario of interest is developed to introduce the study of the loyal wingman drones, where we apply gamification using two adversary teams of such application. Next, in Section 3.2, we break down the problem and divide it into two main modules, a high-level decision module and a low-level control module. Later, in Section 3.3, we formalize the high-level problems to be tackled in this project. We proceed to Section 3.4, where we present the development of the decision-making for both the loyal wingman and the kamikaze drones. Also, in this section, we discuss the approach used to develop the cooperation of the team and the formation definition of the manned-unmanned team. Now, in Section 3.5, we present the two simulators that were entirely developed for this work. The first is a 2D prototype, and the second is a high-fidelity 3D simulator. Next, in Section 3.6, the experiments, results, and discussion using both simulators are presented.

- In Chapter 4, we present and introduce the search and rescue application utilizing drone swarms. In Section 4.1, we develop a search and rescue scenario of interest and present its main entities. Next, in Section 4.2, we analyze the problem and divide it into two main strategies. Section 4.2.1 specifies the main problem to be solved in this application. Moving on to Section 4.3, we develop the solution for the defined problem and present the main heuristics utilized for the search process. Section 4.3.1 covers the decision-making for the autonomous drones and the swarm, explaining how it works. Later, in Section 4.5, we present the experiments conducted for the search application and their results. The experiments are divided into two parts: the first experiment analyzes the developed decision-making, while the second experiment focuses on the high-level aspect of the search. In the second experiment, a probability map is used to guide the swarm, improving the overall efficiency of the search.

- Lastly, Chapter 5 serves as the concluding section of this work, summarizing the main findings and proposing future research ideas for both practical applications.

2 Background

In robotics, behavior refers to the actions or responses of a robot in a specific situation or environment (ARKIN *et al.*, 1998). Behaviors can be pre-programmed (BALDAO *et al.*, 2020; COLLEDANCHISE; OGREN, 2018) or learned through machine-learning algorithms. The learning techniques are able to learn the individual behaviors, sometimes outperforming a handcoded program (MAHADEVAN; CONNELL, 1992).

Robotic behavior can range from simple actions like moving forward, turning, and stopping, to more complex behaviors like recognizing objects, avoiding obstacles, and performing tasks based on sensory input (CHOSET *et al.*, 2005). These behaviors are typically implemented using a behavior-based control architecture (ARKIN *et al.*, 1998), which is composed of a set of simple, reactive behaviors that interact with each other to produce the desired overall behavior (COLLEDANCHISE; OGREN, 2018).

UAVs are commonly referred to as drones, and are a type of aerial robot that have gained popularity in recent years. Drones are typically equipped with cameras, sensors, and other devices that allow them to perform a variety of tasks, such as surveillance, inspection, and delivery. Drones also use behavior-based control architectures to perform these tasks (GARCIA-AUNON *et al.*, 2019).

In order to achieve effective and efficient behavior, UAVs use sensors such as GPS, accelerometers, and gyroscopes to detect their position and orientation, as well as to identify obstacles in their environment. The UAV's model then determine the appropriate action to take based on the situation. For example, an UAV may use its sensors to detect an obstacle in its flight path and then use a programmed behavior to adjust its flight path to avoid the obstacle (RICARDO; SANTOS, 2023b).

Robot behavior can be modified or adapted to different situations, allowing UAVs to perform a wide range of tasks in various environments. This flexibility is a key advantage of robotics, as it allows UAVs to adapt to changing circumstances and perform tasks that would be difficult or impossible for humans to accomplish.

2.1 Decision-making in Robotics

In robotics, decision-making refers to the process by which a robot determines the appropriate action to take based on the information it has received from its sensors and its algorithms (SIEGWART *et al.*, 2011a). The decision-making process typically involves several steps (CHOSET *et al.*, 2005), including:

1. Perception: The robot's sensors detect and collect information about the environment and its surroundings, as seen in the work by Dantas *et al.* (2022), including the presence of obstacles, the location of objects, and the position and orientation of the robot itself.

2. Processing: The robot's programming or learning algorithms process the information collected by the sensors and use it to generate a representation of the environment and its current state. For instance, in the work of Garulli *et al.* (2005), the robot uses SLAM (simultaneous localization and mapping) to create a map of the environment from laser data, and simultaneously estimate its position and orientation within the map.

3. Planning: Based on the representation of the environment, the robot's programming or learning algorithms generate a plan of action, which specifies the sequence of movements and actions that the robot should take to achieve its goal. A comprehensive survey on these methods is described in the work by Gasparetto *et al.* (2015).

4. Execution: The robot carries out the plan of action, using its actuators to move and interact with the environment.

5. Monitoring: The robot continuously monitors its environment and its own actions, and adjusts its behavior if necessary based on new information or changes in the environment.

The decision-making process in robotics can be pre-programmed, where a set of rules and decision trees are defined by the programmer (GOVERS, 2018), or it can be learned through machine learning algorithms (MELO *et al.*, 2021). In the latter case, the robot learns to make decisions based on a training set of data, which allows it to adapt to new situations and make better decisions over time. Furthermore, Reinforcement Learning (RL) serves as a crucial catalyst in advancing the autonomy and adaptability of robotic systems, as demonstrated in the research conducted by Muzio *et al.* (2022). Their work showcases the successful application of RL methods to a Humanoid Robot, enabling the agent to autonomously learn complex behaviors.

Effective decision-making is crucial for robots to perform tasks efficiently and safely, particularly in dynamic and unpredictable environments. The ability to make decisions quickly and accurately is also important for robots that are used in applications such as autonomous vehicles, where split-second decisions can have serious consequences (HUSSAIN; ZEADALLY, 2019).

Behavior and decision-making are two important concepts in robotics, but they refer to different aspects of a robot's functionality. Behavior in robotics refers to the collection of actions and movements that a robot can perform in response to different stimuli from the environment or from its programming. It is often pre-programmed into the robot by the developer, and can include simple actions such as obstacle avoidance, as well as more complex actions such as grasping objects or navigating through a complex environment. The term decision-making, on the other hand, refers to the process by which a robot selects a course of action based on its understanding of the environment and its objectives. This process involves analyzing sensor data, generating a plan of action, and executing that plan, as well as monitoring the environment and adjusting the plan as needed.

While behavior and decision-making are closely related, there are some differences. Behavior is typically more reactive and automatic, and is often used for basic, low-level tasks such as movement or sensing. Decision-making, on the other hand, is more deliberate and strategic, and is used to guide the overall behavior of the robot as it carries out more complex tasks. In general, behavior is more focused on the immediate actions of the robot, while decision-making is more focused on the robot's overall strategy and goals.

2.2 AI Decision-making Techniques

In this section the main techniques for the development of the decision-making capabilities for the UAVs in our context are discussed. Many methods are used to control the behaviors of autonomous agents, with finite-state machines (FSMs) (WAGNER *et al.*, 2006) and behavior trees (BTs) (COLLEDANCHISE; OGREN, 2018) being the most popular techniques.

Numerous approaches have been developed for robotic decision-making in various applications such as autonomous vehicles, where agents require high levels of deliberation (PADEN *et al.*, 2016; SIEGWART *et al.*, 2011b). In complex scenarios, decision-making is often broken down into behaviors, which are modular solutions tailored to address specific sub-tasks (ARKIN *et al.*, 1998). For instance, in a Combat UAV agent designed by Ögren (OGREN, 2012), typical behaviors include: `EvasiveManeuvre`, `DoCombat`, `disEngage`, `FlyHome`, and `Strike`. While some researchers initially recommended more reactive architectures for behavior selection (SIEGWART *et al.*, 2011b; ARKIN, 1998), behavior se-

lection mechanisms utilizing Finite State Machines (FSMs) or Behavior Trees (BTs) are usually designed by humans, with empirical design based on the developer's experience, creativity, and good practices (MILLINGTON, 2019). Furthermore, the process is iterative, with expert evaluation or statistical analysis used to select the most efficient agent configuration for the context (COLLEDANCHISE; OGREN, 2018). Practically, behaviors are typically implemented using hand-coded heuristic rules (BALDAO *et al.*, 2020), with developers relying on formal AI techniques such as search (AKIYAMA *et al.*, 2012), path planning (PINHEIRO *et al.*, 2020), optimization (MACALPINE; STONE, 2018), and machine learning (SCHWAB *et al.*, 2018; MEDEIROS *et al.*, 2020; MELO *et al.*, 2019) to partially or completely develop behaviors (BALDAO *et al.*, 2020).

2.2.1 Finite State-Machines for Decision-Making

Finite State Machines (FSMs) are widely employed in modeling decision-making architectures within simple artificial intelligence (AI) systems. These systems are characterized by the ability to exist in only a finite number of states simultaneously (COLLEDANCHISE; OGREN, 2018). In this technique, each state represents a specific behavior for the agent, and the FSM enables the system to switch between these behaviors in response to various events. The AI developer assumes the responsibility of encoding the behaviors and defining the conditions that prompt transitions between them. A visual representation of a basic FSM is provided in Figure 2.1 to illustrate this concept. This approach involves maintaining a state until a specific event triggers a transition, causing the system to transition into the corresponding behavior associated with that event.

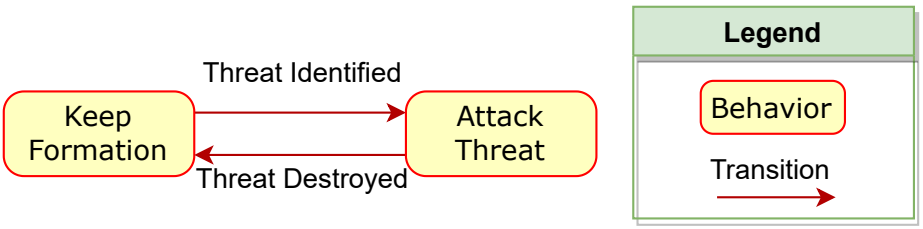

FIGURE 2.1 – Example of a Finite State Machine (FSM).

In the example depicted in Figure 2.1, we can observe a finite state machine (FSM) representing the behavior of a combat aircraft. Initially, the aircraft is in the state of executing the `KeepFormation` behavior when no transitions are activated. However, once a threat is detected and a transition is triggered, the FSM promptly switches to the `AttackThreat` behavior in response to the event. This behavior persists until another transition is triggered, indicating the successful elimination of the threat. Consequently, the aircraft reverts back to its previous state, resuming the execution of the `KeepFormation` behavior.

FSMs are widely employed to the development of AI agents, due to its intuitive struc-

ture to a developer and its inherent simple implementation in any programming language. However, the implementation of FSMs shows some disadvantages (COLLEDANCHISE; OGREN, 2018) such as:

- *Maintainability*: Adding or removing behaviors may require to re-evaluate all the transitions and behaviors of the whole FSM, making FSMs highly prone to design errors and hard maintainability.
- *Scalability*: FSMs with many behavior and transitions are hard to modify and are not so flexible to additions of new dependencies between behaviors.
- *Reusability*: The transitions between behaviors may depend on internal variables, making it unpractical to reuse the behavior in other agents.

2.2.2 Behavior Trees

Behavior Trees (BTs) are one of the most advanced decision-making techniques used currently in the video game industry (IOVINO *et al.*, 2022a; MARCOTTE; HAMILTON, 2017). The philosophy of BTs is to encode behaviors that are highly *modular, reactive, and flexible* to changes. By modularity it is understood that behaviors may be separated and recombined, often with the benefit of flexibility, independence and variety of use. Reactivity means the ability to react to external events. Since most of the problems encountered in the FSM are easily treated by BTs (COLLEDANCHISE; OGREN, 2018), the method has been gaining popularity as an effective method for controlling robots (IOVINO *et al.*, 2022b), where agents have become complex with several behaviors. Therefore, BTs are better in terms of *modularity, scalability, and reusability* in comparison to FSMs. On the other hand, they are harder for a human to interpret and to implement.

A basic BT framework is presented, but keep in mind that actual implementations commonly employ extensions. A BT is composed by nodes, which can be of two types: *composite or leaf*. *Composite* nodes controls the BT logic, while *leaf* nodes actually execute the behaviors or check conditions. When executed, each node returns one of the following execution status: *Success, Failure, or Running*. The *Running* status means the node did not complete executing and the BT must resume running from this node in the next time step. Figure 2.2 illustrates a simple example on how these building blocks may be used.

A composite node is a type of node in a behavior tree that has one or more child nodes. The purpose of a composite node is to combine the behavior of its child nodes in a manner to create more complex behavior for the overall behavior tree. Composite nodes are useful for organizing and structuring the behavior and creating more complex

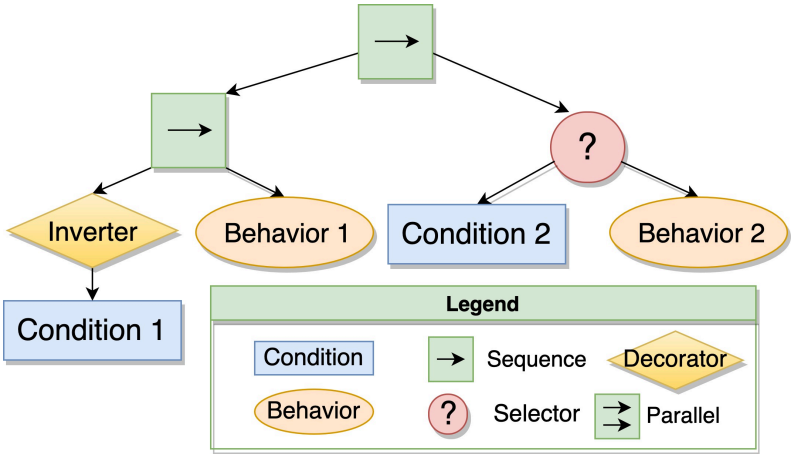

FIGURE 2.2 – Example of generic Behavior-Tree (BT).

behavior from simpler building blocks. By combining different types of composite nodes, along with action and condition nodes, a wide variety of behaviors and decision-making strategies can be created for an agent or robot. The types of composite nodes are dictated by the framework (MARZINOTTO *et al.*, 2014; SILVA *et al.*, 2021). The most common types of nodes are seen in Table 2.1: *sequence, selector, parallel, and decorator*. *Sequence* nodes sequentially execute all its children, following the order from the left child to the right child, as long as they are successful. A selector node is used when any child can perform the task, and will select the first child that is successful, also checking from the left child to the right child. A parallel node executes its children in parallel *i.e* at the same time. The return status of a *parallel* node may be further defined by the framework policy. Table 2.1 presents the return status logic of each node type. Notice that a parallel node may interrupt a running children if the parallel's node success or failure is already defined. A *decorator* node changes the execution status of its child. For example, an *Inverter* decorator inverts the status of its child (Success to Failure and vice-versa), whereas an *AlwaysSucceed* decorator returns Success regardless if the child was successful or not.

TABLE 2.1 – Node types and return logic.

Node type	**Success**	**Failure**	**Running**
Selector	If one child succeeds	If all children fail	If one child returns running
Sequence	If all children succeeds	If one child fails	If one child returns running
Parallel	If N children succeeds	If M-N children fail	If all children return running
Action	Upon completion	When impossible to complete	During completion
Condition	If `true`	If `false`	Never
Inverter (Decorator)	If `Failure`	If `Success`	-

The leaf nodes are the agent behaviors or conditional checks. A *conditional* leaf node checks if a condition holds, then return *Success* or *Failure* depending if the condition is

`true` or `false`, respectively, without the execution of an action. The *Action* node leaf is the behavior *per se*, *i.e*, an action of the agent. In Figure 2.3 a BT composed of two behaviors for a combat aircraft is presented.

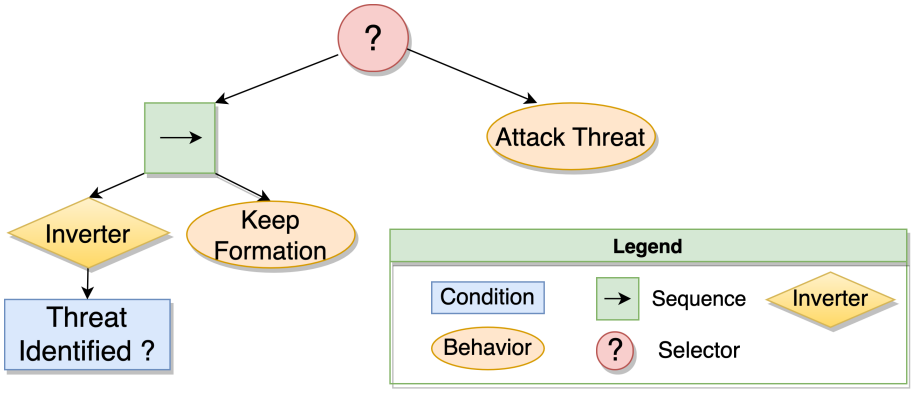

FIGURE 2.3 – Example of a Behavior tree (BT).

In this example we have a simple behavior tree for a combat aircraft, with behaviors of maintaining formation and attack threat. Once a threat is identified, the aircraft's decision making must change its behavior to attack the threat and therefore neutralize the identified threat. A behavior such as `KeepFormation` may be executed for many time steps, so the corresponding leaf node will keep returning *Running* while the behavior is executing. If a threat is identified, the conditional node returns `True`, and therefore the *Inverter* node changes the return to `False`, which causes the *Selector* node to change the running behavior of the aircraft to the `AttackThreat` behavior.

2.2.2.1 Pseudo-code for a Behavior Tree

In this subsection, I present the pseudocode for the implementation of the behavior tree algorithm. Note that this implementation used the LaTeX's pseudo-code formatting, and that different approaches to the implementation of the BT framework can be taken.

The Algorithm 1 defines the `BehaviorTree` class that takes the `root_node` as a parameter during initialization. The `run()` method executes the behavior tree by calling the `execute()` method of the `root_node` and repeatedly calling it until the status is no longer "RUNNING". If the status is "SUCCESS", it prints "Behavior tree succeeded!" to the console, otherwise it prints "Behavior tree failed." to the console.

Algorithm 1 BehaviorTree Class

```
1: procedure BehaviorTree(root_node)
2:     self.root_node ← root_node
3: end procedure
4: procedure run
5:     status ← self.root_node.execute()
6:     while status is "RUNNING" do
7:         status ← self.root_node.execute()
8:     end while
9:     if status is "SUCCESS" then
10:        print("Behavior tree succeeded!")
11:    else
12:        print("Behavior tree failed.")
13:    end if
14: end procedure
```

Now, the Algorithm 2 is an implementation of an `ActionNode` class. As mentioned, an action node is a type of node in a behavior tree that performs an action when executed. In this pseudocode, the `ActionNode` class has a single method called `execute()`, performs the action and provides a status indicating the outcome of the action, whether it was successful, unsuccessful, or still in progress. Note that the function `perform_action()` must be implemented by the designer of the agent.

Algorithm 2 ActionNode Class

```
1: procedure execute
2:     action_result ← perform_action()
3:     if action_result is True then
4:         return "SUCCESS"
5:     else if action_result is False then
6:         return "FAILURE"
7:     else
8:         return "RUNNING"
9:     end if
10: end procedure
```

Similarly, the Algorithm 3 implements a `ConditionNode` class. A condition node is a type of node in a behavior tree that evaluates a condition when executed. In this pseudocode, the `ConditionNode` class has a single method called `evaluate()`, which evaluates the condition and returns a corresponding status whether the condition was `true` or `false`.

Algorithm 3 ConditionNode Class

```
1: procedure EVALUATE
2:     condition_result ← evaluate_condition()
3:     if condition_result then
4:         return "SUCCESS"
5:     else
6:         return "FAILURE"
7:     end if
8: end procedure
```

Algorithm 4 CompositeNode Class

```
1: procedure CompositeNode
2:     self.children ← [ ]
3: end procedure
4: procedure ADD_CHILD(child_node)
5:     Add child_node to self.children
6: end procedure
7: procedure REMOVE_CHILD(child_node)
8:     Remove child_node from self.children
9: end procedure
10: procedure CLEAR_CHILDREN
11:     Remove all child nodes from self.children
12: end procedure
```

The Algorithm 4 describes the `CompositeNode` class, which is a type of node that has one or more child nodes. The class has four procedures: `CompositeNode`, `add_child()`, `remove_child()`, and `clear_children()`. `CompositeNode` initializes the node's children list to an empty list. `add_child()` adds a child node to the children list. `remove_child()` removes a child node from the children list. `clear_children()` removes all child nodes from the children list. These procedures allow the composite node to manage its child nodes and combine their behavior to create more complex behavior for the overall behavior tree.

In Algorithm 5 we have the code for the `SequenceNode` Class. The code iterates through each child node and retrieves its execution status. If any child node returns "FAILURE," the `SequenceNode` immediately returns "FAILURE" to indicate that the sequence has failed. Similarly, if any child node returns "RUNNING", the `SequenceNode` returns "RUNNING" to indicate that the sequence is still in progress. If none of the child nodes return "FAILURE" or "RUNNING", the `SequenceNode` returns "SUCCESS"

Algorithm 5 SequenceNode Class

```
1: procedure EXECUTE
2:     for each child_node in self.children do
3:         status ← child_node.execute()
4:         if status is "FAILURE" then
5:             return "FAILURE"
6:         else if status is "RUNNING" then
7:             return "RUNNING"
8:         end if
9:     end for
10:    return "SUCCESS"
11: end procedure
```

to indicate that all child nodes have successfully executed.

Algorithm 6 SelectorNode Class

```
1: procedure EXECUTE
2:     for each child_node in self.children do
3:         status ← child_node.execute()
4:         if status is "SUCCESS" then
5:             return "SUCCESS"
6:         else if status is "RUNNING" then
7:             return "RUNNING"
8:         end if
9:     end for
10:    return "FAILURE"
11: end procedure
```

Furthermore, in Algorithm 6 the `execute()` method of `SelectorNode` iterates over each child node in the `self.children` list and executes them one by one in ordinate sequence. The `status` variable stores the status returned by each child node's `execute()` method. If any child node returns "SUCCESS", then the `execute()` method of `SelectorNode` also returns "SUCCESS". If none of the child nodes return "SUCCESS", then the `execute()` method of `SelectorNode` returns "FAILURE".

Algorithm 7 DecoratorNode Class

```
1: procedure DECORATORNODE(child_node)
2:     self.child_node ← child_node
3: end procedure
```

In addition, the Algorithm 7 describes the `DecoratorNode` class, which is a type of node that has exactly one child node. The class has only one procedure, which takes a child node as input and assigns it to the `self.child_node` variable. Decorator nodes are used to modify the behavior of their child node in some way, such as inverting the result of the execution of a node, as seen in Algorithm 8.

Algorithm 8 InverterNode Class

```
1: procedure EXECUTE
2:     status ← self.child_node.execute()
3:     if status is "SUCCESS" then
4:         return "FAILURE"
5:     else if status is "FAILURE" then
6:         return "SUCCESS"
7:     else
8:         return status
9:     end if
10: end procedure
```

The following pseudocode in Algorithm 9 demonstrates an example for the `BehaviorTree` class. First, a root_node object of type `SelectorNode` is created. Two child nodes are added to the `root_node` object, a `SequenceNode` object and an `InverterNode` object, using the `add_child()` method. The `SequenceNode` object has a list of child nodes [`ConditionNode()`, `ActionNode()`]. The InverterNode object has a single child node `ConditionNode()`. Finally, a `behavior_tree` object is created using the `BehaviorTree` class, and the `run()` method is called on it to execute the behavior tree.

Algorithm 9 Example Usage of BehaviorTree

```
root_node ← SelectorNode()
root_node.add_child(SequenceNode([ConditionNode(), ActionNode()]))
root_node.add_child(InverterNode(ConditionNode()))
behavior_tree ← BehaviorTree(root_node)
behavior_tree.run()
```

2.3 Cooperation in Robotics

Cooperation in robotics is an essential research topic that focuses on how robots can collaborate to achieve a common goal (CAO *et al.*, 1995). The field has seen significant progress in recent years, with many researchers investigating various aspects of robot cooperation, such as task allocation (RM *et al.*, 2020), communication (ALSAMHI *et al.*, 2020), and coordination (BERLINGER *et al.*, 2021). In particular, multi-robot systems (MRS) have received much attention in the literature due to their potential for improving performance and scalability in various domains. In MRS, the robots work together as a team, each contributing with its unique capabilities to achieve a shared objective (RIZK *et al.*, 2019). For instance, some robots may focus on sensing the environment, while others may perform tasks that require physical interaction. To enable effective cooperation in MRS, many researchers have proposed different approaches, such as centralized and

decentralized task allocation, communication protocols, and coordinated movement control. These approaches aim to improve the robots' ability to collaborate by ensuring that they can share information, coordinate their actions, and adapt to changing situations. Some notable examples of cooperation in robotics include swarm robotics (BENI; WANG, 2004), multi-robot path planning (LAVALLE, 2001), and task allocation in MRS (GERKEY; MATARIC, 2004). Overall, cooperation in robotics remains an active research area with significant potential for advancing the field and enabling new applications.

The Cooperative Engagement Capability (CEC) is an emerging systems-of-systems capability in which multiple systems coordinate their actions through a network to improve their abilities to effectively perform a given task (APL-TEAM, 1995). Highly developed systems of cooperation and mutual support can be found in different sorts of systems, a general consensus is that cooperation is in general advantageous for the group of cooperators as a whole, even though it may curb some individual's resources (HEYLIGHEN, 2009). When discussing systems that rely on cooperation, it is important to define commonly used terms. For example Heylighen (2009) defines *cooperation* as a behavior that benefits another individual, while *altruism* refers specifically to a behavior that benefits another individual at a cost to oneself. By establishing these definitions, we can better understand and analyze how cooperation and altruism function within a given system.

Altruism means that a system performs actions that increase the fitness of another system. The term *Weak* altruism can be defined as behavior that benefits more to another individual than to the individual performing the behavior. A *Strong* altruism denotes behavior that benefits others, but at one's own cost (CAMPBELL, 1983). Both are common and necessary in those highly cooperative systems, which often have a collective organization with full division of labor, including individuals who are only defended by others (*i.e,* an UAV leader) or who are prepared to sacrifice themselves for the defense of others (*i.e,* the LW that sacrifices itself in extreme cases to ensure the protection of the UAV leader). These types of systems are frequently observed in nature among certain species of insects, as well as within human society (CAMPBELL, 1983).

On the contrary, the concept of *Selfishness* refers to a system's tendency to prioritize actions that enhance its own fitness (HEYLIGHEN, 2009). While naturally selected systems do exhibit selfish tendencies as they strive to optimize their own fitness, it is important to note that they are not solely driven by self-interest. In fact, such systems also tend to exhibit a reluctance towards altruism. As a result, engaging in altruistic behavior indirectly diminishes one's own fitness.

Cooperation in swarms of drones has emerged as a fascinating area of research, offering numerous possibilities for enhancing collective capabilities. Various techniques have been explored to enable effective collaboration among drones within a swarm. One commonly employed technique is decentralized control, where each drone operates autonomously

based on local information and simple rules, allowing the swarm to exhibit emergent collective behavior, as the ones seen in the work by (GIACOMOSSI *et al.*, 2021a) where a swarm of drones, through simple rules, exhibit an emergent collective behavior to defend a leader UAV.

Additionally, swarm intelligence algorithms inspired by social insect behavior, such as ant colony optimization, have been adapted to coordinate drone swarms (YANG *et al.*, 2017). These algorithms facilitate information sharing, task allocation, and adaptive decision-making, enabling efficient division of labor and achieving collective objectives. Furthermore, effective communication protocols play a pivotal role in maximizing the capabilities of UAV swarms, facilitating information exchange and ensuring cohesive swarm coordination among drones (CAMPION *et al.*, 2018). By combining these techniques, cooperation in swarms of drones becomes a powerful approach to tackle complex tasks.

2.4 Optimization Algorithms

Once we have the decision-making methods defined and the desired behaviors detailed, one way to refine the AI architecture and improve the behaviors of the UAVs is through optimization algorithms.

Optimization means finding the best solution among many feasible solutions available. Feasible solutions are those that satisfy all the constraints in the optimization problem (ARORA, 2019). The best solution could be minimizing the cost of a process or maximizing the efficiency of a system i.e weight minimization is a prime consideration for aerospace applications (ARORA, 2019; GIACOMOSSI *et al.*, 2022a). In an optimization problem, a function is to be maximized or minimized. The function that is being optimized is referred to as the objective function or fitness function. The function is a quantity such as cost, efficiency, weight, output, and so on. In our context, the optimization problem is for example the maximization of the loyal wingman efficiency in neutralizing threats by defining an objective function using some kind of metric. The variables in the objective function are denoted the design variables or decision variables. Typically it could be the thresholds used to define conditions for behavior switching or parameters of the decision-making architecture, for a optimization problem of the loyal wingman efficiency. From practical considerations, design variables can take values within a lower and an upper limit. For instance, the maximum engagement distance is limited to a certain value. The design variables can be a real or a discrete number, binary, or integer type.

There are many alternative optimization algorithms, such as Particle swarm optimization (PSO) (KENNEDY; EBERHART, 1995) and Covariance Matrix Adaptation Evolution Strategy (CMA-ES) (HANSEN *et al.*, 2003) being frequently used, since they can handle

search-spaces with a large number of variables. PSO is a computational method that is intended for simulating social behavior as a stylized representation of the movement of organisms in a bird flock or fish school (KENNEDY; EBERHART, 1995). On the other hand, CMA-ES is an evolutionary algorithm for difficult non-linear non-convex black-box optimization problems in continuous domain. The CMA-ES is considered as state-of-the-art in evolutionary computation and has been adopted as one of the standard tools for optimization in many research and industrial environments (HANSEN, 2016).

2.4.1 Particle Swarm Optimization (PSO)

Particle swarm optimization (PSO) is an efficient and widely used optimization algorithm in the literature, particularly in the context of unmanned aerial vehicles (UAVs). It has been extensively studied for various applications such as tuning flight control proportional–integral–derivative (PID) gains for quadrotors (WANG *et al.*, 2015; MAC *et al.*, 2016), parameter optimization for UAV swarms (GIACOMOSSI *et al.*, 2022a; GIACOMOSSI *et al.*, 2021a; CRUZ *et al.*, 2012), and more. PSO operates by iteratively improving a population of candidate solutions, known as particles, which are moved within a search-space based on a measure of quality (KENNEDY, 1997). This approach enables PSO to efficiently solve optimization problems involving continuous nonlinear functions and explore large solution spaces. Each particle updates its position and velocity by considering its own experience as well as the best-performing particles in the swarm, facilitating a dynamic exploration of the search space and convergence towards the global optimum.

One of the notable advantages of PSO is its classification as a meta-heuristic algorithm. Unlike traditional optimization methods that rely on problem-specific assumptions and gradients, PSO makes minimal assumptions and does not require differentiability (KENNEDY, 1997). This flexibility allows PSO to handle a wide range of problem types. However, it is important to note that while PSO can be highly effective, it does not guarantee finding the global optimal solution due to its stochastic nature (KENNEDY, 1997). Nevertheless, its versatility and ability to efficiently optimize complex problems make PSO a valuable tool in optimization research and practical applications.

The Algorithm 10 is a pseudo-code for PSO. In PSO, each particle maintains its own position and velocity, and it searches for the optimal solution by adjusting its velocity based on its previous velocity, cognitive component (based on its best-known position), and social component (based on the best-known positions of the swarm). The algorithm iteratively updates the positions and velocities of particles until a termination criteria is met, such as reaching a maximum number of iterations or achieving a desired level of convergence. The best-known positions of both individual particles and the entire swarm are updated throughout the iterations, allowing the particles to explore and converge

towards the global optimum.

Algorithm 10 PSO Algorithm

1: Initialize the swarm of particles with random positions and velocities
2: Initialize the best-known positions of particles in the swarm
3: Set the maximum number of iterations or convergence criteria
4: **while** termination criteria are not met **do**
5: **for** each particle in the swarm **do**
6: Update the particle's velocity using the previous velocity, cognitive component, and social component
7: Update the particle's position based on the new velocity
8: **if** the new position is better than the particle's best-known position **then**
9: Update the particle's best-known position
10: **end if**
11: **end for**
12: Update the best-known positions of the entire swarm
13: Check the termination criteria
14: **end while**

In step 6 of the Algorithm 10, the velocity of each particle is updated using the following equation:

$$\mathbf{v}_i^{t+1} = w \cdot \mathbf{v}_i^t + c_1 \cdot \mathbf{r}_1 \cdot (\mathbf{p}_i - \mathbf{x}_i^t) + c_2 \cdot \mathbf{r}_2 \cdot (\mathbf{p}_g - \mathbf{x}_i^t) \tag{2.1}$$

where $\mathbf{v}_i^{t+1}$ represents the updated velocity of particle i at time $t+1$. The inertia weight w controls the impact of the previous velocity $\mathbf{v}_i^t$. c_1 and c_2 are the cognitive and social acceleration coefficients, respectively, which determine the influence of the particle's personal best ($\mathbf{p}_i$) and the swarm's global best ($\mathbf{p}_g$). Also, $\mathbf{r}_1$ and $\mathbf{r}_2$ are random vectors of the same dimension as the problem space, sampled from a uniform distribution between 0 and 1. Finally, the current position $\mathbf{x}_i^t$ of particle i at time step t.

Similarly, in step 7 of Algorithm 10, the position of each particle is updated using the following equation:

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1} \tag{2.2}$$

where $\mathbf{x}_i^{t+1}$ and $\mathbf{x}_i^t$ represent the updated position and current position of particle i, respectively. The updated velocity of particle i at a time step is given by $\mathbf{v}_i^{t+1}$.

Note that these equations assume vector quantities $\mathbf{v}, \mathbf{x}$, $\mathbf{p}_i$, and $\mathbf{p}_g$. The variable ω is a scalar coefficient, and $\mathbf{r}_1, \mathbf{r}_2$ are random vectors.

2.4.2 The Kuhn-Munkres Algorithm

The Kuhn-Munkres algorithm (MUNKRES, 1957; KUHN, 1955), also known as the Hungarian algorithm or simply Munkres Algorithm, is a polynomial-time algorithm for solving the assignment problem. The assignment problem is a combinatorial optimization problem that involves assigning a number of resources to an equal number of tasks, where each resource has a cost associated with performing each task. This algorithm is commonly used in cooperative problems such as assignment problems (ZHU *et al.*, 2011), where a set of tasks or resources need to be assigned to a set of agents or individuals in the most efficient way possible.

In a cooperative problem, the Munkres algorithm can help determine the optimal assignment of tasks or resources to agents based on their individual abilities, preferences, or constraints. For example, in a manufacturing setting, the algorithm can be used to assign workers to different production lines based on their skills and availability. In a logistics problem using drones, the algorithm can be used to assign drones to delivery routes based on their location and capacity. The Munkres algorithm is particularly useful in cooperative problems because it guarantees the optimal solution, meaning that it finds the assignment that has the lowest cost or highest profit out of all possible assignments. This is important in cooperative settings because it ensures that resources are allocated fairly and efficiently among the agents involved.

The Munkres algorithm works by iteratively finding a minimum-weight matching in a bipartite graph, where the vertices on one side of the graph represent the resources and the vertices on the other side represent the tasks. The algorithm starts with an arbitrary matching and then improves it by finding augmenting paths, which are paths in the graph that alternate between matched and unmatched edges and start and end with unmatched edges. The algorithm terminates when it cannot find any more augmenting paths, at which point it returns the final matching. The matching returned by the algorithm is guaranteed to be optimal, meaning that it has the minimum total cost of any matching.

The Algorithm 11 is the pseudocode for the Munkres algorithm. In this pseudocode, M is the binary matrix representing the final matching, where $M_{i,j} = 1$ if resource i is assigned to task j, and $M_{i,j} = 0$ otherwise.

The Algorithm 11 starts by initializing a matrix M of zeros with the same dimensions as the cost matrix C. It then iterates until there are zeros left in the cost matrix. In each iteration, it finds a zero in C and marks the corresponding row and column as covered. It maintains a set S to keep track of zeros on the alternating path. Marking a row and column as covered means flagging them to track progress, temporarily excluding them from further consideration in subsequent steps. Implementation varies but typically involves setting flags or using data structures. The algorithm enters a nested while loop

Algorithm 11 Munkres Algorithm

```
1: function MUNKRES(C)                                   ▷ C is the cost matrix
2:     Initialize M as a matrix of zeros with the same dimensions as C
3:     while there exists a zero in C do
4:         Find a zero (i, j) in C
5:         Mark row i and column j as covered
6:         S ← (i, j)                    ▷ S is the set of zeros on the alternating path
7:         while true do
8:             Find a non-covered zero (k, l) in row i or column j
9:             if no such zero exists then
10:                Augment the alternating path with minimum uncovered cost
11:                for all (p, q) ∈ S do
12:                    if M_{p,q} = 1 then
13:                        M_{p,q} ← 0
14:                    else
15:                        M_{p,q} ← 1
16:                    end if
17:                end for
18:                Break
19:            else
20:                Mark row k and column l as covered
21:                S ← S ∪ (k, l)
22:                (i, j) ← (k, l)      ▷ Updates the current zero coordinate (i, j) to (k, l).
23:            end if
24:        end while
25:    end while
26:    return M
27: end function
```

where it searches for non-covered zeros in the same row or column as the current zero. If a non-covered zero is found, the corresponding row and column are marked as covered, and the zero is added to the set S. The algorithm then continues to the next iteration using the new zero. If no non-covered zero is found, the algorithm augments the alternating path by selecting the minimum uncovered cost among the covered rows and uncovered columns. It then updates the markings in the matrix M based on the set S and goes back to the beginning of the outer loop. The algorithm terminates when there are no more zeros in the cost matrix, and it returns the resulting matrix M, which represents the optimal assignment.

The Munkres algorithm has a time complexity of $O(n^3)$, where n is the number of tasks (MUNKRES, 1957). This makes it a good solution for solving large instances of the assignment problem, which may not be tractable using other methods. In addition, the Munkres algorithm is relatively easy to implement and has been well studied, with many variants and improvements proposed in the literature (BOURGEOIS; LASSALLE, 1971). It has also been used in a wide range of fields, including computer vision, operations research, and UAV swarms (HERNÁNDEZ *et al.*, 2021). Overall, it is a powerful and versatile tool for solving the assignment problem and is a popular choice for many applications that requires resource allocation (LIU *et al.*, 2023; CHEN *et al.*, 2022)

2.5 Search Algorithms

Search algorithms are fundamental tools used to explore and locate desired elements within a given domain. They employ systematic and efficient techniques to navigate through a search space and identify the most relevant solutions (KORF, 1999). Various search algorithms, such as the Bayes Search (BS) theory (STONE, 1975), Hill Climbing Algorithm (SELMAN; GOMES, 2006), and Lawn Mower Search Algorithm (OUSINGSAWAT; EARL, 2007), offer distinct strategies for different scenarios. The Bayes Search Algorithm utilizes probabilities and updates them based on prior assumptions and collected data. The Hill Climbing Algorithm incrementally moves towards higher-value solutions while being prone to local optima, the same idea used for optimization can be applied to search problems, as seen in Giacomossi *et al.* (2023). The Lawn Mower Search Algorithm imitates the motion of a lawn mower and is suitable for problems with limited information. These algorithms provide valuable approaches to tackle diverse search problems and contribute to enhancing optimization, decision-making, and problem-solving processes in numerous fields.

Search algorithms play a crucial role in optimizing the performance and efficiency of drones and drone swarm applications (GARCIA-AUNON; CRUZ, 2018). In the context of

drones, these algorithms enable autonomous exploration and target localization, taking into account the peculiarities of these systems (number of robots, limited communications and sensing and computational capacities). By leveraging with efficient search techniques, drones can cooperate and effectively navigate through an area to identify potential target locations. Additionally, search algorithms such as Hill Climbing and Lawn Mower Search can be employed to enhance the search capabilities of drone swarms. These algorithms allow the drones to coordinate their movements, optimize coverage, and adapt to complex environments. The application of search algorithms to drone swarms facilitates efficient search and reconnaissance operations (KARACA *et al.*, 2018), enabling them to effectively accomplish tasks like surveillance, and search and rescue.

2.5.1 Lawn Mower Search (LMS)

The Lawn Mower Search (LMS) algorithm (GARCIA-AUNON; CRUZ, 2018; OUSINGSAWAT; EARL, 2007), also referred to as a row, line, or column search, draws inspiration from the movement of a lawn mower and has found applications in optimization problems. In the research conducted by Ousingsawat e Earl (2007), they demonstrated that LMS divides the search space into sub-regions and performs the search in a zigzag pattern, mimicking the motion of a lawn mower. In this pattern, the agent moves in a straight line, and when it encounters an obstacle, it turns to head in a clear direction. This pattern is especially advantageous when dealing with problems characterized by limited information about the location of the object of interest because it performs a exhaustive and organized search. LMS has been extensively validated and proven to deliver optimal solutions, which has contributed to its widespread adoption for search and coverage problems (ALTSHULER *et al.*, 2008).

The search patterns can be enhanced by employing multiple agents to conduct the search simultaneously (GARCIA-AUNON; CRUZ, 2018). Instead of relying on a single agent to traverse the search space in a zigzag pattern, multiple agents can divide and conquer the area, ensuring a more efficient and comprehensive exploration, illustrated in Figure 2.4. By assigning specific regions to each agent, the search process becomes parallelized, reducing the overall search time. Additionally, with multiple agents operating concurrently, the likelihood of overlooking critical areas diminishes. This multi-agent approach enhances the adaptability of LMS, making it even more effective in handling complex scenarios and utilizing available information efficiently.

However, it is important to acknowledge that LMS does have certain limitations, such as energy inefficiency and time consumption. In specific areas, the search process might require considerable time, potentially overlooking small yet crucial areas. Consequently, in more complex scenarios, the need arises for more sophisticated search algorithms that

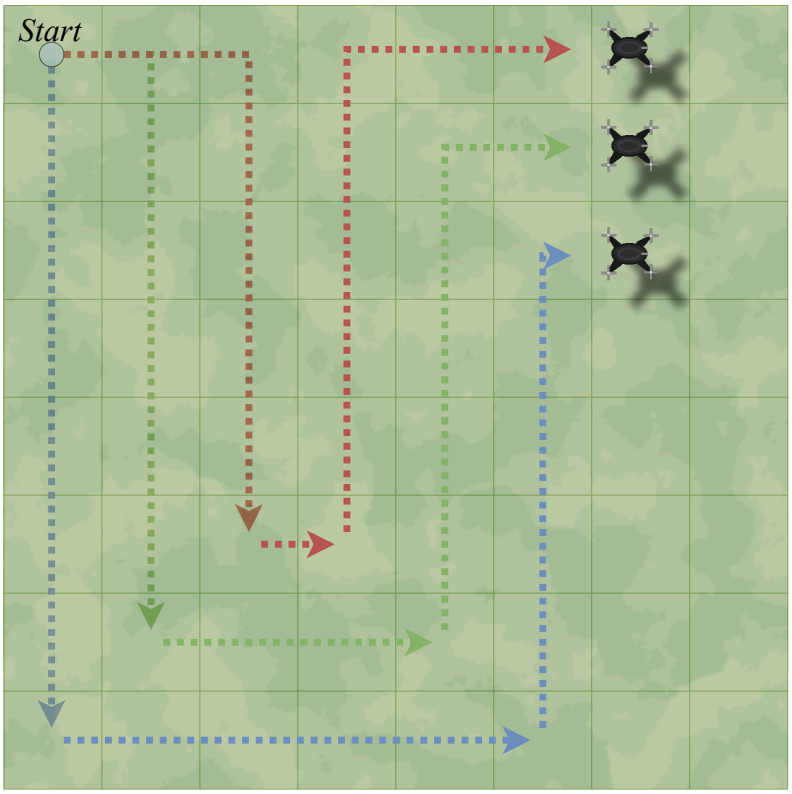

FIGURE 2.4 – Example of a swarm formation executing the lawn mower search pattern. The dashed lines illustrate the paths.

can adapt to diverse situations and effectively leverage the available information.

The following Algorithm 12 is an example of pseudo-code for a basic Lawn Mower Search (LMS) algorithm, it provides a high-level overview of the algorithm. The code can be customized based on the specific requirements of the desired implementation, such as handling boundaries, maintaining a grid of visited cells, or adding additional termination conditions.

Algorithm 12 Lawn Mower Search Algorithm

```
1: procedure LawnMowerSearchAlgorithm(grid)
2:     Initialize variables
3:     Set starting position and direction
4:     while search is not complete do
5:         Move in the current direction until reaching the boundary or a visited cell
6:         Mark the current cell as visited
7:         if reached the boundary then
8:             Change direction
9:             Move one step forward in the new direction
10:        end if
11:        if all cells are visited then
12:            search is complete
13:        end if
14:    end while
15:    return visited cells
16: end procedure
```

2.5.2 Hill Climbing (HC) Search

The Hill Climbing (HC) algorithm (SELMAN; GOMES, 2006) is a powerful optimization technique widely employed in computer science and AI. It is a local search algorithm that iteratively moves towards higher-value solutions in the search space by evaluating neighboring solutions and selecting the one with the highest value. The HC algorithm continues this process until it reaches a peak where no higher-value solution can be found. For convex problems, it will always identify the optimum solution. However, if the problem is not convex, the algorithm can become trapped in local maxima and minima, resulting in suboptimal solutions. To overcome this limitation, variants of the HC algorithm have been developed, introducing randomness and memory to the search process (CHALUP; MAIRE, 1999; RUSSELL *et al.*, 2010), which allows the algorithm to escape local optima and improve the solution. The HC algorithm is particularly useful when information outside the immediate vicinity is unavailable, as it follows the most promising path. However, the challenge of getting trapped in local maxima persists.

In general, HC search offers benefits (SELMAN; GOMES, 2006). Firstly, it requires only a limited amount of memory since it only stores the current state, making it memory-efficient. Secondly, it is relatively easy to implement efficiently, making it accessible even to those with basic programming skills. Lastly, if one or more solutions exist within the search space, the hill climbing algorithm can be highly effective in finding them.

One notable example of the successful usage of the hill climbing strategy was by Lin e Kernighan (1973) in tackling the traveling salesperson problem. They employed the hill climbing approach to find good solutions for this complex optimization problem. The algorithm iteratively improved upon the current solution by making small local modifications, gradually moving towards a better solution. This pioneering work demonstrated the effectiveness of hill climbing in solving challenging problems such as the traveling salesperson problem.

Algorithm 13 Hill Climbing Algorithm

```
1: procedure HillClimbing(initialState)
2:     currentSolution ← initialState
3:     while True do
4:         Neighbors ← generateNeighbors(currentSolution)
5:         bestNeighbor ← None
6:         for Neighbor in Neighbors do
7:             if heuristic(Neighbor) > heuristic(bestNeighbor) then
8:                 bestNeighbour ← Neighbour
9:             end if
10:        end for
11:        if heuristic(bestNeighbor) ≤ heuristic(currentSolution) then
12:            break
13:        end if
14:        currentSolution ← bestNeighbor
15:    end while
16:    return currentSolution
17: end procedure
```

In Algorithm 13, `initialState` represents the starting point of the search. The algorithm iteratively generates neighbouring solutions using the `generateNeighbors` function. It then evaluates each Neighbour using a heuristic function and selects the Neighbour with the highest heuristic value as the `bestNeighbor`. If the heuristic value of the `bestNeighbor` is not greater than the heuristic value of the current solution, the algorithm terminates as it has reached a local optimum. Otherwise, the `currentSolution` is updated to the `bestNeighbor`, and the process continues until termination. Finally, the algorithm returns the solution found.

2.5.3 Bayesian Search (BS) Theory

Bayesian search (BS) theory, also known as optimal search theory (STONE, 1975), is a mathematical framework that deals with finding the best search strategy in situations where the location of a target is uncertain or unknown. It provides a systematic approach for allocating search resources and making decisions to maximize the chances of locating the target efficiently.

The Bayes Search (BS) theory, which emerged during World War II in the search for German submarines (KOOPMAN, 1999), is founded on the principles of Bayes' theorem (JOYCE, 2021). Bayes' theorem serves as a fundamental principle within probability theory, providing a mathematical formulation for updating the probability of a hypothesis based on new evidence. This theorem is essential for Bayesian inference (BOX; TIAO, 2011), a broader framework used to address challenges in statistical modeling and decision-making. The BS theory incorporates Bayesian principles, including prior knowledge, observation data, and the concept of utility, to guide the search process. By balancing exploration (searching new areas) and exploitation (focusing on areas with higher probabilities of containing the target), it optimizes search efforts.

In Bayesian search theory, the search is typically modeled as a sequence of search actions or decisions (TOUSSAINT, 2013). Each action involves selecting a specific search area or strategy based on the available information. The search algorithm then updates its beliefs about the target's location using Bayes' theorem and the observed data, as demonstrated by Wong *et al.* (2005). This process allows the search algorithm to iteratively refine its search strategy based on the accumulated information.

The goal of Bayesian search theory is to find the search strategy that maximizes the expected utility or the probability of successfully locating the target within a given resource constraint. The utility function takes into account factors such as the importance of finding the target, the cost of search efforts, and the time sensitivity of the search mission (STONE, 1975).

The BS theory utilizes Bayesian inference, which is a statistical method for updating probabilities based on observed data and prior beliefs (BOX; TIAO, 2011). In Bayesian search theory, the goal is to efficiently search for a target in an uncertain environment, taking into account prior knowledge about the target's location and characteristics, as well as new observations obtained during the search process. By continuously updating the probabilities using Bayes' theorem, Bayesian search theory enables decision-making under uncertainty and allows for more effective and informed search strategies.

2.5.3.1 Bayesian Inference

Bayesian inference is a method of statistical framework that allows us to update our beliefs or knowledge about a particular event or hypothesis using Bayes' theorem (BOX; TIAO, 2011). It provides a framework for combining prior knowledge or beliefs with new evidence to obtain a posterior probability distribution.

The process of Bayesian inference involves the following steps:

1. *Prior Probability*: Start with an initial belief or prior probability distribution about

the event or hypothesis based on existing knowledge, experience, or subjective judgment. The prior probability represents what we believe before considering any new evidence.

2. *Likelihood*: Collect or observe new evidence or data relevant to the event or hypothesis. The likelihood function measures how likely the observed data is under different values of the parameters or hypotheses being considered.

3. *Posterior Probability*: Apply Bayes' theorem to update the prior probability distribution based on the observed data. The posterior probability distribution represents the updated belief after considering the new evidence.

Bayes' theorem (JOYCE, 2021) mathematically describes the relationship between the prior probability $P(A)$, likelihood $P(B|A)$, and posterior probability $P(A|B)$ as follows:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}, \tag{2.3}$$

where $P(A|B)$ is the posterior probability of event A given the observed data B. $P(B|A)$ is the likelihood of observing the data B given event A. $P(A)$ is the prior probability of event A. Lastly, $P(B)$ is the probability of observing the data B. The posterior probability distribution can be used for various purposes, such as making predictions, estimating unknown parameters, or selecting the most probable hypothesis.

For instance, the usual procedure for locating a missing object involves several steps (STONE *et al.*, 2016). First, hypotheses are formulated about the object's location. Probability functions are then created for the hypothesis, determining the probability of finding the object. These function depends on environment factors, *e.g.* water depth in ocean searches. Next, these probabilities are combined to generate an overall probability map, representing the probability of finding the object. A search path can be constructed, starting from the point of highest probability and gradually scanning through areas with decreasing probabilities. Throughout the search, probabilities are continuously revised using Bayes' theorem. If search efforts yield no results in certain areas, the probability of the object being there diminishes while probabilities for other locations increase. This process continues until the object is found.

One of the key advantages of Bayesian inference is its ability to incorporate prior knowledge or beliefs into the analysis, allowing for a more coherent and intuitive way of updating beliefs based on new evidence (JOYCE, 2021). However, it does require the specification of prior probabilities, which can sometimes be subjective or challenging to determine.

2.5.3.2 Applications of Bayesian Inference

Bayesian inference has found applications in various domains. Here are some examples:

- Machine Learning: Bayesian search is commonly used for hyperparameter optimization in machine learning (TURNER *et al.*, 2021). The goal is to find the best set of hyperparameters for a given model. By incorporating prior knowledge about the hyperparameters and updating the belief distribution based on the model's performance, Bayesian search can efficiently explore the hyperparameter space and identify optimal configurations.

- Robotics: Bayesian inference is applied in simultaneous localization and mapping (SLAM) problems in robotics (CHANG *et al.*, 2007). SLAM algorithms estimate the robot's pose and map of the environment based on sensor measurements. By maintaining a belief distribution over the robot's pose and the map, Bayesian search algorithms can handle uncertainties and refine the estimates as new measurements are obtained.

- Decision-Making under Uncertainty: Bayesian inference are used in decision-making problems where uncertainty is present (CHUNG; BURDICK, 2012). By representing the belief distribution over the possible outcomes and updating it based on new information, Bayesian search can guide decision-making processes in a principled manner, taking into account uncertainties and updating beliefs accordingly.

- Locating Lost Vessels and Airplanes: Bayesian inference was employed through BS theory in attempts to locate lost maritime vessels, such as the USS Scorpion, as well as in the recovery of black boxes in airplane accidents like the Air France Flight 447 disaster in 2009. It can help narrow down search areas and prioritize search efforts based on available information and prior knowledge. Applications of Bayesian search theory can be found in various fields, including search and rescue operations for multiple targets (WONG *et al.*, 2005), wildlife tracking (KÖRNER *et al.*, 2010), and military operations (CAUDLE, 2010). It provides a principled and systematic approach to guide search efforts in situations where uncertainty exists regarding the target's location.

2.6 Collision Avoidance

In this section, we introduce the important topic of collision avoidance. Collision avoidance refers to the set of techniques, strategies, and algorithms implemented to prevent or

minimize collisions between objects, agents, or vehicles in dynamic environments (SIEGWART *et al.*, 2011a). It involves anticipating and reacting to potential collision situations by adjusting the trajectory, speed, or behavior of the robot. The primary objective of collision avoidance is to ensure the safety of individuals, vehicles, or systems operating in shared or congested spaces (ZENG; BONE, 2013). Collision avoidance systems can be found in various domains, including autonomous vehicles (FUNKE *et al.*, 2016), aerial robotics (RICARDO; SANTOS, 2023b), and even maritime navigation (ZHANG *et al.*, 2021). These systems utilize sensors, perception algorithms, decision-making algorithms, and control mechanisms to detect and respond to potential collision threats, thereby mitigating the risk of accidents and ensuring the smooth and efficient operation of the agents involved. We are going to present two prominent methods: Potential Fields (PF) (HWANG *et al.*, 1992) and Acceleration-Velocity-Obstacles (AVO) (BERG *et al.*, 2011a). On one hand, the PF method employs the concept of attractive and repulsive forces to guide agents towards their desired goals while keeping them safe from obstacles. On the other hand, the AVO method utilizes velocity obstacles and acceleration constraints to generate collision-free trajectories for multiple agents.

2.6.1 Potential Field (PF)

A traditional technique for the interaction of multiple entities is the potential fields method (HWANG *et al.*, 1992; CHOSET *et al.*, 2005). In this method, obstacles exert repulsive forces ($|\mathbf{F_r}|$) on the agent, based on the distance d to the obstacles and a repulsion coefficient Q. The formula is derived from the electric potential field, which is given by

$$|\mathbf{F_r}| = \frac{Q}{d^2}. \tag{2.4}$$

In this method, obstacles exert repulsive forces ($\mathbf{F_r}$) on the agent. Let $\mathbf{p} = [x, y]^T$ be the position of the agent in the environment, we define a repulsive potential field to guide the agent's movement. For this, the agent must move according to the repulsive force vector

$$\mathbf{Fr}(\mathbf{p}) = -\sum_{i=1}^{Nobs} \nabla U(\mathbf{p}). \tag{2.5}$$

where $\nabla U(\mathbf{p})$ represents the gradient of the potential energy function $U(\mathbf{p})$ at the agent's position $\mathbf{p}$. The equation calculates the repulsive force vector $\mathbf{Fr}$ acting on the agent at position $\mathbf{p}$ by summing up the gradients of the potential energy function $U(\mathbf{p})$ for all the obstacles $Nobs$ in the environment. The negative sign indicates that the force is repulsive, meaning it pushes the agent away from the obstacles.

As a primary assumption, UAVs must not collide with each other. In potential fields

method, obstacles exert repulsive forces ($\mathbf{F}_r$) on the agent to avoid collision. An implementation of this method is by using the bivariate function (BARNES, 2018), given by

$$f(x, y) = e^{-\alpha(x-x_c)^2-\gamma(y-y_c)^2}, \tag{2.6}$$

where $[x_c, y_c]^T$ is the center of the field and $[x, y]^T$ is a position on which the potential field has an influence. The x and y partial derivatives of Equation (2.6) create a force field that is used for the collision avoidance among the UAVs, described by

$$\begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix} = \begin{bmatrix} -2\alpha f(x,y) & 0 \\ 0 & -2\gamma f(x,y) \end{bmatrix} \begin{bmatrix} (x - x_c) \\ (y - y_c) \end{bmatrix}. \tag{2.7}$$

The normal bivariate function has the advantages of allowing easy tuning of the field's actuation region by the constants α and γ along the x and y axis, respectively. We can see in Figure 2.5, where α and γ are equal, generating a symmetrical field in x and y. The maximum $f(x, y)$ value is unitary for any values of α and γ, which occurs when the entities are very close, facilitating the adjustment of the maximum force value. However, potential fields technique has a disadvantage that needs to be taken into account. There is a possibility of encountering local minima in the scenario (SILVA, 2019), which may cause the UAV to become stationary due to the cancellation of the repulsion and attraction forces from various objects present in the scenario.

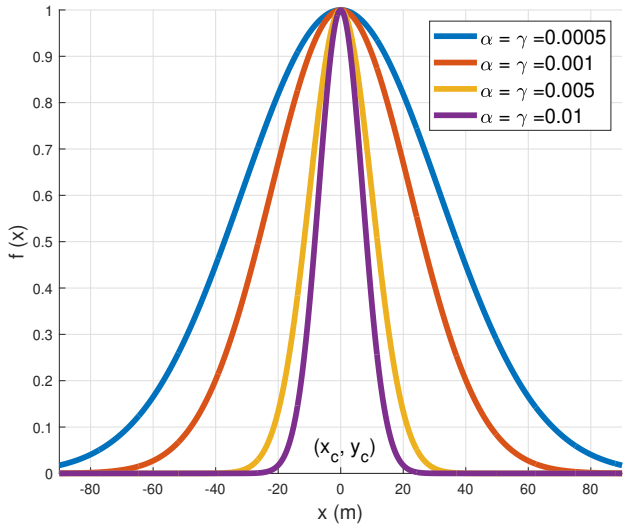

FIGURE 2.5 – One-dimensional normal bivariate function.

By applying the potential fields technique, drones can effectively navigate their surroundings with a high degree of precision and autonomy. This technique involves adapting the approach based on the distances between the drones ($\overrightarrow{d}_{i,k}$) and the distances between

the drone and obstacles ($\overrightarrow{d}_{i,j}$ and $\overrightarrow{d}_{k,j}$). Using the method for creating a repulsion force, a resulting signal is exerted on the drone's controllers, enabling it to avoid the collisions between the obstacles and UAVs that constitute the swarm. The application of this technique is visually represented in Figure 2.6, where two drones, denoted as i and k, interact with an obstacle j. The peaks in the force represent repulsion forces, illustrating how the field's influence depends on the distances between these elements.

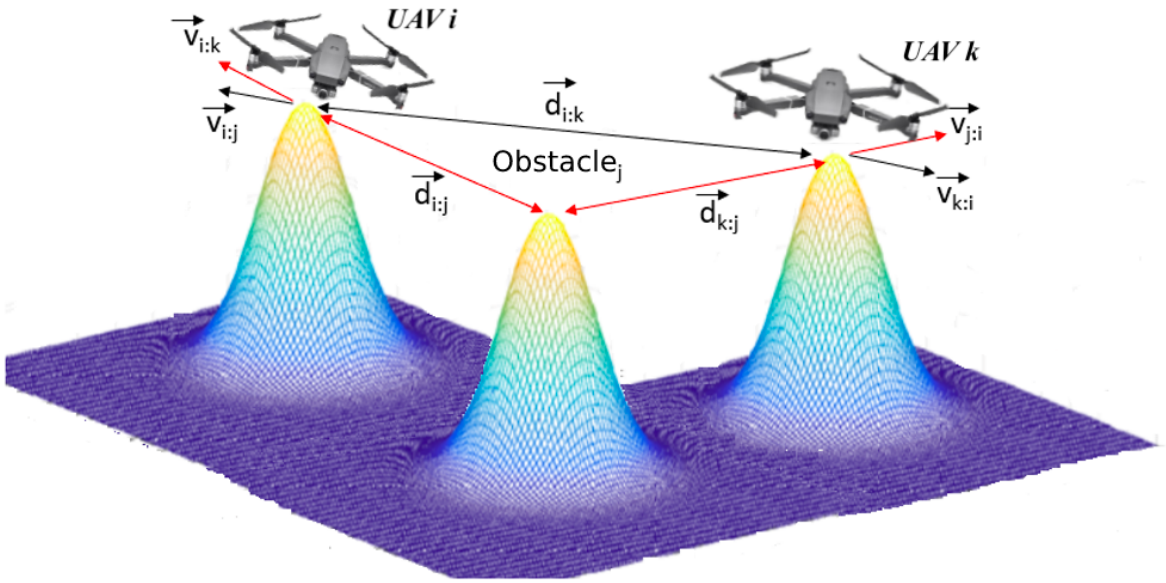

FIGURE 2.6 – Interaction of potential fields between drones and obstacles.

2.6.2 Acceleration-Velocity-Obstacles (AVO)

The Acceleration-Velocity-Obstacles (AVO) method (BERG *et al.*, 2011b) is a prominent technique used for real-time collision avoidance and motion planning in dynamic environments. The AVO method is based on the concept of treating agents as point masses with constant acceleration and velocity limits. It utilizes the notion of *velocity obstacles*, which represent the velocities that would result in a collision between agents if maintained over a given time horizon. By incorporating acceleration constraints, the AVO method generates collision-free trajectories for multiple agents by finding a feasible region in the acceleration-velocity space.

The AVO algorithm consists of several steps, including data initialization, prediction of future agent positions, calculation of *velocity obstacles*, and computation of feasible accelerations. These steps are executed iteratively to obtain collision-free trajectories (RICARDO; SANTOS, 2023a). The algorithm takes into account the relative positions, velocities, and acceleration limits of the agents to determine safe and efficient motion plans. The Algorithm 14 is a high-level overview adapted from Berg *et al.* (2011b) of the

AVO algorithm.

Algorithm 14 Acceleration-Velocity-Obstacles (AVO) Algorithm

Initialization:
Initialize the positions, velocities, and acceleration limits of all agents.
Set the maximum number of iterations and a convergence threshold.

Iterative Process:
while termination condition not met **do**
    for each agent A **do**
        Predict future position of Agent A based on its current velocity and acceleration.
        Calculate *velocity obstacles* for Agent A considering relative positions, velocities, and sizes of nearby agents.
        Compute feasible accelerations for Agent A by finding the intersection between acceleration constraints and the complement of *velocity obstacles*.
        Update accelerations of all agents based on the computed feasible accelerations.
        Update positions and velocities of all agents based on the updated accelerations.
        Check for convergence by evaluating the change in positions or velocities of the agents.
    end for
end while

Trajectory Generation:
Generate collision-free trajectories for each agent based on their updated positions and velocities.

Velocity obstacles are geometric constructs utilized in collision avoidance algorithms to identify regions within velocity space that agents must avoid to prevent collisions with other agents (FIORINI; SHILLER, 1998). They serve as a means to analyze potential collisions and inform the decision-making process of agents. By considering the relative positions, velocities, and sizes of nearby agents in relation to a specific agent, *velocity obstacles* determine the set of velocities that, if chosen by the agent, would lead to a collision with a nearby agent.

The shape and size of *velocity obstacles* are contingent upon various factors, encompassing the relative positions, velocities, and desired safety margin between the agent in focus and nearby agents. Typically, *velocity obstacles* exhibit a conical or wedge-like shape in velocity space, centered around the agent's present velocity. The orientation and dimensions of the cone are dictated by the relative positions and velocities of the nearby agents. In order to avoid collisions, agents need to select velocities that lie outside the boundaries of the velocity obstacle. This can be achieved by calculating the intersection between the agent's acceleration constraints (e.g., maximum acceleration limits) and the complement of the *velocity obstacle*. The resulting set of viable velocities can then guide the agent's decision-making regarding adjustments to its velocity or acceleration, ensuring

the avoidance of potential collisions.

One of the key advantages of the AVO method is its ability to handle complex and dynamic environments (HARALDSEN, 2020). By continuously updating the positions and velocities of agents, the AVO algorithm can adapt to changing scenarios in real-time. It considers both the current and future positions of agents to generate motion plans that avoid collisions. While the AVO method offers significant advantages, it also has certain limitations. It assumes perfect knowledge of the environment and the intentions of other agents, which may not be feasible in practice (BERG *et al.*, 2011a).

While the Acceleration-Velocity-Obstacles (AVO) algorithm has proven to be effective in collision avoidance scenarios, it does have certain disadvantages (DOUTHWAITE *et al.*, 2018). One limitation is its reliance on simplified assumptions, such as constant acceleration and linearity, which may not capture the complexities of real-world agent behaviors. Additionally, AVO can be computationally expensive, especially in environments with a large number of agents, as it requires iterative calculations and frequent updates of positions and velocities. Overall, these limitations highlight the need for further advancements and alternative algorithms to address the drawbacks of the AVO approach.

The time complexity of the method depends on various factors, including the number of agents, the complexity of the environment, and the specific implementation details. Moreover, the AVO algorithm can be parallelized to handle a large number of agents efficiently (BERG *et al.*, 2011a).

The AVO method offers a robust solution for motion planning and collision avoidance in dynamic environments, rendering it highly valuable across various domains. For instance, it finds utility in path planning for autonomous robots operating in dynamic environments (RICARDO; SANTOS, 2023a), collision avoidance for drones navigating airspace (RICARDO; SANTOS, 2023b), and more. As autonomous systems and robotics continue to advance, the AVO method undergoes continuous refinement, effectively addressing its limitations and ushering in a future of safer and more efficient motion planning in complex scenarios.

2.7 Position and Attitude Controllers

To enhance the movement capabilities of drones, it is crucial to enable them to navigate to desired coordinates accurately using a flight control system (FCS). Figure 2.7 presents a simplified flight control system for a 2D model, comprising a position control and an attitude control. Notice that this simplified FCS is based on the work by Giacomossi *et al.* (2022a), and the complete FCS for a 6 degrees of freedom drone encompasses additional elements like control allocators that receives a force and torque commands, and outputs

angular velocity commands for each rotor of the drone.

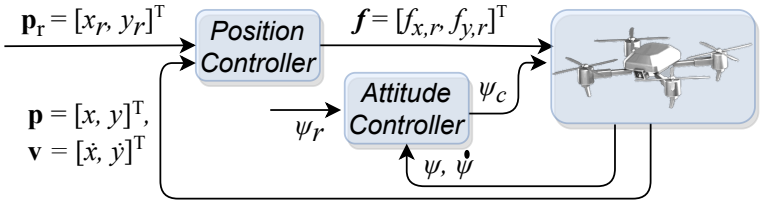

FIGURE 2.7 – Diagram for a simplified flight control system for a 2D drone.

The *position controller* receives as input a position reference $\mathbf{p}_r$, the current position $\mathbf{p}$ and the linear velocity $\mathbf{v}$ and outputs a force command $\boldsymbol{f}$. The *Attitude controller* receives as input a yaw angle reference ψ_r, the current yaw angle ψ and the angular velocity $\dot{\psi}$ and outputs a command $\psi_\mathbf{c}$. Finally, The *UAV block* implements the simplified quadcopter models, which updates the vehicle's velocity and position through numerical integration. This block outputs the position, velocities, attitude and angular velocities of the UAV.

One effective approach is to implement the position controller is through a simplified Position+Velocity $P + V$ controller (GIACOMOSSI *et al.*, 2022a), which calculates the appropriate force required for the drone's rotors, given by

$$\boldsymbol{f} = K_p(\mathbf{p}_r - \mathbf{p}) - K_v\mathbf{v}, f \in [f_{min}, f_{max}] \tag{2.8}$$

where the force ($\boldsymbol{f}$) is proportional (K_p) to the difference in position ($\mathbf{p}$) with the desired coordinates ($\mathbf{p}_r$) and is proportional (K_v) to the current velocity ($\mathbf{v}$) of the drone. By incorporating the $P + V$ controller into the drone's system, it is capable of dynamically adjusting rotor forces based on positional and velocity feedback. This enables precise navigation to the desired coordinates, even in the presence of external disturbances or changes in the environment. The work by Giacomossi *et al.* (2022a) demonstrated the usage of a $P + V$ controller for the drone and studied the optimization of its performance in aggressive maneuvers.

When applying force to the UAV model, it is crucial to consider the physical limitations of the rotors. Hence, it becomes necessary to account for the maximum force that the rotors can generate within our simulator. Additionally, it is important to acknowledge that the acceleration of the UAV model is determined by dividing the applied force by its mass. For simulation purposes, the UAVs can be represented as 2D point mass objects, exhibiting decoupled linear dynamics along the x-y axis, this representation aligns with the 2D prototype developed in the project, which is further detailed in Subsection 3.5.1.

Also, accurate control of the attitude of the UAV is crucial for drones to be able to rotate and move to desired coordinates, here we simplify for a 2D environment and

consider only the yaw angle (rotation around its vertical axis). To achieve this, we utilized the simplified $P + V$ attitude controller. The controller is given by

$$\psi_c = K_y(\psi_r - \psi) - K_{yv}\dot{\psi}, \tag{2.9}$$

where the yaw angle (ψ_c) is proportional (K_y) to the difference in the desired yaw angle (ψ_r) and the current yaw angle (ψ) of the drone, and is proportional (K_{yv}) to the current yaw rate ($\dot{\psi}$) of the drone.

By employing a simplified $P + V$ controller we can enhance a drone's ability to move accurately to desired coordinates in a simulated environment. By considering position, velocity and the yaw angle errors, the controller can generate the forces and angle commands, allowing the drone model to navigate and rotate effectively and accomplish its intended tasks.

2.7.1 Sliding Mode Control (SMC)

Sliding Mode Control (SMC) is a robust control methodology that has gained significant attention in various engineering disciplines due to its proven effectiveness in handling uncertainties and disturbances (RICARDO; SANTOS, 2022a). It was first introduced by Utkin (2013) and has since found widespread applications in fields such as aerospace and aerial robotics (RICARDO; SANTOS, 2022c).

At the core of SMC lies the concept of a sliding surface, a specific manifold in the state space that plays a crucial role in achieving control objectives. The sliding surface separates the state space into two distinct regions: the sliding mode and the switching mode. The fundamental idea of SMC is to force the system's state onto this sliding surface and maintain it there (UTKIN, 2013).

The dynamics of a system under SMC are typically represented by a first-order switching differential equation. Consider a system with state vector $\mathbf{x}$ and control input u, and let $\mathcal{S}$ represent the sliding surface. The system dynamics can be described as:

$$\dot{\mathbf{x}} = \begin{cases} \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})u, & \text{if } \mathbf{x} \in \mathcal{S} \\ \mathbf{f}(\mathbf{x}), & \text{otherwise} \end{cases}, \tag{2.10}$$

where $\mathbf{f}(\mathbf{x})$ represents the nominal dynamics of the system, $\mathbf{g}(\mathbf{x})$ characterizes the control action, u is the control input, and $\mathcal{S}$ denotes the sliding surface.

The control law in SMC consists of two main components: the nominal control law, denoted as $u_{\text{nom}}(\mathbf{x})$, and the sliding control term, represented as $u_{\text{slide}}(\mathbf{x})$. The nominal control law ensures that the system tracks the desired trajectory, while the sliding control

term enforces the system to remain on the sliding surface:

$$u = u_{\text{nom}}(\mathbf{x}) + u_{\text{slide}}(\mathbf{x}). \tag{2.11}$$

Designing an appropriate sliding surface is a crucial step in SMC. The sliding surface is typically defined as a hyperplane in the state space, often in the form of:

$$\mathcal{S} : s(\mathbf{x}) = 0, \tag{2.12}$$

where $s(\mathbf{x})$ is a function that characterizes the desired behavior on the surface.

One of the key advantages of SMC is its robustness to system uncertainties and disturbances. The sliding mode ensures that the system behaves predictably and converges to the sliding surface regardless of parameter variations or external disturbances. Stability analysis in SMC is crucial to ensure the convergence of the system's state to the sliding surface. It involves demonstrating that the sliding mode is globally asymptotically stable under certain conditions, guaranteeing the desired control objective.

While SMC offers robustness, practical implementations must address certain challenges. High-frequency oscillations known as chattering can be an issue, and techniques such as boundary layer design can be employed to mitigate this effect. Additionally, consideration of actuator limitations is essential in real-world applications.

2.8 Tools for UAV Simulation Development

This section explores the tools and methods employed in the development of the simulators for the project. Simulators play a crucial role in providing a virtual environment to test and validate the performance of drone systems without the need for real-world deployment.

2.8.1 Python and Pygame library

Python, being a versatile and widely used programming language, offers numerous advantages for project development. Its readability and simplicity have been recognized by developers worldwide (ROSSUM; DRAKE, 1999). The clean and straightforward syntax of Python promotes code comprehension and reduces the occurrence of errors (LUTZ, 2013). The active and expansive community surrounding Python has significantly contributed to its growth and development. This community has provided extensive documentation and resources for Python, making it easier for developers to access information and assistance

(PILGRIM, 2009). The availability of tutorials and solutions to common programming challenges has been instrumental in accelerating project progress (GORELICK; OZSVALD, 2014). Python's cross-platform compatibility has been well-documented, allowing for seamless deployment of applications on different operating systems (LUTZ, 2013). Developers can create projects on one platform and run them on various systems, such as Windows, macOS, Linux, and mobile platforms (DRAKE; SURESH, 2016).

In robotics, Python is widely used for developing robot control systems and frameworks. Its ease of use and readability simplify the implementation of complex algorithms and control logic. Python's extensive library support enables seamless integration with various hardware components and sensors commonly used in robotics. Libraries such as Robot Operating System (ROS) provide a robust framework for building robot applications, while libraries like PySerial facilitate communication with microcontrollers and devices. Additionally, Python's compatibility with machine learning libraries allows developers to incorporate advanced perception and decision-making capabilities into robotic systems.

Machine learning, a rapidly evolving field, greatly benefits from Python's extensive ecosystem of libraries and frameworks. Python offers popular machine learning libraries such as TensorFlow, PyTorch, and scikit-learn, which provide powerful tools for building and training machine learning models. These libraries streamline the process of data preprocessing, model creation, and evaluation, empowering developers to implement sophisticated machine learning algorithms with ease. Python's simplicity and readability also contributes to the interpretability and transparency of machine learning code, making it easier to collaborate and understand complex models.

Python's usage in robotics and machine learning is further amplified by its integration with specialized hardware platforms. For example, Python libraries like OpenCV enable computer vision tasks, a crucial component in robotics and machine learning applications. Moreover, Python's compatibility with popular robotic platforms like Arduino and Raspberry Pi enhances its capabilities in the realm of robotics. The extensive ecosystem of libraries and frameworks in Python has been a significant advantage for project development. The availability of well-established libraries, including Django for web development (FORCIER *et al.*, 2009), Pandas for data analysis (MCKINNEY, 2018), and TensorFlow for machine learning (RAMALHO, 2017), has saved developers time and effort by providing pre-existing solutions.

Pygame, a popular library for game development in Python, has garnered attention for its ease of use and functionality. Its intuitive API simplifies game development, allowing developers to focus on game logic and design (SWEIGART, 2012). The Pygame library handles graphics rendering, sound playback, and user input processing, making it a favorable choice for both beginners and experienced developers (SWEIGART, 2012). The

cross-platform support of Pygame is achieved through its integration with the Simple DirectMedia Layer (SDL) library (O'CONNOR, 2019). SDL provides hardware abstraction for graphics, audio, and input, enabling Pygame-developed games to run consistently across different platforms (O'CONNOR, 2019). The extensibility of Pygame allows developers to create more advanced games by integrating additional Python libraries. This flexibility enables the incorporation of functionalities such as artificial intelligence, physics simulations, and networking into Pygame projects (SWEIGART, 2012). The supportive community surrounding Pygame has been essential for developers. Accessible resources, comprehensive documentation, and online forums provide assistance and guidance throughout the game development process (SWEIGART, 2012).

2.8.2 Robot Operating System (ROS) and ROS2

Robot Operating System (ROS) (QUIGLEY *et al.*, 2009) is an open-source framework that provides a collection of software libraries and tools for building robotic systems. It was initially developed by Willow Garage and has gained popularity in both academic and industrial settings. ROS offers a flexible and modular architecture, enabling developers to easily create complex robotic applications by leveraging existing software components and libraries.

ROS2 (MACENSKI *et al.*, 2022) is the next-generation version of ROS, designed to overcome limitations and challenges faced by the original framework. While ROS has been successful, ROS2 improves upon its foundation by enhancing performance, scalability, and usability. It introduces a new communication middleware called Data Distribution Service (DDS), which better supports real-time systems and distributed architectures. ROS2 also offers improved security features, better integration with non-ROS systems, and supports multiple programming languages such as Python, C++, and MATLAB.

One of the key features of ROS2 is its utilization of the Data Distribution Service (DDS) as the underlying communication middleware. DDS is a standard for real-time and scalable data exchange in distributed systems. By incorporating DDS, ROS2 improves support for real-time systems and distributed architectures. DDS enables efficient and reliable data communication between nodes in the ROS2 network, facilitating seamless integration and synchronization of data across different nodes. This robust communication infrastructure enhances scalability and performance, making ROS2 suitable for complex robotic applications. DDS also provides advanced features like data filtering, built-in quality-of-service (QoS) settings, and support for fault-tolerant systems, further enhancing reliability and versatility of data distribution in ROS2. By leveraging DDS, ROS2 provides a solid foundation for building distributed robotic systems requiring real-time communication and coordination between multiple components in a network.

The utilization of ROS in a project provides significant advantages. Firstly, ROS offers a modular and reusable framework that allows seamless combination of independent packages and libraries, facilitating the creation of intricate robotic systems. This streamlines the development process and fosters collaboration within the ROS community. Secondly, ROS abstracts away hardware complexities, presenting a consistent interface for robot control and sensor data access. This abstraction enables researchers to focus on higher-level tasks without being burdened by hardware-specific intricacies. Additionally, ROS supports a wide range of robotic platforms and sensors, making it effortless to integrate new hardware components into the project.

Moreover, ROS provides a comprehensive suite of tools and libraries designed to address various aspects of robotics development. These resources include visualization tools for debugging and monitoring, simulation environments for rigorous testing and validation, and robust libraries for perception, planning, and control. Leveraging these tools expedites the development process and allows researchers to capitalize on state-of-the-art algorithms and techniques. Another advantage of employing ROS is the strong support provided by its community. With an active network of developers and researchers, ROS offers extensive assistance through forums and online resources. This collaborative environment ensures access to solutions for common challenges, shared experiences, and a wide variety of packages and libraries.

Lastly, transitioning to ROS2 provides additional benefits. ROS2 addresses limitations of its predecessor by incorporating features such as real-time and distributed system support, improved integration with non-ROS systems, and enhanced security measures. Overall, ROS empowers researchers to efficiently develop intricate robotic systems and remain at the forefront of advancements in robotics technology.

3 Loyal Wingman Application

In this chapter, we delve into the practical application of Loyal Wingman (LW) systems using Unmanned Aerial Vehicles (UAVs), exploring various aspects related to decision-making and scenario-specific assignments. We start by examining a specific Loyal Wingman scenario and break down the associated problems to gain a comprehensive understanding. Following that, we define the problem statement and highlight the challenges involved in the LW-Kamikaze assignment problem. To assess the effectiveness of LW systems, we establish metrics of interest for evaluating the decision-making capabilities. The chapter then proceeds to discuss the development of decision-making algorithms, starting with the 2D prototype and advancing towards the decision-making processes for Kamikaze UAVs, Loyal Wingman UAVs, and a high-fidelity UAV simulator. Finally, we present the experiments conducted to validate the performance and efficiency of the implemented decision-making.

3.1 Loyal Wingman Scenario

To explore the concept of Cooperative Engagement Capability (CEC) within the Manned-Unmanned Team (MUM-T), we present a defense scenario involving two teams comprising fully actuated UAVs. The MUM-T team consists of Loyal Wingman (LW) UAVs that fly in formation alongside a manned leader, providing escort and defense for both the leader and a designated protected area. In contrast, the adversarial team comprises a swarm of kamikaze threats, as illustrated in Figure 3.1. The primary objective of the MUM-T team is to prevent any damage to the leader or the protected area. The mission is considered unsuccessful if the leader or the protected area are destroyed. Furthermore, to support the MUM-T team members, the protected area is equipped with ground assets capable of providing aerial surveillance.

The LW is autonomous and capable of making intelligent decisions based on the situational awareness information provided by the ground assets considering that they do not have onboard sensors to obtain information about the threats, such as cameras or LIDAR. To be able to neutralize the threats, the LW is equipped with two hypothetical types of

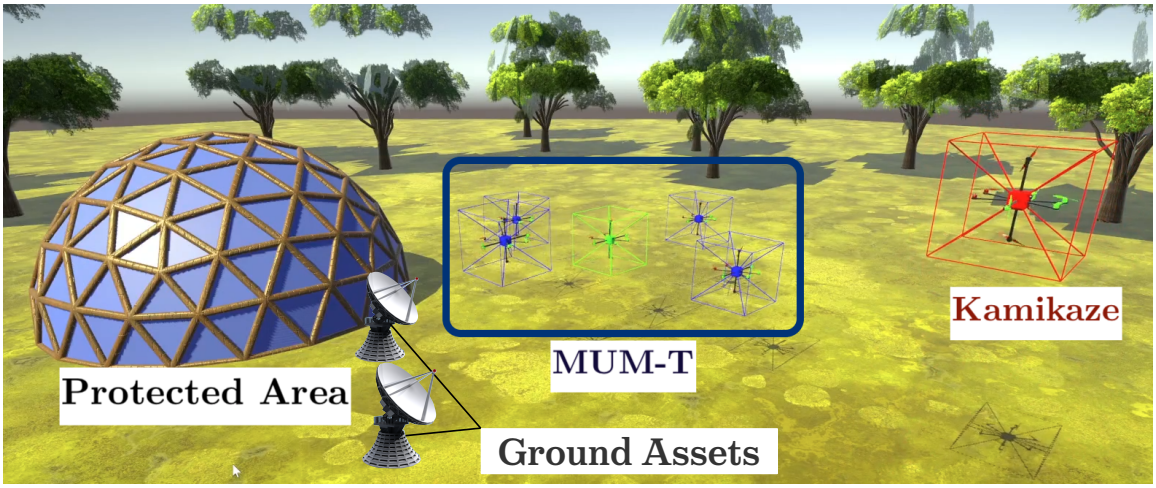

FIGURE 3.1 – Scenario of interest where LW UAVs, highlighted in blue, supported by ground assets, escort a leader UAV and engage kamikaze threats in order to defend a protected area.

weapons, a mid-range *freezing gun* and a short-range *vaporizer gun*, both with a limited number of cartridges and a fixed cool-down time interval, *i.e.*, they become unavailable for a fixed-time period after shooting. The *vaporizer gun* can neutralize the threat, while the *freezing gun* slows down the threat by half of its maximum speed. The weapons' model is simplified, being the hitting success calculated by a given probability. Note that, the *freezing gun* is intended to make the decision space more complex, in practice, this type of weapon is infeasible given current technology, and the *vaporizer gun* is also an idealized weapon that uses energy to destroy only the electronic components of the kamikaze.

The leader is remotely controlled by a human and it is in charge of the formation coordination, *i.e.*, it is capable of passing relative coordinate commands to the LW. As a matter of simplification, we assume that the leader can only command the loyal wingmen to fly within one predefined formation pattern. In this paper, we consider this unique pattern as an uniform-circular formation along the local horizontal plane with a desired radius. We also assume that the leader is always capable to command the loyal wingmen whenever required, *i.e*, it can override the decision-making commands of a specific LW in the MUM-T, either by enforcing the loyal wingmen to return to the formation, or selecting a LW to engage or shoot a specific threat.

On the other hand, the kamikaze UAVs are assumed to explode as soon as they collide with either the leader, an LW, or the protected area. When the explosion is triggered, its effect destroys the target vehicle or damages the protected area. Additionally, once a kamikaze selects a target, it will chase this target until the kamikaze explodes itself or is neutralized by a vaporizer gun. The number of kamikazes is fixed in the simulated scenario, *i.e.*, they immediately re-spawn after being neutralized, thus keeping a steady stream of attacks. The kamikazes have a simplified AI but are assumed to be faster and more numerous than the MUM-T, thus forcing the MUM-T to cooperatively work in order

to effectively neutralize the kamikazes.

The ground assets have a limited detection range and share situational awareness with the MUM-T, *i.e.*, a vector with the state (position, attitude, and linear velocity) of all entities. To simplify the scope of this work, we assume the ground assets to be perfect radars, and that the communication among the MUM-T is perfect, with no delays, packet loss, or bandwidth limitation.

3.1.1 Mission

Therefore, the mission for the Loyal Wingmen is:

The LW shall protect both a protected area and the leading drone against the incursions of a kamikaze swarm.

And, the conditions to end the mission are:

- The leading drone is hit by a single kamikaze drone, or;
- The critical area is hit by five drones, or;
- All loyal wingman were destroyed.

3.1.2 Description of the entities

The following list presents a description of the entities. Note that the parameters are represented using variable names in *snake_case* naming convention.

3.1.2.1 Manned-unmanned teaming (MUM-T)

The MUM-T is composed of `num_loyal_wingman` Loyal Wingman UAVs and a manned-controlled leading UAV. The entities can be described as follows:

- The leading drone is a UAV manually and remotely controlled by a human pilot and represents the commander aircraft;
- Loyal wingman agents are fully autonomous, and there are a fixed number of LW during a simulation;
- The leading drone is in charge of the formation and for passing the formation coordinates to each LW.

3.1.2.2 MUM-T Weaponry

- MUM-T drones have a mid-range freezing gun and a short-range vaporizer gun, both with a limited number of cartridges, given by `cartridges_vaporizing` and `cartridges_freezing`, respectively, and cool-down time of `freezing_cooldown` seconds and `vaporizer_cooldown` seconds, respectively;
- The freezing gun has a range of `freezing_range` meters and slows down the threat by 2/3 of its maximum speed for `time_frozen` seconds with a hitting probability of `freezing_hit_prob`. This weapon intends to make the decision space more complex and in practice the freezing dynamic is infeasible, given current technology;
- The vaporizer gun has a range of `vaporizer_range` meters and destroys the threat with a hitting probability of `vaporizer_hit_prob`.

3.1.2.3 Kamikaze Agents

- The kamikaze drone targets and aims to strike the leading drone, protected area, or an specific LW;
- The Kamikaze drones are embedded with an explosive weapon that is triggered upon contact with their target. This explosive capability not only destroys the targeted drone and itself but also inflicts damage to the protected area.
- It is assumed that the agents can identify these targets, and upon contact, the drone explodes within its range, causing damage to the target;
- Kamikaze drones continuously appear within the range of ground-based radars, so up to `max_num_of_kamikaze` drones are present in the simulated scenario;
- Each Kamikaze drone will randomly select a target when created, and will not change the target until it is destroyed;

3.1.2.4 Ground Assets

- The ground-based radars share situational awareness with MUM-T drones, *i.e.* a vector with the state (pose and velocity) of all entities. In simulation, it is represented as messages carried out between entities.
- Ground-based radars detect and track kamikaze drones up to a limited range of `max_detection_range` meters;
- The communication is abstracted as perfect, with no delays or data losses.

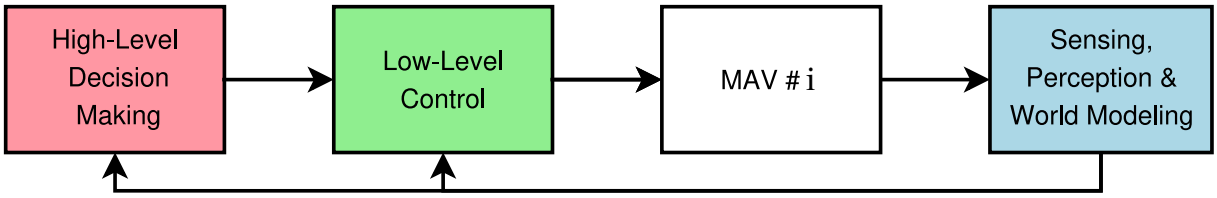

FIGURE 3.2 – Problem breakdown indicating the main software layers.

3.2 Problem Breakdown

In this section, we introduce a problem breakdown according to the scenario of interest. A common approach in robotic cognition involves decomposing the agent into software layers, such as situational awareness, decision-making, and control, as shown in Figure 3.2 for a scenario containing a leader and $n \in \mathbb{Z}_{>0}$ autonomous multi-rotor aerial vehicles (MAVs). In this work, we adopt a similar breakdown and concentrate on high-level decision-making and cooperation as well as low-level path planning, guidance and control tasks, idealizing the situational awareness block in Figure 3.2. To this end, we discuss algorithms in the following sections that rely on modeling algorithms which prepossess raw sensory data and generates a high-level world model. This model includes essential information such as the poses of objects of interest in a global coordinate system.

The HL decision-making layer receives state information from the situational awareness containing the pose and linear velocity of each object of interest in a global coordinate system. Once this information is received, the HL layer process the states using AI-based decision-making algorithms that outputs action commands. There are two types of action commands: rotational and translational movement commands and commands regarding the use of weapons. The LL control, on one hand, translates the movement actions into appropriate state commands that are used to calculate the control input to the MAV, and, on the other hand, allocates the weapon actions using the available resources. Moreover, the MAV module contains the rotational and translational dynamics of the vehicle, the control allocator that translates the control input into actuator commands, the actuator model, and the weapons model. Note that, the weapons' model is simplified, *i.e.*, the aiming and projectile dynamics are instantaneous, being the hitting success calculated by a configurable probability. Lastly, we also assume that the MAVs share a synchronized internal model of the world in the sense that they are aware of the same threats and other agents state due to the ground-assets radars.

Each agent is identified by a unique ID. In this sense, we define a set $\mathcal{I} \triangleq \mathcal{I}_m \cup \mathcal{I}_k$ containing the identifier IDs of all agents, where $\mathcal{I}_m$ is a set containing the identifier IDs of the MUM-T members and $\mathcal{I}_k$ is a set containing the identifier IDs of the kamikazes.

This work addresses only the high-level decision making aspects of the problem breakdown.

3.2.1 Low-Level Control

The low-level layer of this work comprises a collection of techniques based on research papers that serve as the foundation for the control framework proposed in this dissertation. These studies, focus on various aspects of multirotor aerial vehicle (MAV) control, including uncertainties, disturbances, attitude control, and collision avoidance. Regarding the control of our UAVs, the solution implemented is the Sliding Mode Control (SMC) strategy, discussed in detail in Ricardo *et al.* (2023) and in Subsection **??**, which offers robustness and adaptability to uncertainties. This approach allows for precise and stable control, enhancing the UAVs' overall performance. For collision avoidance, the solution employes the Acceleration-Velocity-Obstacles (AVO) method (RICARDO *et al.*, 2023), described in Subsection 2.6.2, which combines velocity and acceleration constraints with obstacle information.

One of the key studies for the low-level layer is the work by Ricardo e Santos (2022d). This paper provides insights into the control of fully actuated MAVs with fixed rotors, considering uncertainties and disturbances. The authors derive dynamic equations and propose a control law based on geometric principles and second-order sliding mode strategy, ensuring robustness and convergence. Through simulations, they demonstrate the superiority of their method over existing techniques, validating its potential for practical implementation.

Another significant contribution in this layer is the study conducted by Ricardo e Santos (2020). Their work addresses the challenges of attitude control for quadrotor UAVs in the presence of disturbances and uncertainties. By modeling the vehicle's dynamics using Gibbs vector and angular velocity, the authors propose a first-order adaptive Sliding Mode Control (SMC) strategy with a gain adaptation law. Their approach guarantees convergence and ideal sliding mode existence. Numerical simulations using a flight control simulator demonstrate the effectiveness and ease of implementation of their proposed strategy.

Regarding the position guidance and control, Ricardo e Santos (2022b) present an approach for robust control of MAVs in dynamic environments. Their work encompasses an outer-loop guidance strategy based on Acceleration-Velocity-Obstacles (AVO) and an inner stabilizing control loop utilizing integral sliding mode. Through numerical evaluations involving an octacopter, the authors showcase the effectiveness of their proposed method in achieving robust position control in the face of dynamic environmental conditions.

Addressing the critical aspect of collision avoidance, Ricardo e Santos (2023a) propose a method suitable for mobile robots with uncertain dynamics operating amidst dynamic obstacles. Their approach leverages a robust continuous-control-obstacles technique, incorporating an overdamped low-pass filter for generating smooth position commands and

a high-order sliding mode differentiator to estimate obstacle accelerations. Numerical evaluations conducted using a quadcopter navigating through moving obstacles validate the method's effectiveness in achieving collision avoidance while adhering to velocity constraints.

In summary, the low-level layer of this work consists of the cited research papers, which form the basis for the control framework developed in this dissertation. These studies contribute with methodologies for addressing challenges related to uncertainties, disturbances, attitude control, and collision avoidance. This dissertation seeks to enhance the field of decision-making for drones by employing and expanding upon the methodologies proposed in the cited works. The primary objective is to construct a high-fidelity UAV simulator that can effectively test and validate the decision-making capabilities of the agents.

3.3 Problem Definition

In this section, the problem to be solved by the decision-making of LW agents is described.

As mentioned, the main mission of the LW agents is to defend both the protected area and the remotely-controlled leading drone against multiple incursions from a swarm of kamikaze drones. Therefore, a LW must effectively engage and disable threats to ensure the safety of both. Furthermore, for this mission to be successful, LW UAVs are expected to have the ability to make intelligent decisions autonomously. Thus, we can define necessary capabilities for this agent, such as the ability to fly in formation with cohesion, and the ability to engage and neutralize imminent threats to guarantee the defense of both the protected area and the leader. To effectively achieve neutralization, agents must have the capability to use embedded weaponry, *i.e*, use weapons intelligently and strategically, and in extreme cases, sacrifice themselves to neutralize a threat.

The main problem of the HL decision-making can now be defined.

Problem I. The main problem is to develop an autonomous intelligence module for the LW MAV to successfully achieve the mission objective, aiming at the smallest loss of LW agents during the attack and defense maneuvers.

Based on this problem it is identified a need for defining a set of expected behaviors for the agents. Therefore, we define the following subproblem:

Subproblem I.1. Design basic behaviors for the LW agents and to develop a decision-making architecture to coordinate these behaviors.

To tackle Subproblem I.1 we also need to design the adversary agents present in the scenario, so we can properly evaluate our solution to Problem I.

Subproblem I.2. Design basic behaviors for the kamikaze agents and to develop a decision-making architecture composed of these behaviors.

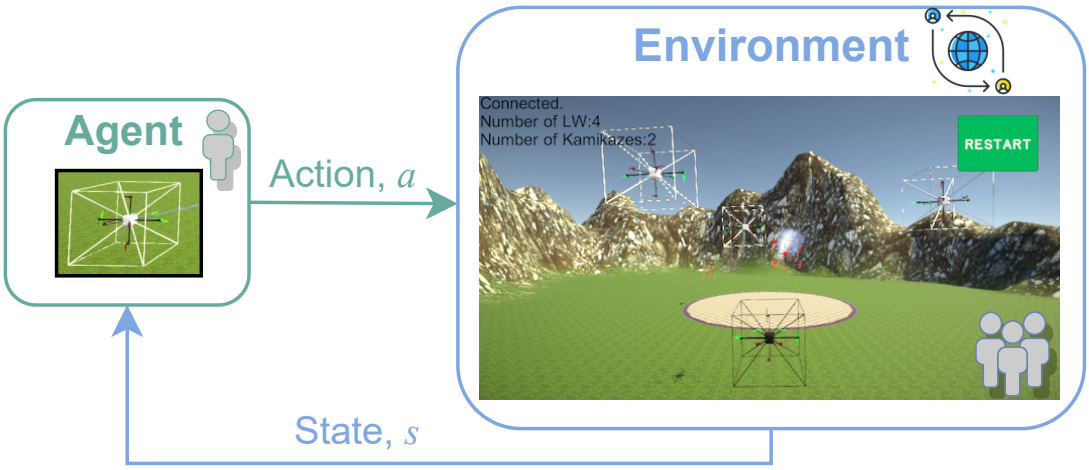

FIGURE 3.3 – Problem I definition diagram.

Therefore, the main agent in our problem is the LW UAV, who will have to perform actions based on the AI-based decision making, as seen in Figure 3.3. The action $a \in \mathcal{A}$, where $\mathcal{A}$ is a set of possible actions, is performed by the agent in a simulated environment which in turn will send the agent's state $\mathbf{s} \in \mathcal{S}$ as feedback. Note that the environment is the *Defense Scenario* and it encompasses all loyal wingmen agents, the leader, and its kamikaze adversaries.

3.3.1 LW-Kamikaze Assignment Problem

One of the subproblems to be also addressed is the allocation (or assignment) of tasks among the LW MAVs. By task we mean the neutralization of a specific kamikaze threat.

Despite the LW being autonomous, the agents must act cohesively so that cooperation between the MUM-T can arise. One way to approach this is that the leader UAV must be able to command the LWs when needed, so he must be able to distribute tasks to the MUM-T members. This is known as the classical assignment problem which is a widely-studied problem applicable to many domains (BURKARD; ÇELA, 1999). It is the problem of choosing an optimal assignment of $n \in \mathbb{Z}_+$ workers to n tasks, assuming that numerical ratings, or costs, are given for each worker's performance on each task. An optimal assignment is the one that makes the sum of the workers's costs for their assigned tasks a maximum (or minimum) (MUNKRES, 1957). Note that there are $n!$ possible assignments, of which several may be optimal. In this sense, consider the following subproblem.

Problem II. Given a set of workers $\mathcal{W}$, a set of tasks $\mathcal{J}$, and a set of costs $\mathcal{C}$ indicating how effectively each worker $w_i \in \mathcal{W}$, where $i \in \{1, \ldots, n\}$, can perform each task $j_i \in \mathcal{J}$ determine the best possible assignment of workers to tasks, such that each task is assigned to one worker and each worker is assigned one task, so the total cost is minimized.

One wishes to choose a set of n independent elements $(c_{i,j})$ of a cost matrix $\mathbf{C}$, where $c_{i,j}$ is the element of the ith row and jth column of $\mathbf{C}$, so that the sum of these elements is minimized. This can be expressed as permuting the rows and columns of $\mathbf{C}$ to minimize the trace of a matrix $\min_{\mathbf{L},\mathbf{R}} \mathrm{Tr}(\mathbf{LCR})$, where $\mathbf{L}$ and $\mathbf{R}$ are permutation matrices, and the cost matrix is

$$\mathbf{C} = \begin{array}{c} \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{array} \begin{array}{c} \begin{array}{cccc} j_1 & j_2 & \cdots & j_n \end{array} \\ \begin{bmatrix} c_{1,1} & c_{1,2} & \cdots & c_{1,n} \\ c_{2,1} & c_{2,2} & \cdots & c_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n,1} & c_{n,2} & \cdots & c_{n,n} \end{bmatrix} \end{array}. \tag{3.1}$$

A well-known solution to the assignment problem is given by the Kuhn-Munkres algorithm, colloquially known as Hungarian algorithm, proposed by H. W. Kuhn (KUHN, 1955) and refined by J. Munkres (MUNKRES, 1957). This algorithm solves the assignment problem assuming the existence of a cost matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$, where the element in the ith row and jth column represents the cost $c_{i,j}$ of assigning the jth task to the ith worker.

3.4 Decision-Making Development

In this section, we present the solution to the decision-making problems, described in Section 3.3, by combining decision-making modules with a distributed coordination strategy. This approach enables each agent to make informed decisions while effectively coordinating their actions with other agents.

To address Problem 2, we will discuss the methodology used, focusing on the AI of both teams of agents present in the scenario of interest. Firstly, we will tackle the development of decision-making for the adversarial agents, specifically the Kamikaze UAVs.

Subsequently, we will shift our attention to the development of decision-making for the Loyal Wingman (LW) UAVs. The decision-making process for the LW is the main focus of this part of the work due to its inherent complexity. Initially, we will develop rules of engagement to formalize the expected behaviors for the agent. From these rules, we will derive modular behaviors that compose an AI architecture using the Behavior Tree technique.

This section will emphasize the development of the behavior architecture for both the Kamikaze and LW agents. The behavior design approach relies on empirical evidence

and the implementation of hand-coded heuristic rules, as opposed to learned behaviors through reinforced learning. The behaviors are determined empirically through a systematic process of trial-and-error. Furthermore, we will discuss the approach to cooperation within the swarm, utilizing the Munkres algorithm for task allocation. Finally, we will formalize the selected formation for the team.

3.4.1 Decision-Making for the Kamikaze UAV

To address Problem I effectively, we need to tackle Subproblem I.2 first. This subproblem requires the functionality of kamikaze agents, which can be achieved through simplified yet effective AI. For this purpose, we have chosen the Finite State Machine (FSM) technique, introduced in Subsection 2.2.1, to develop the decision-making module for the kamikaze agents due to their straightforward implementation and intuitive structure for simpler decision-making modules. FSMs are widely used for modeling decision-making architectures in simple AI systems (COLLEDANCHISE; OGREN, 2018; GIACOMOSSI *et al.*, 2021b).

In the FSM technique, each state (denoted as s) represents a specific behavior for the agent. A FSM can transition between different behaviors in response to events. In Figure 3.4, we present the decision-making module that has been developed for the kamikaze UAV. Notice that each UAV is equipped with an identical decision module.

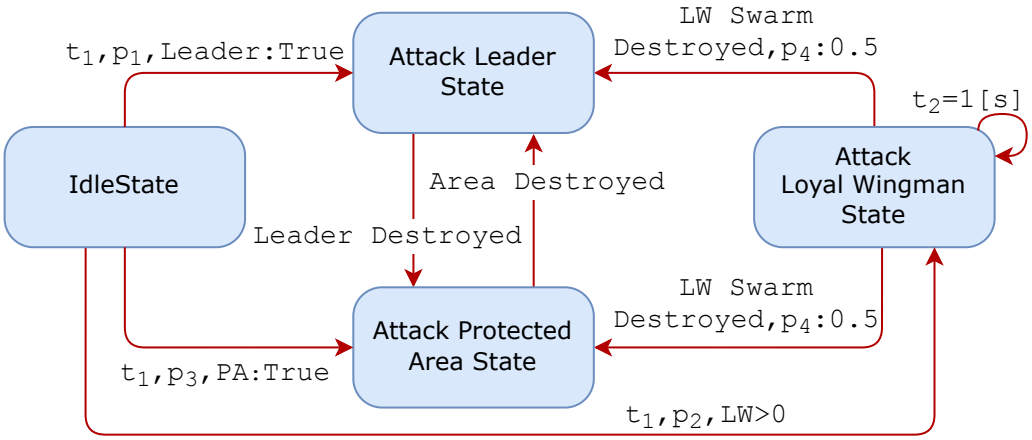

FIGURE 3.4 – Decision-Making module for a kamikaze UAV using FSM.

In our solution, the kamikaze has a set $\mathcal{K}$ of four behaviors, described in Table 3.1. The initial behavior is *IdleState*, where the agent is idle for t_1 seconds and then selects a target to attack based on the probabilities p_1, p_2, and p_3, which in this work are equally distributed probabilities used for each target type. Once a target is selected, the FSM will transition to the representative state of the selected target and will remain in that state until the agent is destroyed. The exception is *Attack Loyal Wingman*; this behavior

targets the closest LW, and will select the closest LW every $t_2 = 1$ s, to avoid frequent target switching. Once there are no more LW, the FSM transitions to attack the leader or the protected area with a probability of 50% each.

TABLE 3.1 – Behavior description for a Kamikaze drone.

Behavior	Description
Wait State	Default behavior, a target is randomly selected.
Attack Leader	The kamikaze selects the Leading Drone as target.
Attack Protected Area (PA)	The kamikaze selects the Protected Area as target.
Attack Loyal Wingman (LW)	The kamikaze selects the closest Loyal Wingman as target.

Note that kamikaze operate with only a decentralized decision-making capability, preventing direct AI-driven collaboration to counter threats. Nonetheless, this limitation does not deter them from potentially converging on the same target independently, thus engaging in a form of indirect cooperation. Furthermore, also note that drones are equipped solely with explosive capabilities as their primary weaponry.

3.4.2 Decision-Making for the Loyal Wingman UAV

In contrast, the behaviors to be executed by the LW MAV are more complex given their desired capabilities, consequently, its AI architecture requires further elaboration. To effectively solve Problem I and Subproblem I.1, we implement its decision-making module using the BT technique. This decision was made based on the inherent advantages that BTs offer when compared to FSMs, as mentioned in Subsection 2.2.2.

3.4.2.1 Rules of engagement

To gain a preliminary understanding of the general behavior of a Loyal Wingman and develop an AI that exhibits the desired behavior, we have established a set of engagement rules. These rules serve to formalize the functioning of these agents, enabling us to analyze and create an AI system that accurately represents the desired behavior. By adhering to these rules, we can lay the foundation for a comprehensive understanding of how loyal wingmen operate and ensure the decision-making aligns with the intended objectives. The rules are the following:

- By default, the LW must maintain a flying formation surrounding the Leading Drone while no threats are detected.
- When a kamikaze drone approaches the weapons range, the LW agent selects an appropriate neutralization method.

- The available neutralization methods include the vaporizer gun for short-range engagements, the freezing gun for mid-range engagements, and as a last resort, sacrificing the LW agent when no ammo is left.
- The LW agent will break away from the formation and engage a threat if it enters the designated range. In such cases, the LW will transition to pursuing the threat, aiming to approach it and neutralize it. However, it will only do so if the vaporizer gun is available.
- The LW agent identifies the closest kamikaze drone within the range of its freezing gun and attempts to freeze it. However, this action is only taken if the freezing gun is available.
- The LW agent selects the nearest kamikaze drone within the range of its vaporizer gun and neutralizes it. This action is performed only if the vaporizer gun is available.
- The LW must rejoin the formation to protect the leader once a pursued threat has been destroyed or is no longer within range.
- A weapon is considered available if it has remaining ammo and is not currently cooling down.

Based on the rules of engagement, we can derive an initial set of behaviors for the LW agent. These behaviors are summarized in Table 3.2.

TABLE 3.2 – Behavior description for an LW UAV in the 3D environment.

Behavior	**Description**
Chase Threat	The Loyal Wingman leaves the formation to pursue a threat in order to neutralize it.
Go To Formation	The Loyal Wingman returns to formation.
Approach Formation	The Loyal Wingman approaches the leader from a safe distance from the formation.
Vaporize Threat	Performs an attack using the vaporizer gun.
Freeze Threat	Slows down a kamikaze for a limited period of time.
Sacrifice Attack	When there is no ammunition left, the LW will disable the threat by sacrificing itself as a weapon of last resort.

3.4.2.2 Entity Formulation Using Use-Case

Use cases play a crucial role in software engineering as they provide a means to model a system by presenting a coherent story of the system's behavior (SOMMERVILLE, 1992). They serve multiple purposes such as documenting system requirements and facilitating

the understanding of the system's context. A use case typically comprises a scenario that represents an instance of the system's usage, based on specific inputs from the actors or conditions in the environment. Each scenario describes various ways the system can behave, including failure or exception cases. In this section, we present a use case model that illustrates certain behaviors from the list of agent behaviors described in Table 3.2. This use case will be utilized during the initial development of the simulated environment and is used to illustrate a task for the agents.

Table 3.3 outlines the following use case scenario where a kamikaze threat enters the protected area, prompting the mobilization of a Loyal Wingman to neutralize the threat. The Loyal Wingman UAV departs from the protective formation around the leader and engages the threat using its available weapons. Once the threat is successfully neutralized, the Loyal Wingman UAV returns to the formation.

TABLE 3.3 – Use Case description for a Loyal Wingman UAV.

ID:	**Title:**
US01	Eliminate a kamikaze threat
Main Actor:	
Loyal Wingman	
Brief Description:	
As a *Loyal Wingman*, I want to eliminate *threat*, so that the *Leading Drone* and the *protected area* are secure.	
Detailed Description:	
• A *Kamikaze* threat enters the *protected area. *(Use case trigger)* • The *Ground assets* identify this new *threat* and communicate its coordinates to the *loyal wingmen* and the *Leading Drone*. • The *Loyal Wingman* receives information about a new *threat* identified by the *Ground assets*. • The *Loyal Wingman* closest to the *threat* makes a decision and decides to attack the *threat*. • The *Loyal Wingman* informs the *Leading Drone* of his intention to attack and requests permission to leave the formation. • The *Leading Drone* receives the request and allows the *Loyal Wingman* to neutralize the threat. • The *Loyal Wingman* leaves formation and initiates *threat* pursuit. • The *Leading Drone* redefines the formation of remaining *loyal wingmen* in formation to avoid breaches in defense. • The *Loyal Wingman* approaches the *threat* and performs a *freezing gun* attack to slow down the *kamikaze*. • The *Loyal Wingman* approaches the *threat* from a distance that allows the use of the *vaporizer gun*. • The *Loyal Wingman* attacks with the *vaporizer gun* and eliminates the *threat*. • The *Loyal Wingman* requests permission from the *Leading Drone* to return to formation. • The *Lead Drone* receives the request and allows the *Loyal Wingman* to return to formation. • The *Leading Drone* sets a new formation and sends the positions in formation to each *Loyal Wingman*. • The *Loyal Wingman* returns to formation.	
Acceptance Criteria:	
The *Loyal Wingman* must defend the *protected area* and the *Leading Drone* from the *kamikaze* attack without taking any damage.	

3.4.2.3 Behavior Tree development for the Loyal Wingman UAV

Therefore, based on the Problem 2 description, Table 3.2, and guided by the use case presented in Table 3.3, we have identified the modular behaviors for the decision-making of the LW. The six behaviors can be separated into two groups, three movement behaviors: `Chase Threat`, `Go To formation`, and `Approach Formation`; and three offensive behaviors: `Vaporize Threat`, `Freeze Threat` and `Sacrifice Attack`.

Utilizing a systematic empirical and incremental methodology, we have derived a behavior tree architecture for loyal wingmen by employing the modular behaviors described in Table 3.2. The resultant architecture is represented in Figure 3.5. Note that each

LW possesses a distinct and autonomous instance of this decision-making module. This architecture represents the culmination of iterative efforts in refining and enhancing the behavior tree, incorporating insights gained throughout the experimental developmental process using both simulators developed for this project.

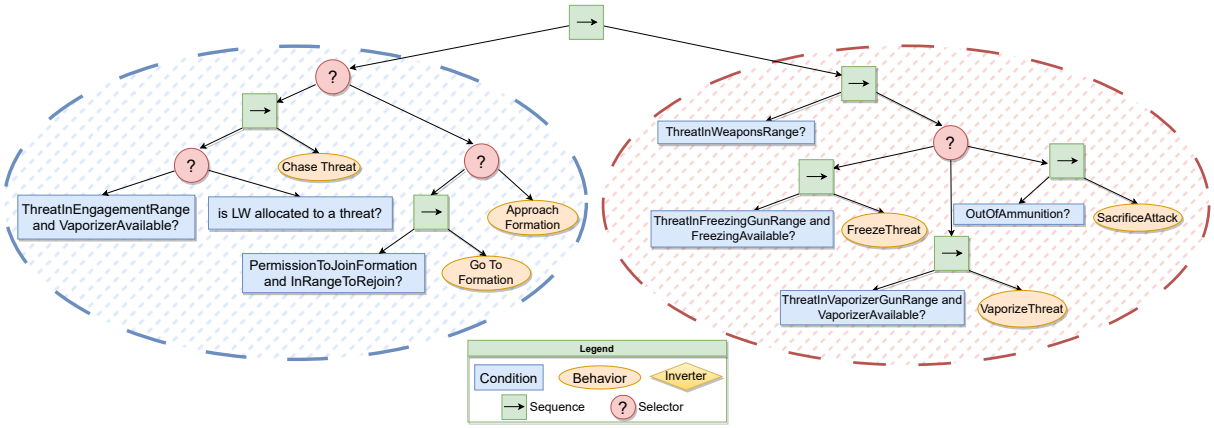

FIGURE 3.5 – Behavior tree for the loyal-wingman UAV. The blue-circled behaviors represent movement behaviors, and the red-circled behaviors depicting offensive behaviors.

This decision-making module will run by default the `Go To Formation` behavior, where the LW keeps a flying formation surrounding the leading drone while there are no threats identified. Whenever a kamikaze drone crosses the engagement range or the LW is allocated to a task, it enters into the `Chase Threat` behavior, for which the vaporizer weapon must be available. In this case, the agent leaves formation to pursue the threat and tries to neutralize it using available weapons. Once a kamikaze is in range of use of the weapons, the LW selects a neutralization method. The selected method may be the vaporizer gun, the freezing gun or a sacrifice as last resort when there is no remaining ammo. A weapon is *available* if there is still ammunition and if it is not cooling down after being fired. Note that the engagement distance and the range of use of both weapons are decision-making parameters, with the latter being less or equal than the nominal range of the weapons, thus leaving the use of weapons as a strategy criterion.

The neutralization strategy embedded in this BT is to first select the closest threat within range of his freezing gun (mid-range) and then try to freeze it. Once the threat is slower, it is safer to approach it, then the LW uses its vaporizer gun (short-range) to eliminate the threat. Note that the respective gun must be available at the time that this weapon needs to be used. The `SacrificeAttack` behavior is executed when there are no remaining ammo in both weapons, it causes the LW to collide with an imminent threat, thus using its body-frame as a protective weapon. Immediately after the threat is eliminated or becomes out of range, the LW returns to formation, thus ensuring the protection of the leader. In order to join the formation, the LW executes the `Approach Formation` behavior, that follows the sequence present in Figure 3.6. Initially, the LW

must approach the formation from a safe distance $d_f \in \mathbb{R}_{>0}$ and send a request to the leader requesting permission to rejoin. The leader receives and processes the request, allowing or not the entry into formation. The LW waits for the permission to be granted and for the coordinates in the formation. The safe distance d_f is a parameter.

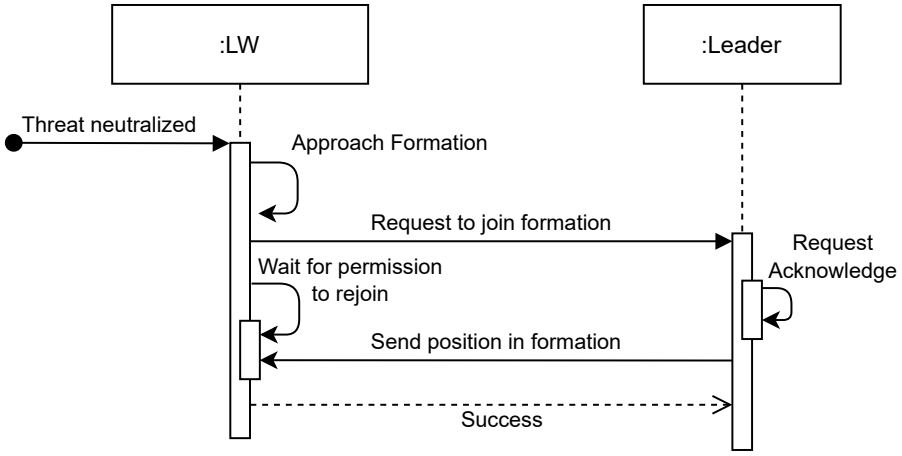

FIGURE 3.6 – Join formation sequence diagram.

Note that the decision-making modules described in this section make decisions based only on its internal state and the states of the other agents present in the scenario, which are received through messages from the ground assets. Also, it is important to reiterate that the leader is capable of overriding the decisions made by the module of a specific LW, by either enforcing the loyal wingmen to return to formation, or selecting a specific LW to engage a selected kamikaze.

3.4.2.4 Cooperation and Selfishness Behaviors

The cooperative behavior of the UAVs in our system is illustrated in Figure 3.7. In frame (a), the leader allocates LW_1 to neutralize the threat K_4. Here, LW_1 acts altruistically for the benefit of the leader's defense. We also see the presence of another threat, K_1, in this frame. During the displacement towards K_4 shown in frame (b), K_1 enters LW_1's selfish engagement radius (d_s). At this point, LW_1 identifies the presence of a threat within d_s and starts behaving selfishly. It decides to temporarily overwrite the leader's command and neutralize K_1, which poses a great risk to itself. Finally, in frame (c), once K_1 has been destroyed, LW_1 returns to its altruistic behavior and continues to neutralize the threat assigned by the leader.

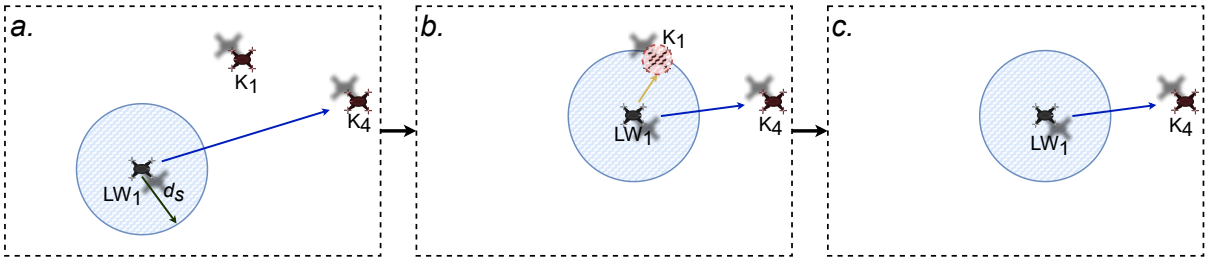

FIGURE 3.7 – Altruistic and Selfish behavior sequence.

Adaptability and balancing of altruistic and selfish behaviors are critical for many robotics applications such as search and rescue missions. The research conducted by Roesner *et al.* (2019) showcases the ethical capabilities of a swarm of rescue drones. These drones can exhibit altruistic behavior by prioritizing the swift rescue of victims. Additionally, they can also demonstrate egoistic behavior, focusing on prolonging their own activity and emphasizing exploration of the surrounding area rather than immediate rescue efforts.

Our approach to cooperative robot defense scenarios has demonstrated the LW's ability to balance between these behaviors according to the situation's demands. The inspiration for this approach came from the well-known technique of potential fields in robotics, which is widely used for navigation and obstacle avoidance (BARNES, 2018). The potential field approach has been extended and applied in various forms for cooperative robotics, including drone swarm formation (SUN *et al.*, 2020) and multi-robot systems (ZHANG *et al.*, 2010).

Our developed decision-making approach enables the LW to behave both altruistically and selfishly, depending on the circumstances. The altruistic behavior is evident when the LW neutralizes a threat assigned by the leader, even if it puts itself at risk. However, the LW can exhibit selfish behavior as well. For instance, when a threat enters the selfish engagement radius, the LW identifies the threat's existence and starts behaving selfishly by temporarily overwriting the leader's command and neutralizing the highest risk threat, which is the one closest to it. Once it neutralizes the threat, the LW returns to its altruistic behavior and cooperates with the system as a whole, working to neutralize the threat allocated by the leader.

3.4.2.5 LW-Kamikaze Task Allocation

Now, let us address the solution of Subproblem 2.3 by employing the Hungarian algorithm, introduced in Subsection 2.4.2, for the task assignment between the LW. The Hungarian algorithm is a widely recognized and extensively studied method in the literature, known for its efficiency in solving the assignment problem and guaranteeing optimal assignment of tasks to resources.

As seen in Figure 3.8, the workers in our context are interpreted as the LWs available and the tasks are the neutralization of the kamikaze threats. The distance of the loyal wingman to a threat can be described by the euclidean distance $d(\mathbf{p}, \mathbf{q}) \triangleq \|\mathbf{p}_i - \mathbf{q}_j\|$, where $\mathbf{p}_i \in \mathbb{R}^3$ and $\mathbf{q}_j \in \mathbb{R}^3$ are the current position of the ith LW and the jth kamikaze threat, respectively. So the cost of the ith LW to neutralize the jth kamikaze is

$$c_{i,j} \triangleq \begin{cases} d(\mathbf{p}_i, \mathbf{q}_j) + A_v P, & d(\mathbf{p}_i, \mathbf{q}_j) \leq d_a, \\ +\infty, & d(\mathbf{p}_i, \mathbf{q}_j) > d_a, \end{cases}$$

where $A_v \in \{1, 0\}$ is a variable that is 1 if the LW weapons are available, and 0 otherwise. The constant P represents an additional penalty cost to assign a LW that does not have an available weapon, as it is important to avoid the allocation of unavailable LWs. Lastly, the engagement range is represented by the radius $d_a > 0$, and the threats outside this range will not be considered during the assignment.

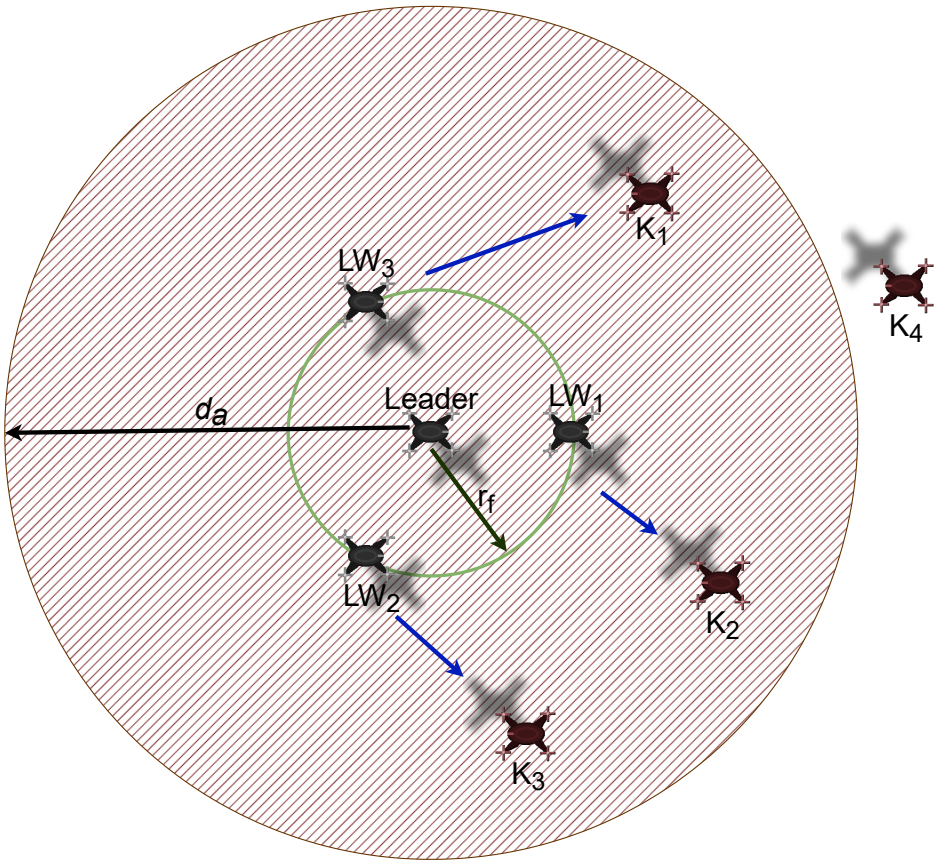

FIGURE 3.8 – Diagram of LW-Kamikaze task allocation problem.

The execution of the Hungarian algorithm outputs the assignment of the lowest cost, through the $\mathbf{C}$ matrix, representing the optimal LW-Kamikaze assignment. In the example provided in Figure 3.8 we have the optimal LW-Kamikaze task allocation indicated by the blue arrows, note that the threat K_4 is not considered, since it is outside d_a.

3.4.2.6 Formation Definition

In this work, we consider that the leader defines the formation pattern as an uniform-circular formation along the local horizontal plane with a desired radius $r_f > 0 \in \mathbb{R}_{>0}$, as shown in Figure 3.9. Given that there are a total of $n_f \in \mathbb{Z}_{>0}$ LW in formation, the positions occupied by the Loyal Wingmen are defined by the set

$$\mathcal{P}_f \triangleq \left\{ \mathbf{r}_g^{\ell/g} + r_f \mathbf{D}_3(j\alpha + \phi)\mathbf{e}_1 \right\}_{j=1}^{n_f}, \tag{3.2}$$

where $\mathbf{r}_g^{\ell/g}$ represents the position vector of the leader. $\mathbf{D}_3(\cdot)$ represents a rotation matrix around the $\vec{z}_g$ axis. This matrix is used to rotate the positions of the LW within the formation. The specific rotation angle is given by the expression $j\alpha + \phi$, where j varies from 1 to n_f (the total number of LW in the formation), $\alpha = 2\pi/n_f$ is the step angle, and $\phi \in [-\pi, \pi)$ is an offset angle that can be used to rotate the formation around the leader. Finally, $r_f \mathbf{D}_3(j\alpha + \phi)\mathbf{e}_1$ represents the vector pointing in the direction of the first axis ($\mathbf{e}_1$) after applying the rotation matrix $\mathbf{D}_3(j\alpha + \phi)$ and scaling it by the radius r_f. This vector defines the position of each LW relative to the leader.

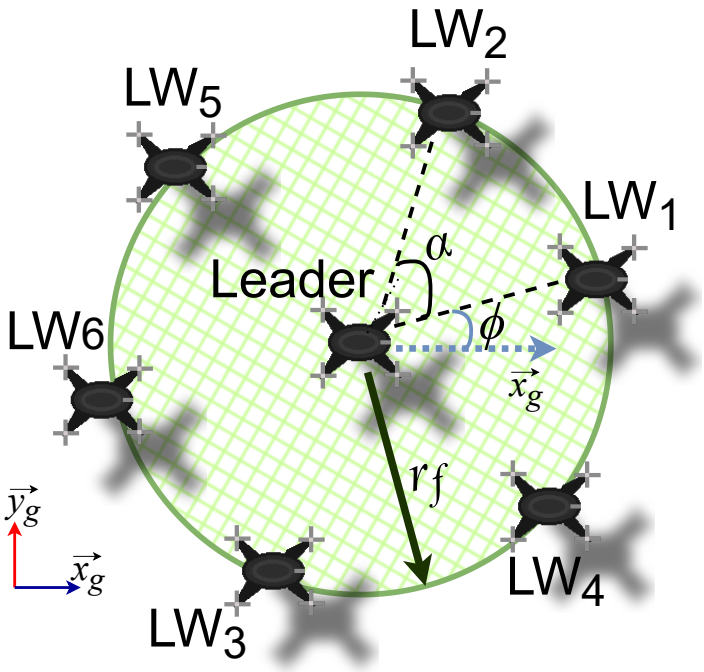

FIGURE 3.9 – Uniform-circular formation pattern.

Notice that the assignment of the LW to the available positions in formation is sequential, based on the order of the acknowledged request to join the formation by the leader, and not necessarily match the vehicle ID.

3.5 Drone Swarm Simulation

In this section, we introduce two prototypes that have been developed for the purpose of our research project. The first prototype is a 2D model which provides a high-level investigation of the problem under study. The second prototype, on the other hand, is a 3D model with high-fidelity dynamics. The development of these two prototypes has been driven by the need to explore the problem space from multiple perspectives and to test our hypotheses in different environments. By utilizing these prototypes, we aim to gain a better understanding of the problem, as well as to evaluate the feasibility of potential solutions. In the following sections, we will provide a description of each prototype and the methodology used in their development.

3.5.1 High-Level 2D Prototype

A simulator based on *Python* was entirely created for the purpose of enabling an initial study of the context, in order to obtain the agent architectures for the kamikazes and the LW, as well as some preliminary results and parameter identification. The simulator uses open-source code that is both user-friendly and customizable. The *Pygame* library was utilized for visual representation and user interaction. The simulator's focus is on evaluating decision-making techniques through the implementation of a defensive scenario that abstracts a simulation with simplified physics.

The prototype developed serves as a versatile tool with the capacity to generate statistical data and execute optimization algorithms for the Loyal Wingman scenario. For the purposes of this study, we implemented the Particle Swarm Optimization (PSO) algorithm, introduced in Section 2.4.1. However, it is important to note that the simulator is adaptable and can accommodate the integration of alternative optimization algorithms, depending on specific requirements of future research extensions. The techniques used in creating the simulator are described in Section 2.8, while further implementation details can be found on Github[1] for interested readers.

3.5.1.1 Graphical Interface

A running simulation is seen in Figure 4.1. The scenario region is an open area of 17,100 m^2, divided into cells (9 x 19), where each square is equivalent to 100 m^2 (10 x 10 m). Drones are assumed to fly at altitudes of approximately 40 m.

The graphical interface is composed of three main elements as seen in Figure 4.1. The green line is used to delimit the area where the kamikaze drones will appear. The simulator

[1] https://github.com/luizgiacomossi/Simulation_Wingman/tree/loyal_attacking

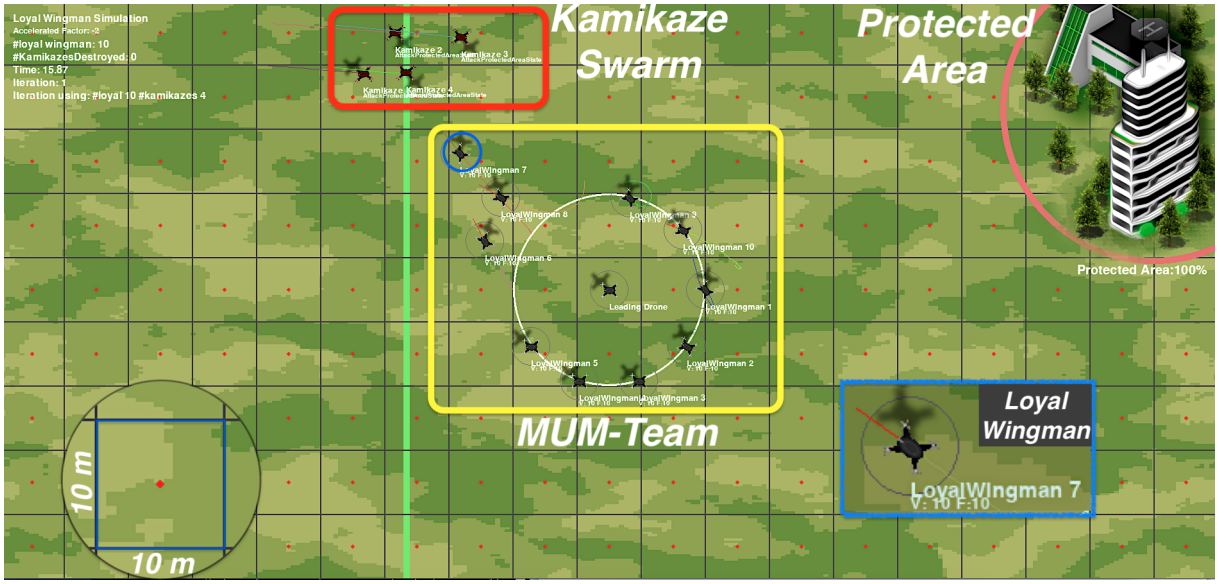

FIGURE 3.10 – Simulator view, we see the MUM-T being attacked by a kamikaze swarm. The protected area is seen on the upper-right corner. A LW is seen up-close.

also presents data about an iteration in the upper left corner, such as the number of drones on both teams, number of kamikazes destroyed by the MUM-T, execution time, iterations executed and number of LW remaining.

The user can control the position of the leading drone by point-and-click actions and the Loyal Wingman will follow in formation. The kamikaze and the Loyal Wingman drones are autonomous and the user has no control over them. The simulation run-time is adjustable in order to facilitate parameter optimization and can be changed by the user to run faster than real time.

Figure 3.11 provides a close-up view of the MUM-T under attack, with two groups of Loyal Wingmen leaving the formation to neutralize the threats. Loyal Wingman 7 can be seen using a vaporizer gun to destroy a kamikaze, while Loyal Wingman 5 employs a freezing gun. It is worth noting that each drone is equipped with a collision avoidance radius represented by a gray circle, and a red vector indicating the direction and velocity (magnitude is proportional). Additionally, a label displaying the number and type of ammunition available for both weapons is also provided for each drone. Also, note that there are no obstacles in this scenario, as the primary goal revolves around studying the offensive and defensive strategies employed by the participating teams.

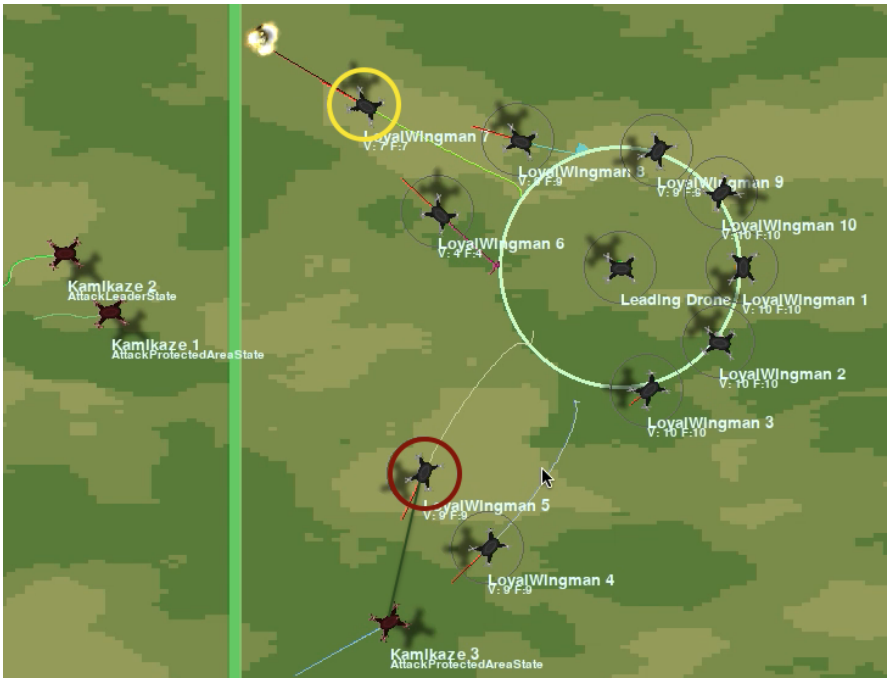

FIGURE 3.11 – MUM-T under attack of a kamikaze swarm, leader at the center and Loyal Wingmen in formation. LW in yellow destroying a kamikaze (vaporizer gun). LW marked in red is slowing down a kamikaze (freezing gun).

Lastly, it is worth noting the effective functioning of the potential fields technique for collision avoidance. Despite its simplicity, the technique played a crucial role in preventing loyal wingman agents from colliding with each other, and with the leader, thus enabling efficient team defense maneuvers.

3.5.2 High-Fidelity 3D Simulation

This section outlines the simulation and adaptation process for the high fidelity simulator developed in this project. As described in Section 3.2, our solution consists of two main modules: a high-level module responsible for decision-making by the agents, and a low-level module that handles path planning, collision avoidance, and control of the simulated UAVs.

The high-level module is a composite of decision-making algorithms that control the behavior of the agents. Meanwhile, the low-level module handles the technical details of controlling the UAVs, such as determining their optimal path and collision avoidance with other objects in the simulation. By integrating these two modules in a virtual environment, we have created a comprehensive simulator that represents a high-fidelity scenario, which allows us to test various scenarios and make data-driven decisions about how to optimize the behavior of our agents.

In all simulations, the UAVs used are non-planar fully actuated octocopters, similar

to the one depicted in Figure 3.12. These octocopters have a total mass of 1 kg, an arm length of 0.5 m, and an inertia matrix given by $\text{diag}(0.015, 0.015, 0.015)\ kg \cdot m^2$. For a comprehensive description of the dynamics and control methods employed for this model, please refer to the relevant details provided in Giacomossi *et al.* (2022b).

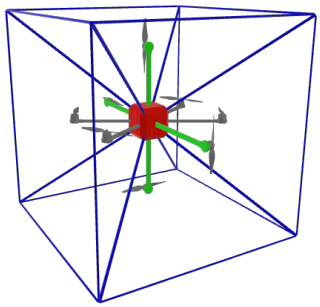

FIGURE 3.12 – Schematic representation of a non-planar fully actuated octocopter. To enhance safety during takeoff, landing, and general operation, a protective cage is depicted as a blue box surrounding the drone. This cage serves the purpose of safeguarding the rotors and providing overall protection to the system.

3.5.2.1 Computational Architecture

This section describes the computational architecture used to implement the proposed scenario. A simulation framework is built as a proof of concept for the development of the proposed techniques. The infrastructure is divided into two computers, as seen in Figure 3.13. The first one is responsible to compute the LL control, physics, 3D visualization, and interface with the human pilot through a joystick. The second one is responsible to run the HL decision-making modules of the agents being simulated. The computers communicate on a local network using the Robot Operating System 2 (ROS2) middleware, which is built on an open-source data distribution standard (DDS) middleware that provides features such as discovery, serialization, and transportation[2].

The LL control, physics, and interface with the human pilot through a joystick are coded in MATLAB. The choice of implementing in Matlab is driven by the fact that it is the primary language supported for control techniques, and it benefits from extensive support within the research community. The 3D simulator was developed using the Unity engine in C#. On the other hand, the decision-making algorithms were completely implemented in the Python programming language, due to its large community, support, and availability of libraries, especially for machine learning.

The choice of this software architecture stems from the project's division into two major aspects: one for decision-making and the other for control techniques. The decentralized software architecture offers advantages. Firstly, it provides resilience and fault

[2]ROS2 Documentation: https://www.ros.org

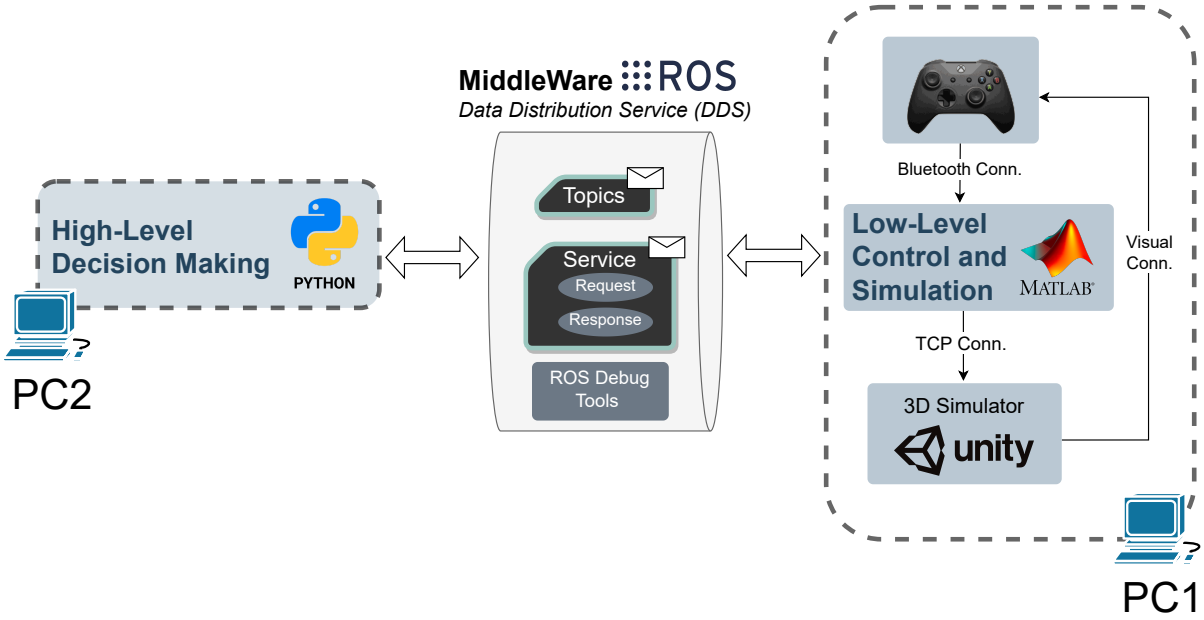

FIGURE 3.13 – Diagram of the computational implementation.

tolerance by distributing components, thereby mitigating the impact of individual failures, enhancing system reliability. Furthermore, this architecture is scalable, allowing for efficient expansion of the system to more drones. Lastly, it yields improved performance and lower latency, as tasks can be executed locally, reducing the need for communication through a centralized network. These features render decentralized architecture a valuable choice for systems, particularly with an eye toward future applications embedded in drones.

The data communication interface between the HL decision-making layer and the LL control layer is implemented using the framework provided by ROS2 (QUIGLEY *et al.*, 2009). It is based on topics used to publish and receive customized messages in a publish-subscribe pattern (BASS *et al.*, 2003) that allows the layers to share important information asynchronously. The customized messages contain information regarding the action commands from the HL decision-making module and the status update of the vehicles calculated by the LL control and simulation sub-module, as described in Table 3.4. For ease of implementation, each vehicle is identified by a unique ID. The LL control and simulation sub-module also sends unidirectional information containing the pose of the vehicles via socket to the 3D simulator that works as a visual feedback to the pilot.

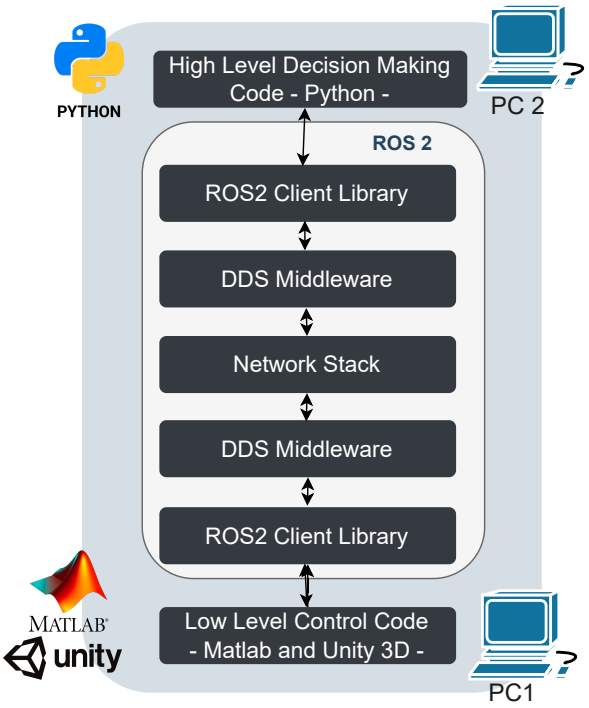

FIGURE 3.14 – Data flow diagram.

TABLE 3.4 – Table of commands.

Type of command	*Command*	*Parameters*
Movement	`GO_TO_POSE`	Target position and attitude.
	`SET_FORMATION_POSE`	Leader ID and relative position and attitude w.r.t. to the leader.
	`CHASE`	Threat ID to be pursued and neutralized.
Weapon	`SHOOT`	Orders the firing of a specified gun on a desired target

The decision-making process for the loyal wingman drones and kamikaze drones is depicted in Figure 3.14. This process takes place in the HL Decision Making layer, implemented in Python. To facilitate communication between this layer and the middleware DDS, a Python client library provided by the ROS2 framework is used. The middleware is responsible for serializing the information and transmitting it through the network stack.

Once a topic is published, the ROS2 framework ensures that the message is delivered to all the subscribed nodes within the network. On the low-level control PC, the middleware receives the message intended for nodes subscribed to the corresponding topic. It then deserializes the information and utilizes a MATLAB client library to incorporate the received data into the LL control of the drones.

Furthermore, the MATLAB client is responsible for transmitting the drone's information to the 3D simulation running in Unity using a socket connection. This two-way data flow enables seamless interaction between the high-level and low-level components. Similarly, the data flow from the low level to the high level follows the same process described above.

Note that drone state information is transmitted from the simulation layer in the low-level to the high-level layer, comprising messages containing vehicle pose and weapon status. This is the sole message relayed from low-level computer to high-level. On the other hand, decision-making sends predefined messages to the low-level with commands as seen in 3.4. This message exchange occurs synchronously at a predefined time interval in the code, which can be adjusted by the developers.

3.5.2.2 Logical Model of the High-Level Decision-Making Module

The class diagram depicted in Figure 3.15 presents the key components of the High-Level Decision-Making Module responsible for implementing the desired scenario. These components include the following:

- `Simulation` : This is the main component responsible for controlling the execution, creation, and destruction of decision-making modules for the simulated agents in the scenario. It also includes the world model, which processes the received situation awareness and distributes it to the respective agents. This application runs each agent using multi-threaded parallelism. The term simulation here is used to refer to the abstraction that involves the simulator's high level functions, this component is a wrapper that encapsulates the world model as well as the agent components. Note that the situation awareness is computed in the LL layer, and is received through messages that updates the world model. It represents an abstraction of the situation, or the states, of the agents present in the scenario.

- `Evaluate Simulation`: The component responsible for implementing the evaluation metrics of the scenario's agents plays a crucial role in assessing their performance. It incorporates the metrics described in Section 3.6.1 to provide a comprehensive evaluation of the agents' behavior and effectiveness. Additionally, this component includes the functionality to visualize and demonstrate the results through graphics.

- `OptimizationPSO`: This component encompasses the optimization algorithm (PSO) specifically designed to enhance the efficiency of the LW decision-making process. While this component was not utilized in the high-fidelity simulator, it plays a crucial role in the 2D prototype.

- `Input Processor`: The component responsible for reading and processing user commands performs the task of capturing and interpreting input from the user, including keyboard and mouse interactions. This functionality enables the system to respond accordingly based on the user's commands and actions.

- `Protected Area`: The entity serves as a representation of the critical area that the MUM-T (Manned-Unmanned Teaming) system aims to protect. This entity encompasses the localization of the protected area within the environment and includes essential information about its current state. The Protected Area entity contains specific geographic coordinates or spatial boundaries that define the extent and location of the critical area. This information is crucial for the MUM-T system to identify and operate within the designated protected zone.

- `UAV Vehicle`: The component responsible for implementing the model, dynamics, and animation of the UAV utilized in the system integrates various elements, including position and attitude control components, to effectively control the models. It ensures a simplified representation of the UAV's physical characteristics, dynamics, and movement, enabling simulation and animation within the 2D prototype. Note that this is an abstract object that represents a generic aerial vehicle. The LL layer is responsible to update the dynamics and states of the drones.

- `Message Manager`: The component plays a vital role in the system by efficiently managing and parsing messages to be sent and received using the ROS 2 interface. Its primary responsibility is to handle the communication between different entities within the system, ensuring seamless data exchange. By utilizing the ROS 2 interface, the component facilitates reliable and efficient message transmission. It handles the parsing of incoming messages, extracting relevant information, and delivering it to the appropriate components or modules within the system. Similarly, it prepares and formats outgoing messages, ensuring they adhere to the specified message structure and protocol. This component serves as a crucial bridge between different parts of the system, enabling effective coordination and collaboration. It handles message routing, ensures message integrity, and manages any necessary data transformations or conversions.

- `Position Controller`: Implements the position control law for UAVs. Only used in the 2D prototype.

- `Attitude Controller`: Implements the attitude control law for UAVs. Only used in the 2D prototype.

- `Kamikaze` : The class that implements the kamikaze UAV is a subclass of the UAV Vehicle, inheriting its properties and functionalities. This specialized component incorporates the specific behaviors and characteristics of the kamikaze drone, enabling it to perform its unique functions within the system. By extending the Vehicle MAV class, this component benefits from code reuse and maintains a consistent structure with the rest of the UAV hierarchy.

- `Leading Drone`: The component responsible for implementing the leading drone is specifically designed to enable user control through the utilization of an Input Processor class. The Input Processor class plays a crucial role in reading and interpreting mouse commands, granting users the ability to position the leading drone as desired within the scenario. Furthermore, this class incorporates the aggregation of a formation manager and a task allocator, enhancing the functionality and capabilities of the leading drone. With this integrated component, the leading drone have direct control over the formation of the MUM-T and also is capable to distribute tasks within the team.

- `Finite State Machine`: The component responsible for decision-making through state machines is specifically designed to handle the decision-making process for kamikaze drones within the application. This component utilizes state machines as a powerful tool for modeling and controlling the behavior of kamikaze drones. By leveraging state machines, the decision-making process becomes structured and organized, enabling the kamikaze drones to transition between different states based on specific conditions and events. This approach facilitates efficient and effective decision making, tailored specifically for the unique requirements of kamikaze drones within the application.

- `Loyal Wingman`: The component responsible for the implementation of loyal wingman drones is a specialized subclass of the UAV Vehicle. This component instantiates the BT framework and weapon components, forming a cohesive unit for controlling and defining the behavior of loyal wingman drones. By extending the UAV Vehicle class, this component inherits essential properties and functionalities. The Behavior Tree component provides a framework for organizing and executing the decision-making logic of the loyal wingman drones, while the Weapon component handles the functionality and interactions related to the drones' weapons. Together, these components enable the loyal wingman drones to exhibit sophisticated behaviors and effectively fulfill their designated roles within the system. In addition, the component includes a Task Manager class. The Task Manager is responsible for handling the tasks allocated by the leader drone. It manages and assigns tasks to the loyal wingman drones based on the instructions received from the leader, ensuring efficient coordination and collaboration within the system. Together, these components enable the loyal wingman drones to exhibit sophisticated behaviors, execute assigned tasks, and effectively fulfill their designated roles within the system.

- `Behavior Tree`: The decision-making framework implemented in this component enables the autonomous control of loyal wingman drones. Designed specifically for this purpose, the framework allows the drones to make independent decisions

based on various factors and criteria. By leveraging this framework, the loyal wingman drones can navigate, respond to dynamic situations, and carry out tasks autonomously, without direct human intervention. The decision-making framework incorporates a range of algorithms, techniques, and models that facilitate intelligent decision-making. It takes into account factors such as situation awareness inputs, environment states, mission objectives, and predefined rules to determine the most appropriate actions for the loyal wingman drones to take.

- `Weapon`: The component responsible for implementing the weapon models used by the Loyal Wingman drones in the scenario is crucial for their functionality and engagement capabilities. By incorporating this component, the Loyal Wingman drones gain the ability to effectively deploy and utilize their designated weapons within the scenario. The weapon models are designed to simplify the effects, and interactions of the specific weapons used by the drones. This component abstracts the necessary functionality to handle weapon mechanics such as targeting, firing, ammunition management, and damage calculations. It ensures that the weapon systems operate in accordance with the defined rules and behaviors.

- `Vaporizer`: The component responsible for implementing the loyal wingman's vaporization weapon is a specialized extension of the Weapon component. This subclass focuses specifically on the functionality and characteristics of the loyal wingman's vaporizer weapon.

- `Freezing` : The component responsible for implementing the loyal wingman's freezing weapon is a specialized extension of the Weapon component. This subclass is specifically designed to handle the unique functionality and behavior of the loyal wingman's freezing weapon.

3.5.2.3 Graphical Interface

In Figures 3.17 and 3.16 we have the complete view of a running simulation. Note that there are two ways to visualize the simulation, one with photo-realistic 3D visualization and another in 2D with a strategic view from the top, as a matter of simplification and focused on presenting information about what the agents are "thinking". This 2D viewer shows legends containing additional information regarding the decision-making and current state of the agent, and draws the destination waypoint and radius of engagement of each agent. There is also the presence of performance metrics in the 2D viewer at the bottom of the figure. Both visualizers demonstrate the simulation that is being performed on the low-level control and simulation layer of the system, and it uses the same messages transmitted in the ROS middleware to illustrate the simulation.

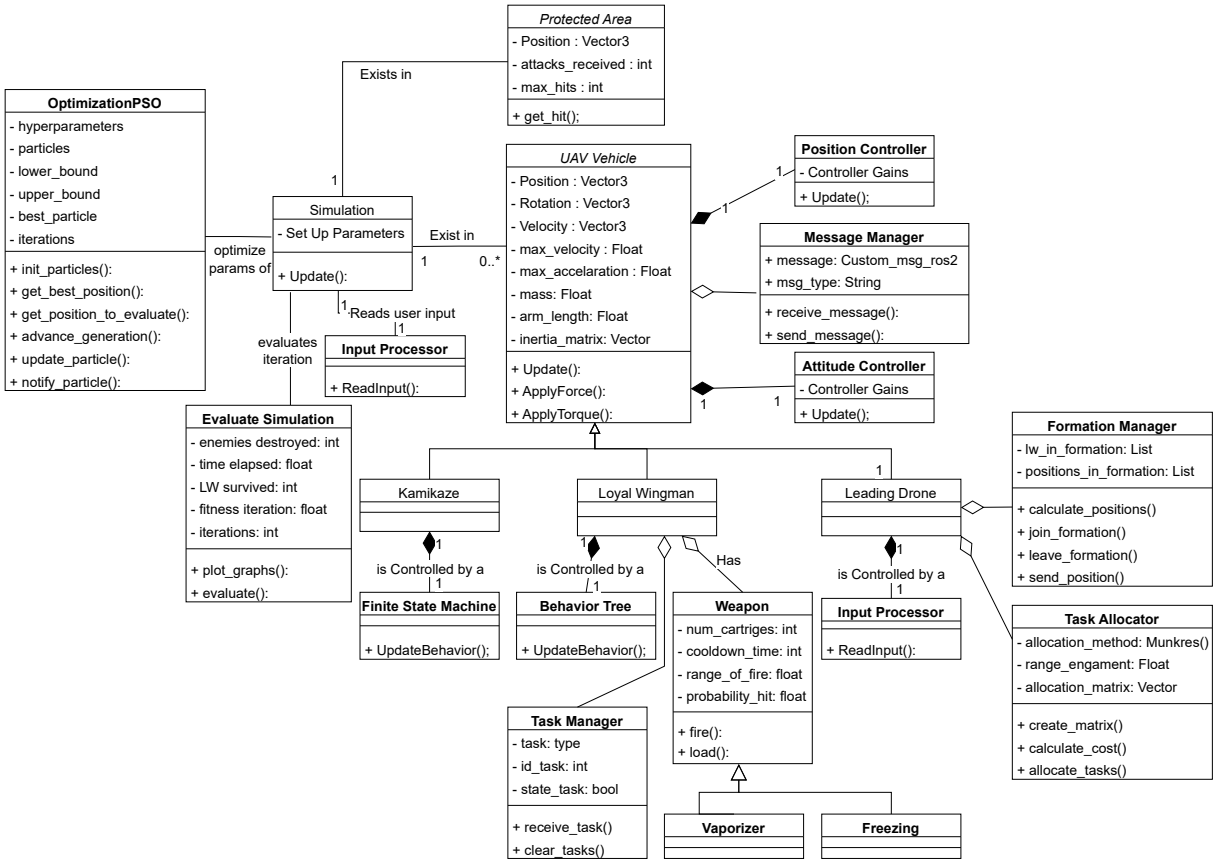

FIGURE 3.15 – Class diagram of the components that implement the high-level decision-making module.

The developed simulator is versatile and can be customized to suit alternative scenarios and different types of drones, as depicted in Figure 3.18. In the shown figure, we observe an alternative scenario featuring the utilization of an under-actuated UAV model. It is worth noting that the 3D visual representations do not affect the functionality of the decision-making layer, but rather focus on enhancing the dynamics and control aspects.

3.5.3 Communication Interface

This section's objective is to provide comprehensive documentation and clarification on the implementation of the system's messaging system, facilitating the replication of research studies.

As we have two main layers of software, it is necessary to define a communication interface between such modules, *i.e.*, how the decision commands are translated into control commands for the UAVs. This interface serves as an important bridge, enabling effective coordination and synchronization between the software modules, ensuring smooth and accurate control over the UAVs.

As defined in Figure 3.4, the Kamikaze UAVs have four distinct behaviors: `idle State`, `Attack Leader State`, `Attack Loyal Wingman State`, and `Attack Protected`

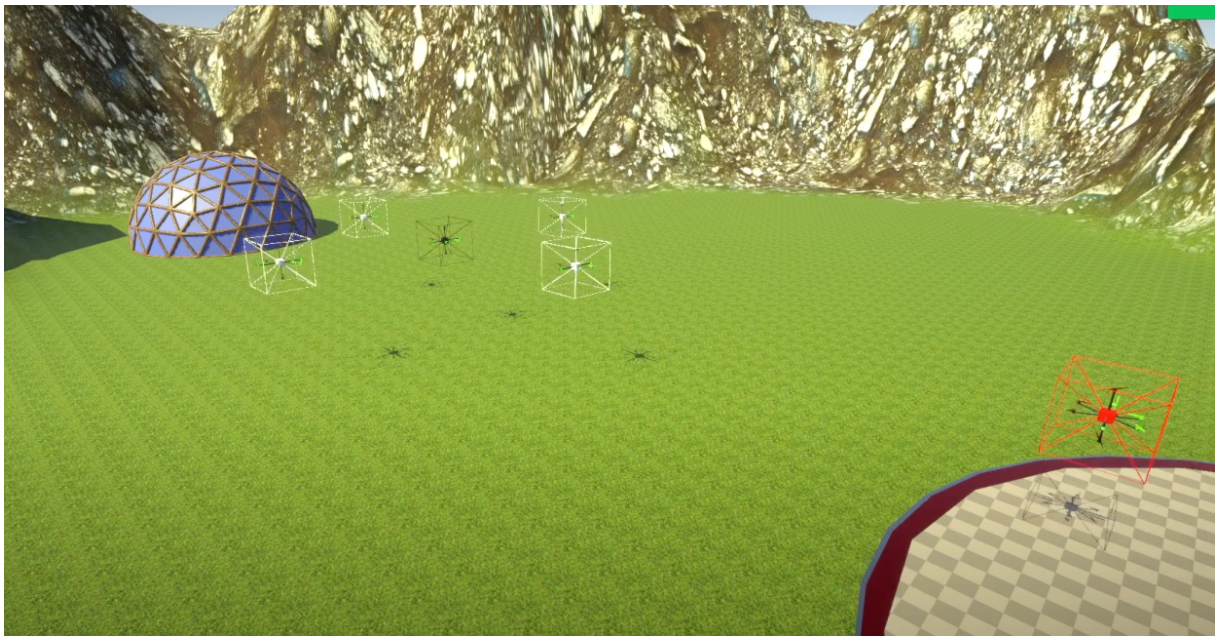

FIGURE 3.16 – full view of a running simulation using fully-actuated octacopters. Blue Dome represents the protected area.The Black Drone represents the manned-drone, which is operated by a human pilot; White Drones symbolize the loyal wingman drones, which operate in a collaborative manner. Red Drone represents a kamikaze threat, signifying a potential danger.

FIGURE 3.17 – full view of a running simulation, we can see the decision-making visualizer and 3D simulator in the upper right part of the figure. The position of each agent can be seen in the legend at the top left. Each drone have a unique color, and have a legend that presents its current behavior, a velocity vector, the ammunition of its weapons and the availability, and a line indicating the agent's destination waypoint. Note that both visualizers represent the same situation from 2D and 3D viewpoints.

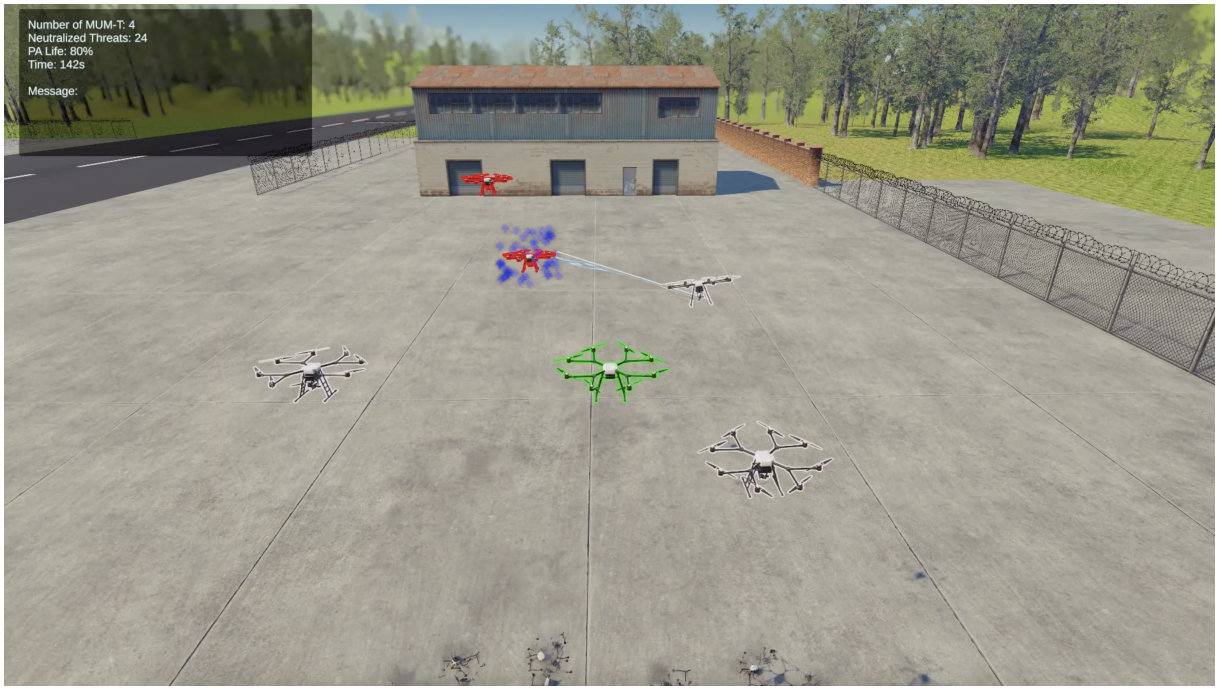

FIGURE 3.18 – Alternative visualization of the scenario involving under-actuated UAVs. The simulation showcases a fleet of drones in full motion, with each drone distinguished by its distinctive color scheme. The red drones symbolize the Kamikazes. The white drones symbolize the loyal wingmen. Meanwhile, the green drone represents the leader.

`Area State`. To facilitate the translation of these behaviors into actionable instructions, the HL-LL interface exclusively converts them into target position commands. This approach is implemented considering that the Kamikaze behaviors do not necessitate any attitude changes, streamlining the communication between the higher-level and lower-level software components.

The `Idle State` demands the Kamikaze to stay stationary in its current position, and to do so the target position informed to the LL guidance algorithm must be equal to the Kamikaze current position, *i.e.*,

$$\check{\mathbf{r}}_g^{j/g}(t) = \mathbf{r}_g^{j/g}(t),$$

where $\mathcal{I}_k$ is the set containing all Kamikazes and $j \in \mathcal{I}_k$ is the ID of the Kamikaze agent under consideration. $\check{\mathbf{r}}_g^{j/g}(t)$ represents the target position informed to the guidance algorithm for the Kamikaze agent with ID j at time t, and $\mathbf{r}_g^{j/g}(t)$ represents the current position of the Kamikaze agent with ID j at time t.

On the other hand, `Attack Leader State`, `Attack Loyal Wingman State`, and `Attack Protected Area State` can be executed by generating the target position as

$$\check{\mathbf{r}}_g^{j/g}(t) = \mathbf{r}_g^{k/g}(t), \tag{3.3}$$

where k is the ID of the desired target. $\check{\mathbf{r}}_g^{j/g}(t)$ represents the target position informed to the guidance algorithm for the Kamikaze agent with ID j at time t, and $\mathbf{r}_g^{k/g}(t)$ represents the current position of the desired target with ID k at time t.

As shown in Table 3.2, the LW has four movement behaviors: `Chase Threat`, `Go To formation`, `Approach Formation`, and `Sacrifice Attack`.

To execute the `Go To Formation` behavior, the HL-LL interface of the LW informs the target position to the guidance algorithm as

$$\check{\mathbf{r}}_g^{i/g}(t) = \mathbf{r}_g^{\ell/g}(t) + \check{\mathbf{r}}_g^{i/\ell}, \tag{3.4}$$

where $\check{\mathbf{r}}_g^{i/g}(t)$ represents the target position for LW agent i in the global coordinate system at time t, $\mathbf{r}_g^{\ell/g}(t)$ represents the current position of the leader agent (denoted by ℓ) in the global coordinate system at time t, $\check{\mathbf{r}}_g^{i/\ell}$ represents the target position for agent i relative to the leader agent (i.e., the offset or displacement from the leader).

The equation (3.4) states that the target position for agent i is determined by adding the leader's current position $\mathbf{r}_g^{\ell/g}(t)$ to the target position $\check{\mathbf{r}}_g^{i/\ell}$ relative to the leader. This means that the agent's target position is set as an offset or displacement from the leader's position.

Similarly, to execute `Approach Formation`, the target position is calculated as

$$\check{\mathbf{r}}_g^{i/g}(t) = \mathbf{r}_g^{l/g}(t) + \check{\mathbf{r}}_g^{i/l} + \mathbf{e}_3^{\mathrm{T}} d_f.$$

where $\check{\mathbf{r}}_g^{i/g}(t)$ represents the target position for the LW agent i in the global coordinate system at time t, $\mathbf{r}_g^{l/g}(t)$ represents the current position of the reference point (denoted by l) in the global coordinate system at time t. The reference point could be a leader or any other point in the system, $\check{\mathbf{r}}_g^{i/l}$ represents the target position for the LW agent i relative to the reference point (i.e., the offset or displacement from the reference point), $\mathbf{e}_3$ represents the unit vector in the direction of the z-axis in the global coordinate system, and d_f represents a scalar value that determines the distance to be maintained from the reference point along the vertical axis.

As the formation is defined on the local horizontal plane, the LW adopts a simplified approach to join the formation by incorporating a vertical offset relative to its target position within the formation. This vertical offset strategy allows for straightforward coordination and alignment, ensuring the LW seamlessly integrates into the formation.

Lastly, to execute the `Chase Threat`, the target position informed to the guidance algorithm of the LW must be set as an offset relative to the target kamikaze position, *i.e.*,

$$\check{\mathbf{r}}_g^{i/g}(t) = \mathbf{r}_g^{k/g}(t) - d_t \frac{\mathbf{r}_g^{k/\ell}(t)}{\|\mathbf{r}_g^{k/\ell}(t)\|}, \tag{3.5}$$

where $\check{\mathbf{r}}_g^{i/g}(t)$ represents the target position for the LW agent i in the global coordinate system at time t, $\mathbf{r}_g^{k/g}(t)$ represents the current position of the target Kamikaze agent with ID k in the global coordinate system, d_t is a scalar value that determines the distance to be maintained from the target Kamikaze agent during the chase. Finally, $\mathbf{r}_g^{k/\ell}(t)$ represents the relative position of the target Kamikaze agent with respect to the reference point (usually the leader).

The Equation (3.5) states that to execute the `Chase Threat` behavior, the target position for the LW agent is determined by subtracting a vector from the target Kamikaze agent's position. This vector is calculated as the product of d_t and the normalized relative position vector $\mathbf{r}_g^{k/\ell}(t)/\|\mathbf{r}_g^{k/\ell}(t)\|$. By subtracting this vector from the target Kamikaze agent's position, the LW agent aims to position itself at a distance d_t behind the target Kamikaze agent, aligned with the relative position vector.

3.5.4 Messages definition

This section is dedicated to elucidating the specifications of the defined messages that circulate between both layers of software.

The interface for data communication between the high-level decision-making layer and the low-level control layer is established through the use of middleware provided by ROS 2 (QUIGLEY *et al.*, 2009), introduced in Section 2.8.2, as illustrated in Figure 3.13. This interface is designed using a publish-subscribe pattern, where topics are used to send and receive customized messages, enabling the layers to exchange important information asynchronously. The customized messages contain crucial information, including action commands from the high-level decision-making module and the current status updates of the vehicles, which are calculated by the low-level control and simulation sub-modules. These details are outlined in Table 3.4.

To facilitate implementation, each vehicle is assigned a unique ID. Moreover, the low-level control and simulation sub-module transmits unidirectional information to the 3D simulator via a socket, providing the pilot with visual feedback regarding the vehicles' pose. This communication occurs between two separate computers connected to the same local network. For example, the `HLmsg` message can be transmitted from the high-level decision-making node to the low-level control node, as depicted in Figure 3.19. It is important to emphasize that the communication framework we have proposed is versatile and distributed, enabling the transmission of any pre-defined message.

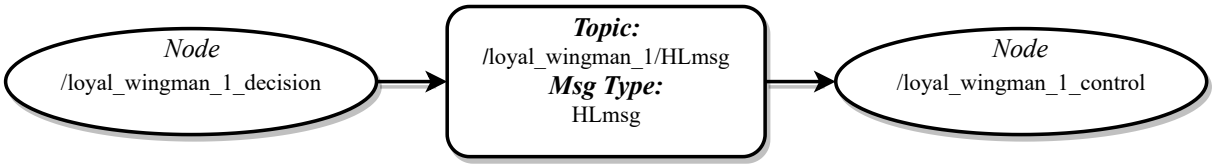

FIGURE 3.19 – ROS 2 network diagram of the communication between software layers.

Therefore, the software layers communicate by message passing through topics, using the subsequent topic patterns:

- `loyal_wingman_id/HLmsg`
  - For commands related to the decision of a Loyal wingman UAV.

The parameter `id` represent the identifier of a specific Loyal Wingman UAV, i.e, the Loyal Wingman 1 will be referenced by the topic:

- `loyal_wingman_01/HLmsg`

A topic for a Kamikaze UAV will follow the subsequent pattern:

- `kamikaze_id/HLmsg`
  - For commands related to the decisions of a kamikaze UAV.

The parameter `id` represent the identifier of a specific Kamikaze UAV, so for example the kamikaze 1 will be referenced by the topic:

- `kamikaze_01/HLmsg`
  - Note that the kamikaze drone does not have a `cmd_weapon` since it has no weapons besides its explosive capability.

Lastly, a topic is created to configure the simulation, where parameters such as the number of agents to be simulated and the position of the protected area are defined using the following topic:

- `/Config`
  - The decision-making layer receives the message using this topic to initialize the correct number of decision-making nodes.

3.5.4.1 Messages Specification

The set of messages transmitted between the decision-making and low-level control modules seen in Table 3.4 is described in more detail.

1. **SETUP**:
   Description: message containing information to initialize the simulation.
   - *nLW*: `int64`; the number of LW agents to be simulated.
   - *nKamikaze*: `int64`: the number of Kamikaze agents to be simulated.
   - *PAcoord*: `Vector3`: the coordinates of the protected area.
2. **HLCMD**:
   Description: This message is a wrapper containing the decisions of a agent, it is sent from the decision-making module to the low-level control layer.
   - *cmd_id*: `int64` (enum): the kind of content of the HL command, the low-level will select one of the following based on this id.
   - *formation_cmd*: `SetFormationPose`; contains a SetFormationPose command message.
   - *goto_cmd*: `Goto`; contains a GOTO command.
   - *chase_cmd*: `Chase`; contains a CHASE command.
   - *block_cmd*: `Block`; contains a BLOCK command.
   - *shoot_cmd*: `Shoot`; contains a SHOOT command.
3. **BLOCK**
   Description: blocks an attacking enemy drone.
   - enemy_id: `int64`; the id of the enemy drone to be blocked.
   - distance: `float64`: safe distance to be kept from the enemy.
4. **CHASE:**
   Description: chases an enemy drone.
   - *vehicle_id:* `int64`; the id of the enemy drone to be chased.
   - *distance*: `float64`: distance to be kept from the enemy during the pursue.
5. **GOTO**
   Description: move the drone to a specific pose using the path planning algorithm. Obstacle avoidance will taken into account by this action.

- *goal_pos:* `Vector3`; desired position;
- *goal_att:* `Vector3`; desired attitude.

6. **SET_FORMATION_POSE**
   Description: keeps the drone flying in formation.
   - *leader_id*: `int64` (enum); the vehicle id to be followed.
   - *goal_rel_pos*: `Vector3`; the drone's position within the formation.
   - *goal_rel_att*: `Vector3`; the drone's attitude within the formation.
7. **SHOOT**
   Description: shoots with a specific weapon.
   - *gun_type:* `int64`; specifies a weapon based on an id. Vaporizer is 1 and freezing is 2.
   - *target_id:* `int64`; the drone's unique identifier in which the gun must be shot.
8. **VECTOR3**
   Description: represents a vector in free space.
   - *x:* `float64`; value in x–axis
   - *y:* `float64`; value in y–axis
   - *z:* `float64`; value in z–axis

The messages containing the situational awareness (SA) are send from the low-level layer using a `SAarray` and received by the decision-making module. The messages are the following:

1. **SA**
   Description: it contains the state of a agent being simulated.
   - *is_alive:* `bool`; information if the agent is alive or destroyed.
   - *ammo_freeze* int64 ammunition of the freezing gun.
   - *is_freeze_available*: `bool`; freezing gun available for usage.
   - *ammo_vaporizer*: `int64`; ammunition of the vaporizer gun.
   - *is_vaporizer_available*: `bool`; vaporizer gun available for usage.
   - *pos:* `Vector3`; current position of the agent.
   - *vel:* `Vector3`; current velocity of the agent.
   - *att:* `Vector3`; current attitude of the agent.

2. **SAarray**:
   Description: represents an array of SA messages, it contains all the states of the agents being simulated in a single message.

   - *sa_msg:* `SA[]`; array of SA containing the a SA message for each agent in the simulation.

3.5.5 Simulation Setup

Utilizing a distributed architecture, our simulator necessitates a well-defined initialization sequence, as depicted in Figure 3.20. This sequence comprises two distinct layers: the high-level decision-making layer and the low-level control and dynamics layer. The initialization process commences with the low-level layer, responsible for determining the amount of simulated loyal wingman and kamikaze agents. Subsequently, the low-level layer dispatches a configuration message to the decision-making layer, encompassing the number of decision modules to be initialized and the precise location of the protected area.

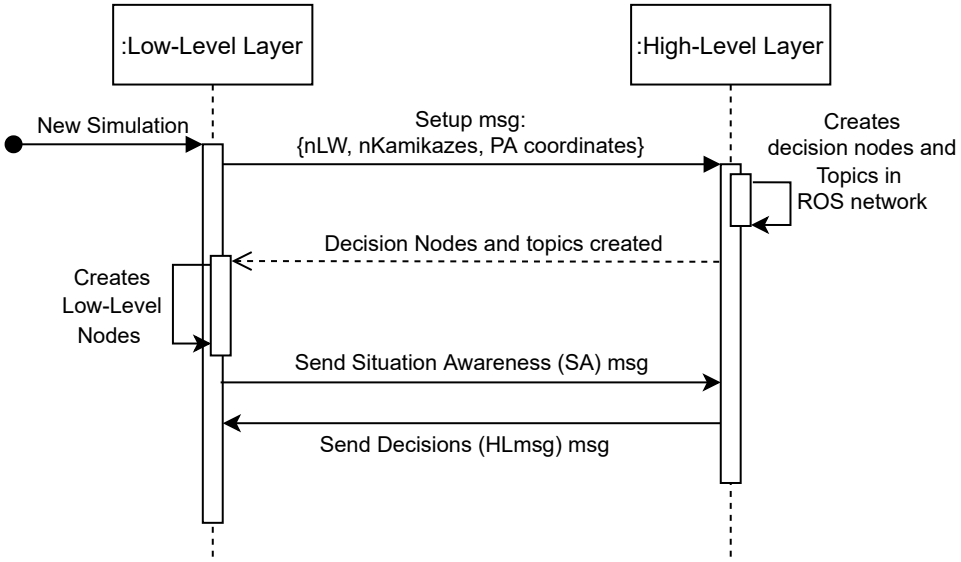

FIGURE 3.20 – Setup Sequence diagram.

Following the initialization of ROS nodes and topics by the decision-making layer, it enters a waiting state for the commencement of the low-level simulation. During this time, the control and dynamics layer dynamically detects the initialized nodes and generates the corresponding low-level nodes within the network, typically requiring a few seconds. Upon successful creation of all nodes in the network, the simulation commences, and the present scenario is relayed to the decision-making layer for analysis and decision-making of each agent. The decision-making layer reciprocates by transmitting decision messages

through the network, thereby initiating an ongoing communication loop that persists until the simulation's conclusion.

3.6 Results and Discussion

In this section, we present the experimental setup and results of the study. The experiments were designed to evaluate the performance of the proposed system. We first describe the setup used in our experiments, followed by a detailed description of the scenarios and the metrics used to evaluate the performance of the system. We then present the results of the experiments and analyze them in the context of the research questions posed in this study.

3.6.1 Metrics of Interest

The implemented swarm intelligence is investigated against the following metrics of interest:

- ED (Enemies Destroyed): The number of kamikazes destroyed before the game is over.
- LWS (LW survived): The number of LW agents that survived the iteration.
- ST (survival time): The duration of the simulation until the leader or the protected area is destroyed.

Based on these metrics, a fitness function was empirically developed, and is used to optimize the behavior parameters by evaluating each iteration for the optimization algorithm (KENNEDY; EBERHART, 1995) as:

$$Fitness = ED^2 + 10ST - 1000LWS. \tag{3.6}$$

The idea of Equation (3.6) is based on the reward for a bigger number of kamikazes destroyed and a longer duration time of each iteration. There is also a big discount if there are LW drones left when the iteration ends, as we consider that these drones are not being effective in fulfilling the mission, which reinforces them to remain alive for longer. Once again, it is important to emphasize that this function has been empirically calibrated, and these weights have proven effective in practical tests. However, it may not necessarily represent the optimal function.

Note that this metric is intended for parameter optimization purposes. Therefore, in a scenario where reducing the intelligence of kamikaze drones might appear to improve overall performance according to this metric, it does not necessarily indicate an enhancement in the performance of the loyal wingmen. Consequently, when evaluating performance solely based on this quantitative metric, caution is advised. When comparing efficiency across different scenarios, it's essential to take into account the intelligence of kamikaze drones as well.

3.6.2 High-Level Prototype Experiments and Results

In this subsection, we describe the experiments performed for the loyal wingman 2D scenario, also the metrics for evaluation and the results are presented. The simulations were performed with 10 LW and 4 kamikaze drones, once one kamikaze is destroyed another is created, maintaining a steady stream of attack. The simulation set-up parameters are present in Table 3.5.

TABLE 3.5 – Weapons and PSO parameters.

Weapon	**Parameter**	**Value**
Vaporizer Gun	vaporizer_cooldown	1 [s]
	cartridges_vaporizing	10
	vaporizer_hit_prob	95%
	vaporizer_range	10 [m]
Freezing Gun	freezing_cooldown	1 [s]
	cartridges_freezing	10
	time_frozen	5 [s]
	freezing_hit_prob	85%
	freezing_range	30 [m]

PSO	**Value**
num_particles	40
inertia_weight	0.7
cognitive_parameter	0.6
social_parameter	0.8
UAV Parameters	**Value**
Mass	1 [kg]
K_v	4.5
K_p	0.625
max_speed	2 [m/s]

3.6.2.1 Experiments Description

In order to evaluate the efficiency of intelligence tailored for the LW agents, we conducted two monte carlo experiments. The primary aim was to examine the proposed BT and its associated parameters, seen in Figure 3.21, and to assess its effectiveness in practice.

The experiments are:

- *First Experiment*:
  - Objective: find a effective architecture for the behavior tree using the behaviors;
  - Method: manually find `engagement_distance` and `distance_formation`.

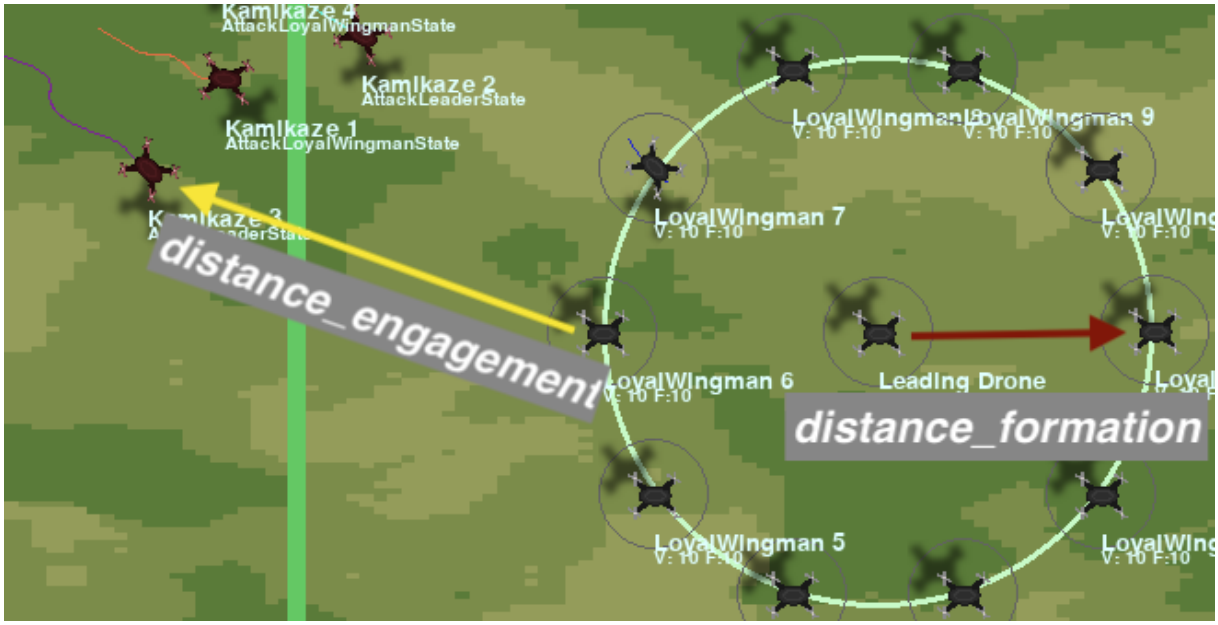

FIGURE 3.21 – Parameters description.

- *Second Experiment*:
  - Objective: optimize the BT efficiency with optimal parameters using PSO;
  - Method: optimize the `engagement_distance` and `distance_formation`. for maximization of the fitness function (3.6). The PSO is set using the parameters in Table 3.5 for 60 generations.

3.6.2.2 Results and Discussion

In this section, we present the results obtained from simulations conducted using the 2D high-level prototype. We begin by presenting the quantitative results, followed by a qualitative observation of the simulations. Finally, we discuss the findings from each approach in detail.

The results and the parameters obtained in the experiments are shown in Table 3.6. Using these parameters, 500 iterations were performed to obtain the statistical values of mean and standard deviation (std) for each metric.

TABLE 3.6 – Experimental Results.

Exp.	**Metrics**				**Parameters [m]**	
		ED	**ST [s]**	**LWS**		
1	*Mean*	99.45	131.8	1.73	`distance_engagement`	36.06
	Std	8.46	13.56	1.12	`distance_formation`	13.00
2	*Mean*	102.1	167.6	1.37	`distance_engagement`	30.66
	Std	7.68	13.52	0.87	`distance_formation`	10.01

First, assuming a reference to analyze the results the fact that the 10 LW used have only 100 ammunition for threat elimination (10 for each LW in vaporizer gun), and that

this weapon has a 95% hit rate, *i.e.* at best we would get 110 kills – no drone misses a shot and can eliminate one threat by sacrifice when out of ammo – and a long-term average of 105 eliminations (5 misses and 10 eliminations by sacrifice).

Based on our observations, the results presented in Table 3.6 indicate that the LW agents utilizing the developed BT demonstrate effectiveness as an autonomous defense system. In the first experiment, the agents collaborated to eliminate an average of 99.45 kamikazes, which accounted for 90.4% of the total threats. The standard deviation for this data was 8.46 kamikazes. These findings highlight the efficacy of the LW agents, even when employing empirically determined parameters.

In the second experiment, we optimized both parameters to maximize defense effectiveness. The cooperative efforts of the agents led to an average elimination of 102.1 kamikazes, accounting for 92.8% of the total threats. This represents an increase of approximately 2.7 enemies eliminated per iteration compared to the first experiment, with a smaller standard deviation. Furthermore, the second experiment resulted in a significantly improved survival time, which was 27.2% longer compared to the first experiment. Additionally, there were 20.8% fewer LW survivors at the end of the second experiment, indicating a potential increase in LW effectiveness.

Through the application of parameter optimization, we observed a significant improvement in the effectiveness of decision-making, as evidenced by smaller standard deviations across all indicators. These findings indicate that the optimized parameters resulted in decision-making that was closer to the optimal average of 105 threats eliminated. This improvement highlights the efficacy of our approach in achieving more consistent and effective outcomes by applying the PSO to the solution.

3.6.2.3 Qualitative Analysis

The described results are also available in video[3]. Figure 3.22 showcases a visual representation of how a group of LWs effectively cooperate to eliminate threats. Initially, when the kamikaze swarm surpasses the engagement limit, three LWs break formation to neutralize the threats (a). Subsequently, the LW group employs their freezing guns to slow down the threats (b). In (c), the group successfully eliminates the threats, and all LWs return to their original positions in formation. This video provides a visual demonstration of the cooperative strategies employed by the LWs to ensure effective threat elimination.

[3]https://www.youtube.com/watch?v=bWk9x7zUZuM

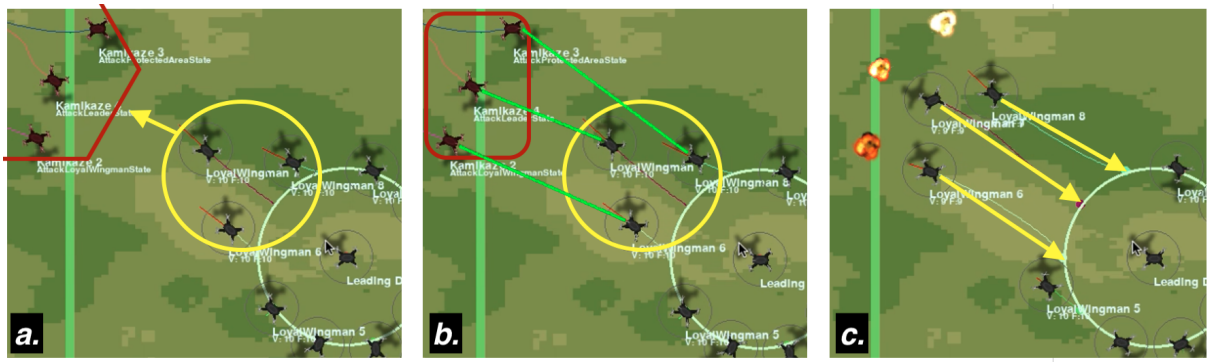

FIGURE 3.22 – Group attacking – cooperative work to neutralize threats.

Next, in Figure 3.23, we observe a remarkable instance where a LW sacrifices itself to protect the MUM-T leader. In frame (*a*), the yellow LW can be seen firing its vaporization weapon and missing the target, prompting it to retreat from the imminent threat (indicated by the yellow arrow). Meanwhile, the blue LW also detects the threat but exhausts its ammunition, leading it to execute the *SacrificeAttack* behavior as a strategic decision. This act of self-sacrifice demonstrates the LW's willingness to prioritize the safety and preservation of the manned leader over its own survival.

FIGURE 3.23 – Sacrifice attack to preserve MUM-T as last resort tool.

Continuing the sequence of events, in Figure 3.24, we witness a collaborative effort between two LWs to disable a threat. Initially, in frame (*a*), both the yellow and orange LWs retreat after failing to hit their respective targets. While evading, the yellow LW identifies a kamikaze in pursuit of the orange LW, as depicted in frame (*b*). Ultimately, the orange LW successfully freezes the threat, enabling the yellow LW to eliminate it in frame (*c*). This cooperative attack showcases the tactical coordination and effective utilization of their freezing and elimination capabilities by the LWs.

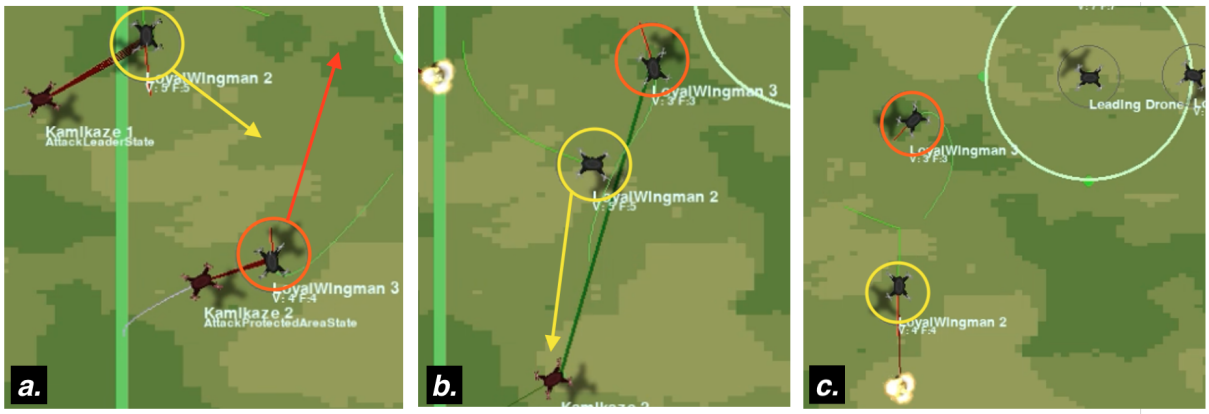

FIGURE 3.24 – Cooperative work to neutralize threat.

Lastly, in Figure 3.25, we observe the emergence of group responsibilities based on the kamikaze objective. It is noteworthy that the LWs marked in yellow often assume the role of launching forward attacks and providing superior lateral protection to the designated *Protected Area*. In frame (a), we can see how three LWs break formation to safeguard the *Protected Area*. This allocation of responsibilities highlights the adaptive behavior of the LWs, as they dynamically adjust their positions and actions to effectively protect the designated area and respond to potential threats.

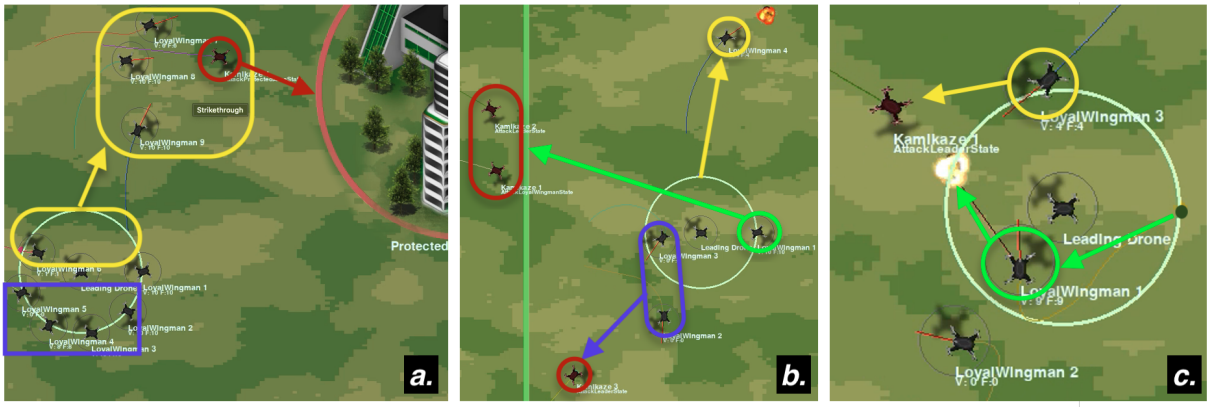

FIGURE 3.25 – Loyal Wingman specializations according to regions in formation

Additionally, the LWs marked in blue assume the responsibility of addressing threats originating from the bottom. As evident in frame (b), when a threat emerges from that direction, the blue LWs break formation, potentially leaving the leader unprotected. However, it is worth highlighting the crucial role of the green LW, which specializes in protecting the leader, as depicted in frames (b) and (c). In frame (c), we can observe the yellow LW returning to the formation after eliminating a threat and subsequently moving to neutralize another kamikaze targeting the leader. These instances underscore the strategic positioning and specialized roles played by the different LWs, ensuring comprehensive defense and safeguarding the integrity of the leader.

3.6.3 High-Fidelity 3D Simulator Experiments and Results

This section provides a comprehensive analysis of the outcomes achieved through the utilization of a high fidelity simulator. We employ a series of simulations to evaluate the effectiveness of our method, enabling a thorough examination of its performance. Specifically, we delve into three key aspects. First, in Subsection 3.6.3.2, we present the qualitative results of the high-level decision making process. Next, Subsection 3.6.3.3 explores a Monte Carlo simulation that offers valuable insights. Lastly, in Subsection 3.6.3.4, we highlight the computational load associated with the overall simulation, shedding light on its resource requirements.

3.6.3.1 Experiments Description

To evaluate the effectiveness of the method, we conducted a Monte Carlo simulation consisting of 122 iterations. Monte Carlo simulation is a computational technique that utilizes random sampling to model and analyze complex systems or processes (HARRISON, 2010). It involves running multiple iterations of a simulation, each time sampling from a set of random variables to represent uncertain parameters or inputs. By repeating the simulation numerous times, Monte Carlo simulation allows for the assessment of the system's behavior and the estimation of outcomes, such as probabilities. This method finds applications in various fields, enabling decision-making and optimization in situations where analytical solutions are difficult or impossible to obtain.

We conducted the simulations with four LW agents and two Kamikaze agents, where the Kamikaze agents respawned in an random position once neutralized. The protected area was located at the origin. To standardize the simulation loop, the leader UAV remained stationary and hovered at the fixed point (5, 0, 5) meters.

The LW have a maximum speed of 1.5 m/s, being the Kamikazes 50% faster. The freezing and vaporizer guns have an ammo of 10, a cooling down time of 1 s, and a hit probability of 95%. The freezing gun can reduce the Kamikaze speed by half during 5 s.

In all simulations, the UAVs are non-planar fully actuated octocopters, as the one depicted in Fig. 3.1, with a total mass of 1 kg, arm length of 0.5 m, and inertia matrix equal to $\text{diag}(0.015, 0.015, 0.015)\ kg \cdot m^2$. Moreover, the UAVs are subject to a sinusoidal force disturbance with the same amplitude of $0.6N$ but with different phase shifts, and to a sinusoidal torque disturbance with the same amplitude of $0.05N \cdot m$ but also with different phase shifts.

The LL control and simulation, coded in MATLAB, use the Euler integration method and runs with a frequency of 100 Hz. The actuator dynamics are modeled by first-order differential equations with a time constant of 0.01 s. The force and torque coefficients

of the rotors are 1.2838×10^{-5} N/(rad/s)^2 and 3.0811×10^{-7} Nm/(rad/s)^2, respectively. It is worth mentioning that the adopted model is typically used in the MAV literature to describe the set consisting of a brushless motor, an electronic speed controller, and a propeller (BEZERRA; SANTOS, 2022).

3.6.3.2 High-Level Decision Making Qualitative Results

In this section, we present the qualitative results of a series of simulations conducted using the proposed computational architecture. These simulations were performed to assess the effectiveness of the overall method, specifically the proposed high-level decision-making solution. The results discussed in this subsection are also available in video [4]. Furthermore, the simulations were utilized to empirically discover an efficient LW decision-making architecture, as well as to develop behaviors and fine-tune decision-making parameters. Subsequently, we demonstrate the operating sequences observed during scenario execution.

First, in Figure 3.26, we see the simulation result of the case where a Kamikaze threat is identified and neutralized. Initially, in frame (a) we see the MUM-T in formation and an identified Kamikaze threat in red. The LW UAV closest to the threat leaves the formation and chases the Kamikaze by executing the `chase threat` behavior. As a result, all remaining LW UAV members in formation will readjust to new positions defined by the leader, so no gaps for attacks are present.

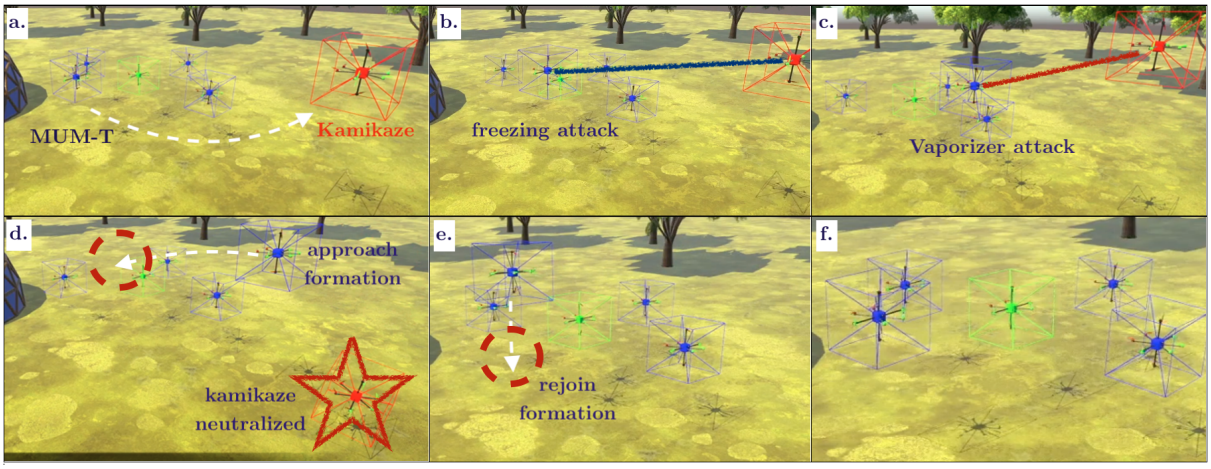

FIGURE 3.26 – Demonstration sequence of engagement to a Kamikaze threat, in red. The leader is seen in the center, in green, and four Loyal Wingman in blue.

Consequently, in frame (b), we have the LW that left the formation attacking the Kamikaze with the freezing gun, this strategy aims to facilitate the approach for neutralization since this weapon considerably reduces the Kamikaze's speed. Now in frame (c)

[4] https://youtu.be/YQjDRBW-0nk

the LW gets even closer to the frozen Kamikaze and uses its vaporizer gun to effectively destroy it. Consequently, we see in frame (d) that the Kamikaze is destroyed and the LW returns to formation, initially the LW must approach the formation from a safe distance executing the `approach formation` and request re-entry for the leader, who must allow its entry and readjust the formation with the addition of this agent. In frame (e) we see the LW successfully re-entering the formation and sequentially executing the behavior `go to formation`. Finally, in frame (f) we see all the agents keeping formation after the success of the mission.

Furthermore, in Figure 3.27, we observe two sequences of cooperative neutralization that emerged during the simulations, where two Loyal Wingmen performed tasks together to neutralize Kamikazes. In the first sequence, frames (a)-(c), we have LW1 freezing the Kamikaze from a longer distance, consequently, we observe LW2 in (b), which also identified the same threat, neutralizing it using the vaporizer gun. In the second sequence, frames (d)-(f), we see a similar case from a different view. In both cases, the participating LW1 and LW2 take complementary actions to eliminate a common threat.

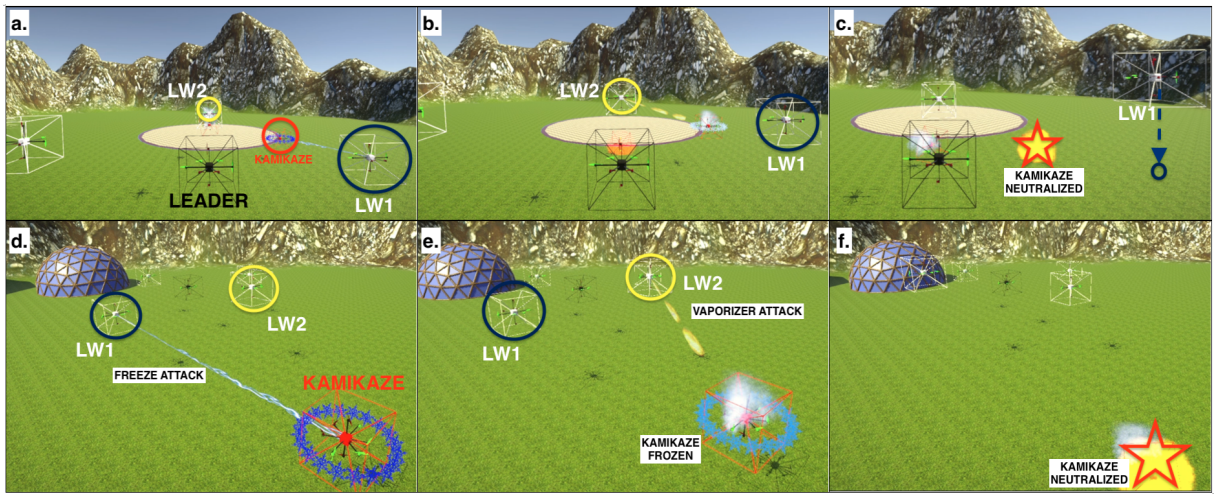

FIGURE 3.27 – Demonstration of a cooperative neutralization sequence. In this simulation, the leader is in seen in black. The LW1 are represented in the blue circle, LW2 in the yellow circle and the Kamikazes in red.

Now we present the sequence leading to the sacrifice of an LW, in Figure 3.28. The case begins in frame (a) with three Loyal Wingmen in formation, after LW4 is destroyed. In frame (b), we observe the LW1, which only has one cartridge remaining in its weapons, engaging two Kamikazes, note that after the neutralization of the first Kamikaze, it collides with the other one present in the attack, thus eliminating two threats. In frame (d) we have LW1 out of ammo and starting its sacrifice attack after a new threat enters the area. In frame (e) we have the successful elimination of the Kamikaze, and consequently, the leader readjusts the relative position in formation of LW2 and LW3. Finally, in frame (f) we have a new MUM-T formation composed only by the remaining LW2 and LW3.

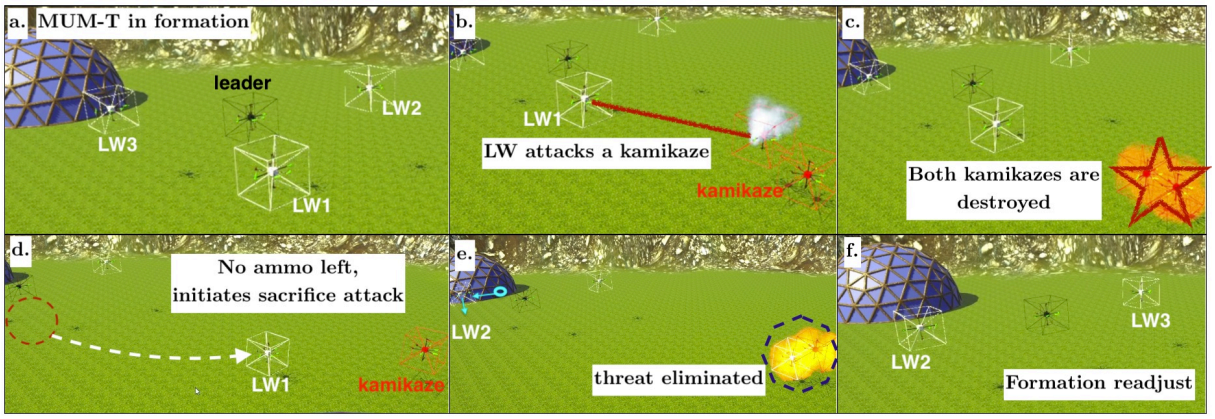

FIGURE 3.28 – Demonstration of the sacrifice neutralization sequence. In this simulation, the leader is in seen in black. The LW are represented in white and the Kamikazes in red.

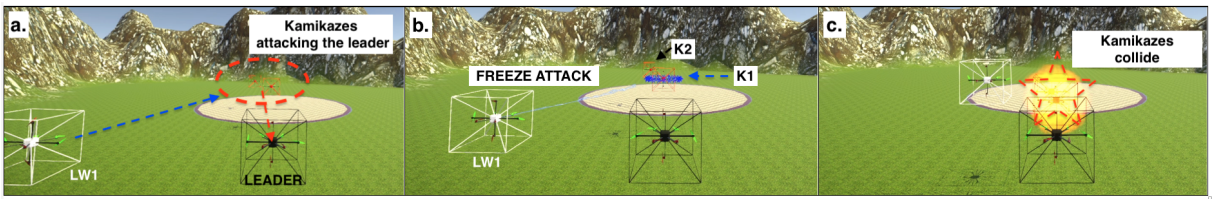

FIGURE 3.29 – Freezing attack being used as a neutralization method. In this simulation, the leader is seen in black. The LW1 in white and the Kamikazes K1 and K2 in red.

Figure 3.29 illustrates how unexpected behaviors can arise in a multi-agent system. In frame (a) we observe two Kamikazes heading toward the leader. In the sequence, in frame (b), we have LW1 identifying and freezing the closest threat K1. As a consequence, we observe that K1 collides with K2, since K1 has been frozen and K2 which is closely following K1 cannot avoid the collision in time. This indirect neutralization of K2 by freezing K1 highlights how emergent behaviors can have unexpected consequences in a multi-agent system.

3.6.3.3 Monte Carlo Results

Figure 3.30 displays the insightful outcomes derived from the Monte Carlo simulation. The average survival time, excluding the impact of the sacrifice attack behavior, amounts to an impressive 169.3 seconds, while the number of Kamikazes eliminated reaches a commendable count of 34. This achievement is particularly noteworthy, considering that the MUM-T possesses a vaporizer gun on each LW with a limited ammunition capacity of 10, rendering it capable of directly neutralizing a maximum of 40 Kamikazes. Consequently, these results reflect a remarkable utilization of the MUM-T's neutralizing capability, accounting for 85% of its total potential.

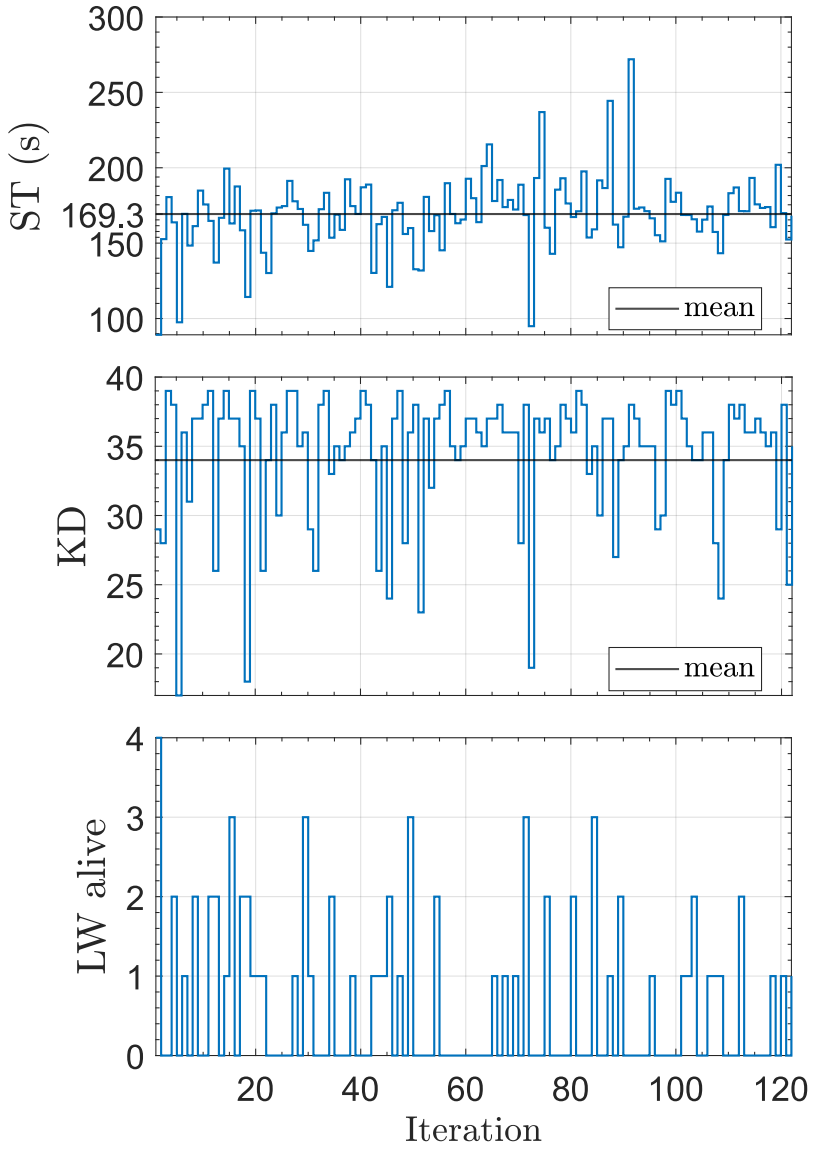

FIGURE 3.30 – Monte Carlo simulation results. ST stands for survival time and KD stands for Kamikazes destroyed.

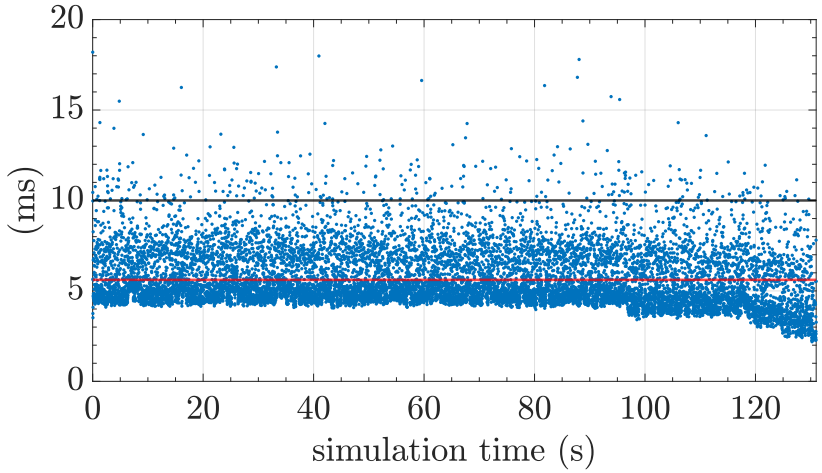

FIGURE 3.31 – Overall simulation iteration time. Legend: • iteration time, red line is the iteration time average, and block adopted sampling time.

3.6.3.4 Computational time

To assess the computational burden of the simulation, we conducted a typical run using the same parameters as those in Subsubsection 3.6.3.2. This allowed us to estimate the time and computational resources required for the simulation. During this run, we executed both the LL and HL modules for all the MAVs on a single computer. Specifically, we used a laptop equipped with a 12th generation Intel Core i7-12700H processor running at 2.30 GHz, with 16 GB of RAM. The total simulation time, from the beginning to the leader destruction, was about 131 s. In only 3.4% of a total of 13100 time steps, the sampling time of 0.01 s was exceeded.

Similar to the computational analysis conducted in Dini e Saponara (2022), we plotted the iteration time against the simulation time in Figure 3.31. This plot also includes the average iteration time and the adopted sampling time. Overall, the results indicate that the adopted sampling time is rarely violated in most iterations. It is noteworthy that these results are satisfactory, as real-time performance is achieved even with a code that is not optimized for embedded implementation.

4 Search and Rescue Application

In this chapter, we delve into the field of search and rescue (SAR) operations using Unmanned aerial vehicles (UAVs). With an increasing number of disaster events and emergency situations worldwide, traditional search and rescue methods often face challenges in terms of time efficiency, accessibility, and risk management. By exploring the potential of drones as versatile and agile tools, we investigate their capability to revolutionize search and rescue operations. This chapter encompasses an analysis of the underlying AI techniques to face the challenges associated with the usage of drones for search and rescue operations, as well as showcases a simulated application and the significant impact they have had on enhancing the effectiveness and safety of rescue missions.

In the following sections, we outline the key components of our study. Firstly, we introduce the scenario developed for our investigation, providing necessary context. Next, we delve into the problem to be solved, highlighting the specific challenges we aim to address in the domain of search and rescue. Moving forward, we present our devised search solution and the decision-making process tailored for both individual drones and the collective swarm. This approach encompasses autonomous decision-making capabilities, as well as collaborative strategies to optimize the overall search and rescue mission. Finally, we present the experiments and results. Our evaluation is divided into two distinct approaches. The first approach centers around the development of autonomous and collective decision-making, emphasizing their ability to effectively navigate and coordinate within the swarm. The second approach focuses on the evolution of the search strategy itself, exploring improvements and refinements that enhance the search efficiency by adding a probability map to the search.

4.1 Search and Rescue Scenario

Developing a search and rescue scenario using drones offers a valuable opportunity to study and address the intricacies of this complex problem. By simulating various scenarios and evaluating the performance of drone swarms in locating missing individuals, we can gain valuable insights into the effectiveness of different search strategies and algorithms.

This scenario development involves designing environments, incorporating factors such as obstacles, and limited visibility to mimic real-world search scenarios. Overall, it offers a valuable platform for in-depth analysis and the advancement of strategies to improve the efficiency and effectiveness of search and rescue operations.

As an application of the search problem, we propose a scenario, seen in Figure 4.1, where a UAV swarm is applied in a search mission to find a missing individual in a forest region of approximately $60.800\,m^2$. The scenario area is divided into $(38 \times 16\,m)$ squares, where each square is equivalent to $100\,m^2$ $(10 \times 10\,m)$. N represents the number of UAVs involved in the search mission within this area.

As the swarm's fundamental assumption, we consider that the UAVs lack prior information regarding the position of the missing target. Furthermore, it is essential for the UAVs to avoid collisions with one another and navigate around obstacles present in the surroundings, specifically represented as trees. The drones operate within a shared UAV swarm network, enabling the exchange of mission-related data, including the positions of all mission components and the location of the identified target. The target is deemed located when a drone is within a distance of less than two squares (less than 20 meters), as depicted in Figure 4.1 by a circular representation around the target.

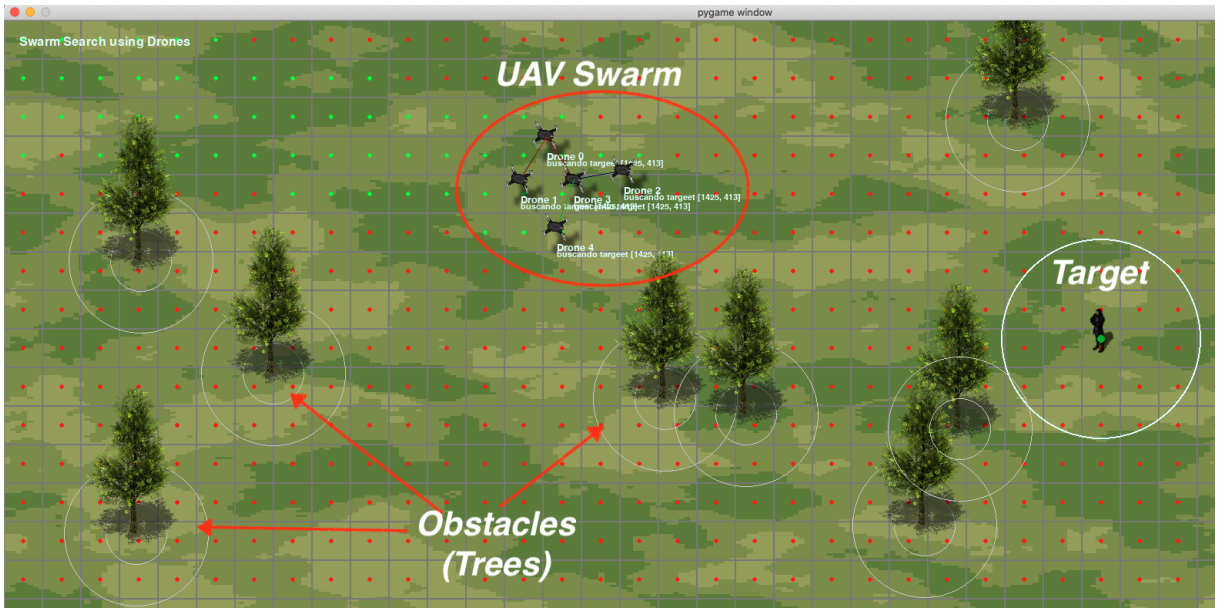

FIGURE 4.1 – Search and Rescue scenario, a swarm of drones, highlighted within the red circle, is deployed. The environment is populated with obstacles, symbolized as trees, while the primary objective is to locate and rescue a lost person, represented as the target.

4.1.1 Mission

Therefore, the mission for the search and rescue swarm present in the scenario is:

The drone swarm is tasked with efficiently scanning the designated search area to identify the location of the missing person.

And, the conditions to end the mission of the search are:

- One or more drones located the missing person, and;
- 80% of the drones are within the range of R_{person} of the missing person;

Note that, when a person is identified, the patrolling drones will immediately alert the commander of the mission. The commander, in turn, will promptly assess the situation and take appropriate measures to successfully rescue the person and conclude the mission.

4.1.2 Description of the entities

The following list presents a description of the entities.

4.1.2.1 Drone Swarm

- The agents in question possess complete autonomy, operating independently, and the simulation accommodates a predetermined and unchanging quantity of UAVs.
- The UAVs are equipped with inexhaustible battery capacity, abstracting the gradual depletion of charge from the UAVs' batteries. Note that this is a hard assumption taken in order to focus on the intelligence aspects of the problem, which is the main focus of this work.
- The UAVs are equipped with an advanced vision system that possesses an idealized capability to identify the missing person with 100% accuracy.
- To facilitate accurate target identification through the vision system, flight restrictions are imposed to ensure that the drones maintain a lower altitude than the treetops. This restriction enables the UAVs to effectively detect targets within a range of 10 meters, optimizing their ability to locate and identify the target.
- The UAVs establish communication through a shared network, enabling them to exchange information and access an identical copy of the map. This synchronized map allows the drones to update and read the information in this map.

4.1.2.2 Missing Individual

- The individual is someone who has become disoriented and is currently immobilized within the region of interest.
- At each new iteration, a random position is chosen for the individual on the map, which remains fixed throughout that iteration, i.e. stationary target.

4.1.2.3 Centralized Ground Control System (GCS)

- The ground-based radars share situational awareness with the drones, *i.e.* a vector with the state (pose and velocity) of all entities. In simulation, it is represented as messages carried out between entities.
- Ground-based radars detect and track the drones up to a limited range delimited by the desired region of search.
- The communication is abstracted as perfect, with no delays or data losses.

4.2 Problem Breakdown

In order to optimize patrolling in autonomous scenarios, the task can be broken down into two sub-tasks: low-level and high-level patrolling. Low-level patrolling involves using sensors to detect objects of interest within the area. Meanwhile, high-level patrolling focuses on directing the search towards specific subregions of the area that are more likely to contain the object of interest. This can be accomplished by utilizing additional information, such as previous search patterns and knowledge about the types of objects that are typically found in certain areas. By employing a combination of these tactics, an autonomous system can effectively and efficiently search the patrolling area while minimizing the risk of overlooking crucial objects or areas. Therefore, the task can be broken down into:

- Low-Level Strategy and Tactics for anomaly detection and classification. An efficient patrolling drone is able to identify anomalies and classify them including a number of properties, e.g. speed and orientation.
- High-Level Strategy and Tactics for searching the area of interest. An essential part of a patrolling task is to perform the task as efficiently as possible with respect to identifying the target and minimise the risk of missing an object of interest.

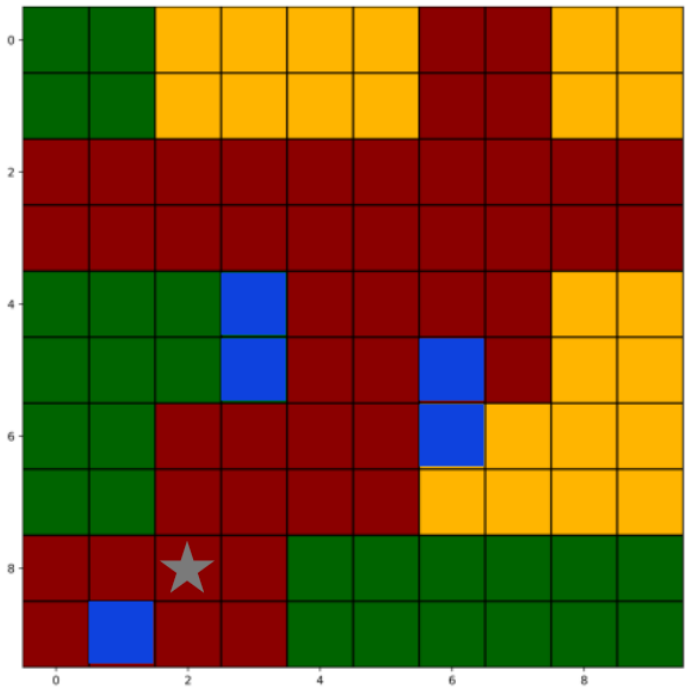

FIGURE 4.2 – Example of a probability map. Green, Yellow and Red cells represent low, medium and high probability regions, respectively. No flight zones are illustrated as Blue cells. The star represents the object of interest.

Figure 4.2 illustrates an example of a probability map used in the search and rescue scenario. The map is color-coded to represent different levels of probability to successfully locate the missing person. Green cells indicate regions with low probability, yellow cells represent areas with medium probability, and red cells are high probability regions. Additionally, blue cells are designated as no flight zones, indicating areas where the drones are not allowed to fly. Also, the star symbolizes the object of interest, which is the target in the search area whose location is unknown to the drones. By analyzing this probability map, the drones can make informed decisions regarding their search patterns and prioritize regions with higher probabilities, enhancing the efficiency of the search and rescue operation.

Anomaly detection and image recognition are rapidly advancing fields, with potential applications in patrolling. Dantas *et al.* (2022) demonstrated this potential by adapting these techniques to patrolling tasks. Building on this foundation, our current work aims to address high-level strategies and tactics for patrolling in this context.

We assume a previously provided probability grid map of the search region, as shown in Figure 4.2. In this map, the region is divided into equally sized cells, where each cell represents the likelihood of finding a person in that sub-region. The probabilities are previously estimated by a human expert and are updated simultaneously by the search UAVs using search algorithms. To ensure seamless coordination in the swarm, we assume the drones share the same map and are capable of updating the probabilities on the map

simultaneously. Furthermore, we assume that the drones are equipped with advanced sensors that can effectively identify the object of interest.

4.2.1 Problem Definition

The primary goal of the mission is to locate a target whose existence was previously unknown. To achieve this objective, we employ a swarm search strategy coupled with an intelligent heuristic. Our evaluation focuses on understanding how the search time for an unknown target is impacted by factors such as the number of drones involved and the presence of obstacles. We establish a baseline for the shortest possible time required to locate the target by assuming the drones possess knowledge of its position and can directly move towards it.

Furthermore, we need to investigate the influence of the proposed heuristics and the implementation of swarm intelligence on metrics of interest. These metrics include the mission time, which means the point when all drones have successfully reached the location of the unknown target, as well as the occurrence of local minima, where drones become unable to move. By analyzing these factors, we gain insights into the effectiveness of our approach and its ability to optimize the search process.

4.3 Search Solution Development

In this section, we present and discuss the solutions for the search and rescue scenario problem. We introducing the solution for autonomous decision-making in individual drones, highlighting how each drone independently makes decisions to its search process. Following, we delve into the solution for collective decision-making within the swarm, showcasing how the drones collaborate and coordinate their actions to maximize the efficiency of the overall search and rescue mission. Furthermore, we discuss the adaptation of Bayesian search techniques specifically tailored for the search and rescue problem. This approach utilizes the probabilistic framework to enhance the search strategy and improve the chances of locating the target efficiently. Finally, we explain the process of utilizing a probability map, which provides a representation of the likelihood of finding the target in different sections of the map. We explore how this map is dynamically updated through the utilization of Bayesian search techniques, ensuring that the drones adapt their search patterns based on the evolving information and improve their search efficiency.

4.3.1 Decision-Making for the Search and Rescue Drone Swarm

Our decision-making can be divided into two categories: autonomous intelligence, referring to a single UAV, and collective swarm intelligence, referring to the group. Therefore, decisions are decentralized to each UAV, and cooperative, as the group must cooperate to attain the mission. The implemented intelligence modules are described in Table 4.1.

TABLE 4.1 – Proposed heuristics for Swarm Intelligence.

Swarm Intelligence	**Module**	**Description**
Autonomous Intelligence	Target Search	Target search is performed on a discrete map.
	Behavior	Intelligent drone behavior through a finite state machine.
	Collision Avoidance	Potential fields for collision avoidance between drones and obstacles.
Collective Intelligence	Swarm Search	Target search in regions mapped as not visited using a common map.
	Blockage Prevention	Prevents drones from getting stuck in local minima present in the scenario.
	Mission Closure	When a drone finds the target the coordinates are shared and all drones move to this position.

4.3.1.1 Autonomous Intelligence

In the target search module, the UAVs are equally distributed among the scenario lines according to the number of drones used, and each drone begins the search for the target autonomously, by searching in a line on the neighboring cells. Thus, the drones perform a horizontal sweep across the scene.

The behavior module was developed to control the autonomous behavior of the drones, consisting of a finite state machine (FSM), with each state representing a behavior, only one state is executed at a time, with events leading to behavior changes (COLLEDANCHISE; OGREN, 2018). Our FSM for the search and rescue UAV can be seen in Figure 4.3, structured with two main states: `SearchState` and `SeekState`. There are also two auxiliary states: `GoToClosestDrone` and `RandomTarget`, used to avoid the blockage by local minimuma. Note that there is a FSM for each drone in the swarm.

By default, the drone operates in the `SearchTarget` state, representing its behavior of searching for the missing target. Upon locating the target, either by the drone itself or by another agent in the swarm, the drone transitions to the `SeekState`. In this state, the

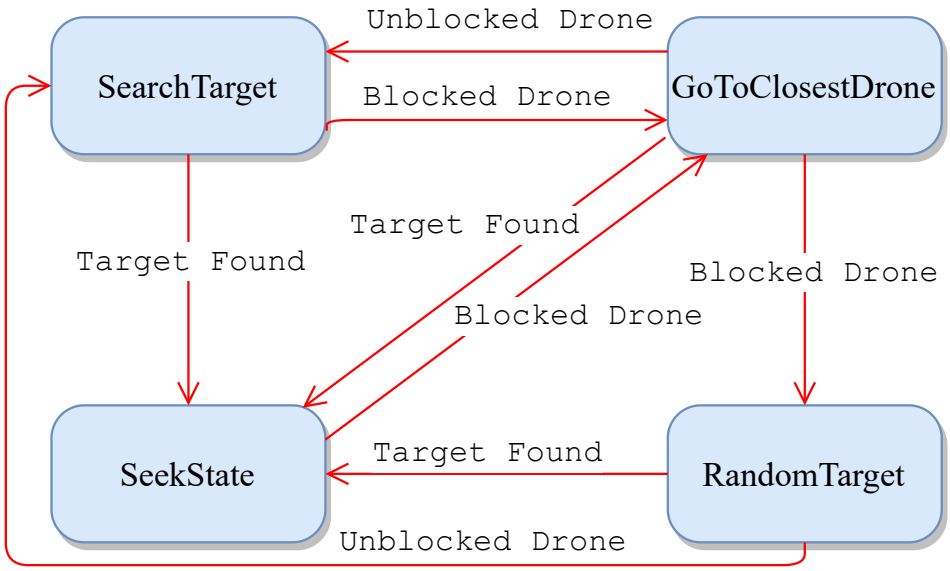

FIGURE 4.3 – Finite State Machine that controls the drone's behavior.

target's coordinates are discovered and shared with all agents in the swarm. Subsequently, the drone ceases its search and proceeds directly to the identified coordinates.

In the event that the drone encounters an obstacle hindering its movement in a specific region of the map, the agent detects the lack of motion, by analysing the velocity of the vehicle and the time without any representative movement, and triggers a transition to the `GoToClosestDrone` behavior. In this state, the UAV navigates towards the coordinates of the nearest agent in the team, employing it as an initial heuristic to resolve the situation.

As a final resort, if the drone remains blocked despite attempting to reach the closest agent, it transitions to the `RandomTarget` state. In this state, the drone selects a random target within the map to navigate towards, aiming to free itself from the obstruction. Once the drone becomes unblocked, the FSM reverts back to its default `SearchTarget` state. Notice that the drone can transition to the `SeekState` from any state in the FSM.

As part of its autonomous intelligence, there is also a dedicated module for collision avoidance. The potential field technique, described in Section 2.6.1, which employs the normal bivariate function, was applied to calculate repulsive forces among UAVs and obstacles, efficiently solving this problem. Through experimentation, a value of $\alpha = \gamma = 0.0001$ was found to be efficient for the UAVs' field force, while $\alpha = \gamma = 0.005$ was determined to be effective for collision avoidance with obstacles.

In Figure 4.4, the execution of the potential fields (PF) technique is depicted through simulation. The trajectory of each drone is traced on the map, clearly demonstrating their ability to avoid collisions both between agents and with obstacles. Notably, the gray circumference surrounding the tree and drones highlights the area of influence of the potential field.

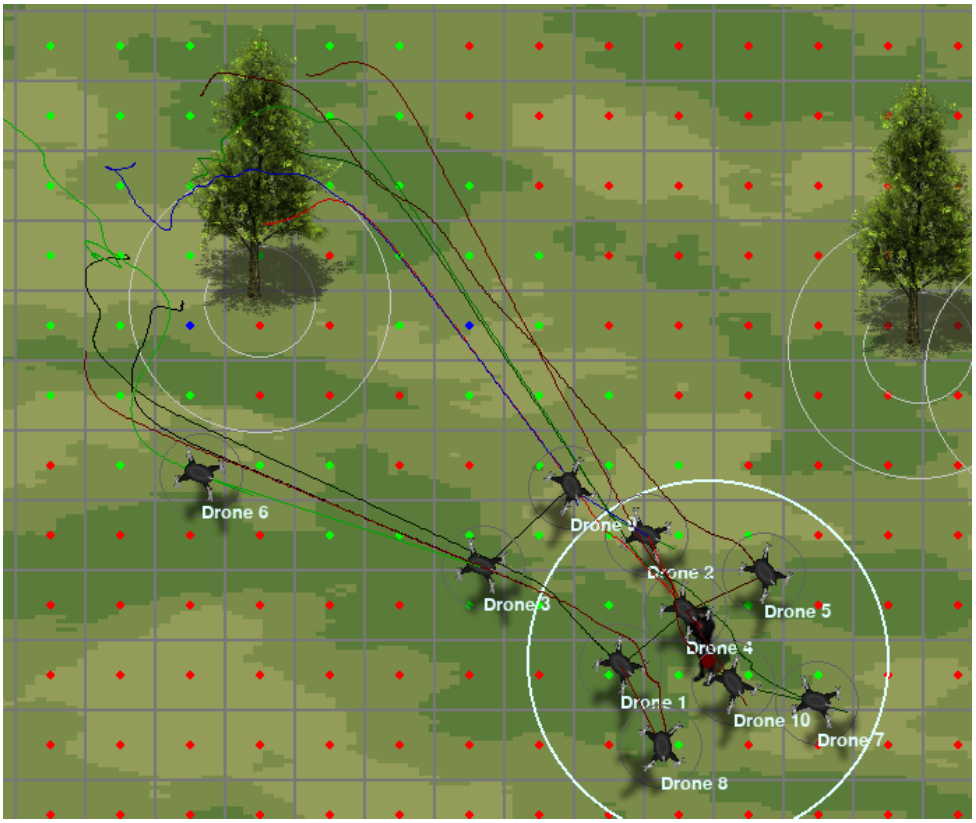

FIGURE 4.4 – Drones avoiding collision during simulation execution.

4.3.1.2 Collective Intelligence

In the collective intelligence module, we have the swarm search module, responsible for selecting unvisited cells of the shared map. Each cell $h(w, l)$ can have one of two possible states, indicated as

$$h(w,l) = \begin{cases} 1 & \text{if visited,} \\ 0 & \text{otherwise,} \end{cases} \tag{4.1}$$

where w varies from 1 to 38 and l varies from 1 to 16, according to the discrete area of the scenario. The default state is unvisited, when a cell is visited, the agent informs the swarm and the cell state is changed.

Figure 4.5 illustrates the marking of visited cells on the map, represented by a green center dot, while cells not visited by any member of the swarm are marked in red. The map is collaboratively constructed and shared among all agents. As new cells are explored, the map is continuously updated and shared, ensuring that all agents have an identical representation throughout the execution.

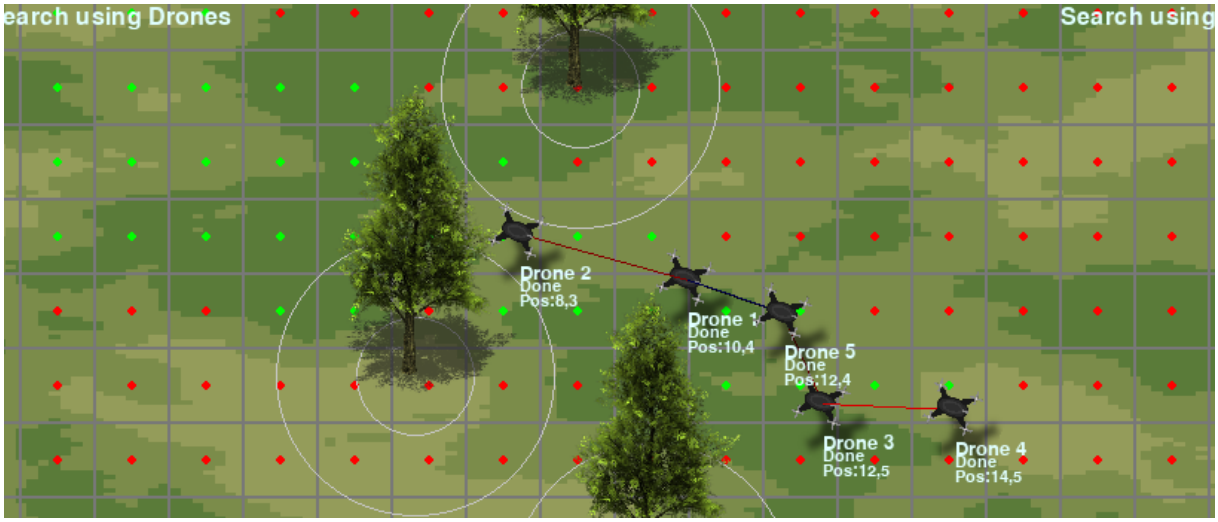

FIGURE 4.5 – To provide clear visual indication, we represent visited cells in green and not visited cells in red.

The mission starts by distributing the UAVs evenly across the rows of the map, as depicted in Figure 4.6. Subsequently, the search for unvisited cells performing the lawn mower search commences, with each agent selecting an unvisited cell from among its eight neighboring cells (eight-connected) to proceed.

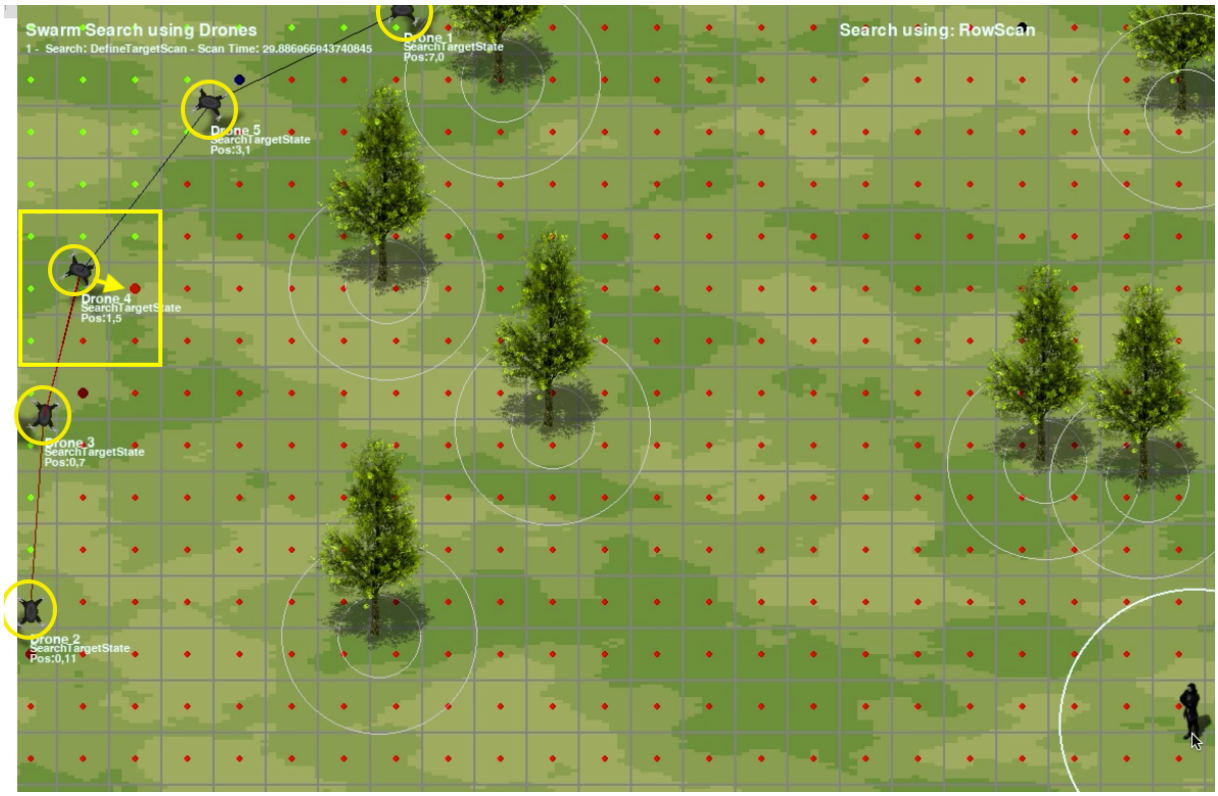

FIGURE 4.6 – The mission commences with the deployment of UAVs, which are clearly identifiable as they are marked in yellow and distributed across the map. In the designated yellow square, we observe a drone actively selecting an unvisited cell among the eight neighboring cells to continue the search process.

In the scenario where all eight neighboring cells of a UAV have already been visited, the algorithm resorts to selecting a random unvisited cell on the map as the next destination. This case is depicted in Figure 4.7, where a UAV with eight visited cells is displayed within the dashed rectangle at the bottom. The agent proceeds by choosing a random unvisited cell on the map to continue its exploration, as indicated by the yellow arrows.

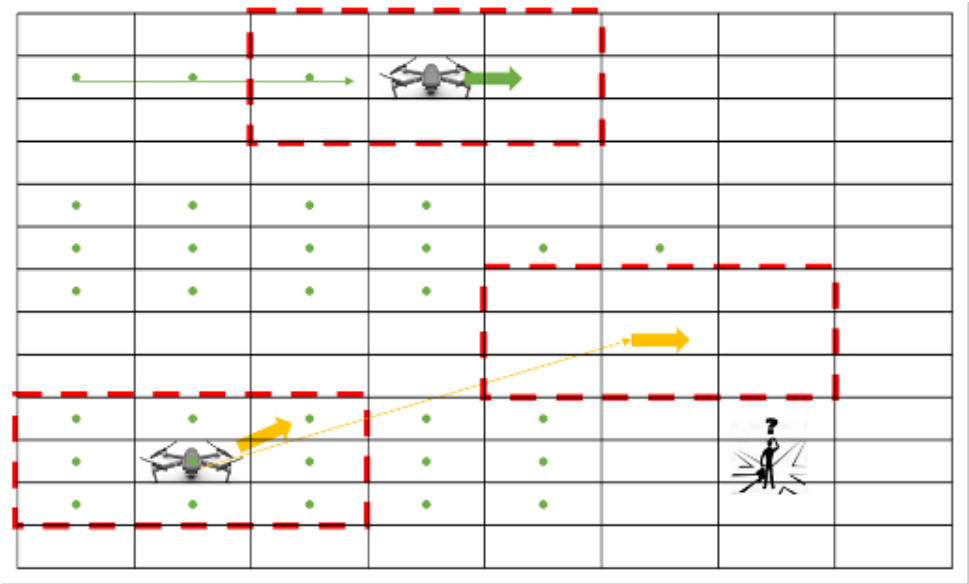

FIGURE 4.7 – Implementation of the unvisited cell search. We can see a drone in a position where it is surrounded by visited cells. In this case, the drone will randomly select a cell to proceed, illustrated by the yellow arrows.

The collective intelligence of the drone swarm utilizes heuristics to overcome potential field's local minima, referred to as the *Stationary State*. This state prevents a drone from becoming immobilized in a particular location within the scenario. When a UAV recognizes that it is unable to move further due to being trapped in a local minimum i.e., enters the *Stationary State*, it initiates a search within the UAV swarm network for the nearest available UAV and attempts to navigate towards its coordinates. This process is illustrated in Figure 4.8. If the UAV in the Stationary State is still unable to move after this attempt, it proceeds to search for the second closest drone and endeavors to move towards it. If the drone remains blocked, the agent selects a random cell on the map and repeats this procedure until it exits the *Stationary State*, returning to the default *SearchTarget* state.

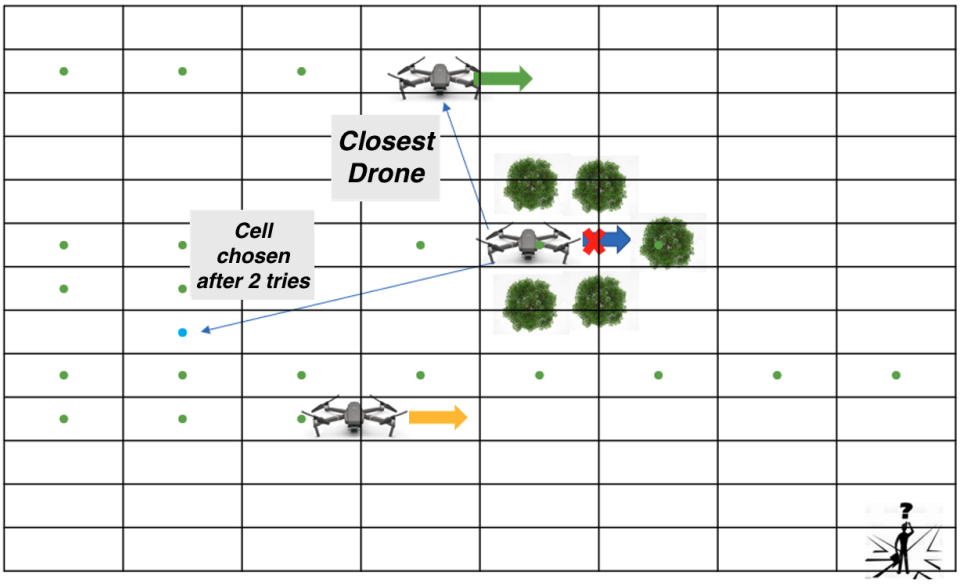

FIGURE 4.8 – Drone stuck in obstacle looking for the nearest drone.

When a drone successfully locates the target within a distance of two squares, the entire UAV swarm initiates the mission closure phase. During this phase, the UAV that identified the target shares its coordinates with the other drones in the swarm network. This information dissemination ensures that all drones align their movement towards the identified target coordinates, as depicted in Figure 4.9. By coordinating their actions, the swarm of drones collectively converges towards the target, facilitating efficient mission completion.

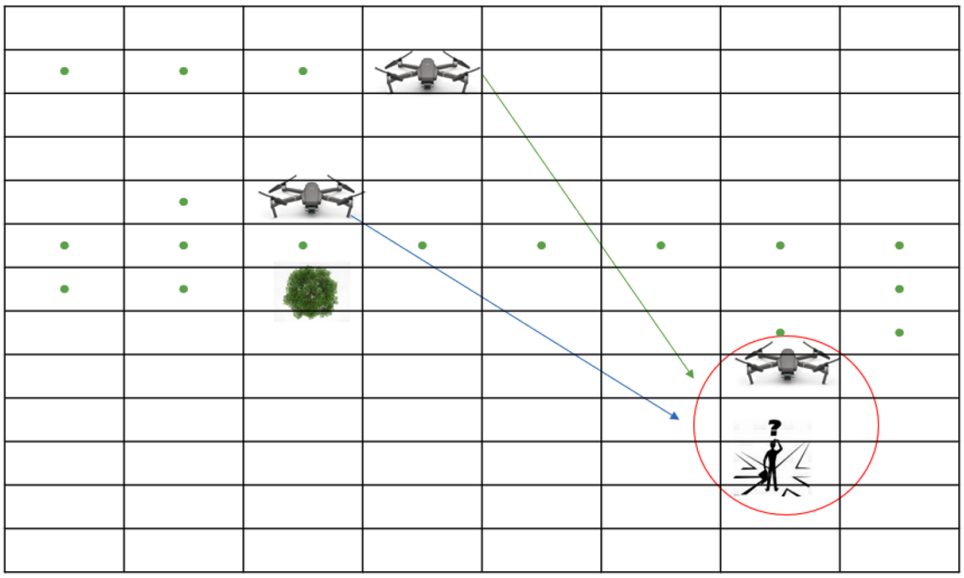

FIGURE 4.9 – Mission closure phase, target is located by one UAV in the swarm. The drone then shares the location on the person and all drones navigate towards the individual.

Figure 4.10 showcases the simulation of the swarm in the mission closure phase. Notably, the successful sharing of the target location within the swarm network can be

observed. All UAVs effectively navigate towards the shared target coordinates, effectively concluding the mission. It is worth noting the cohesive formation of the swarm around the target, where the UAVs maintain their relative positions without any collisions. This coordinated movement ensures a smooth and efficient execution of the mission objectives.

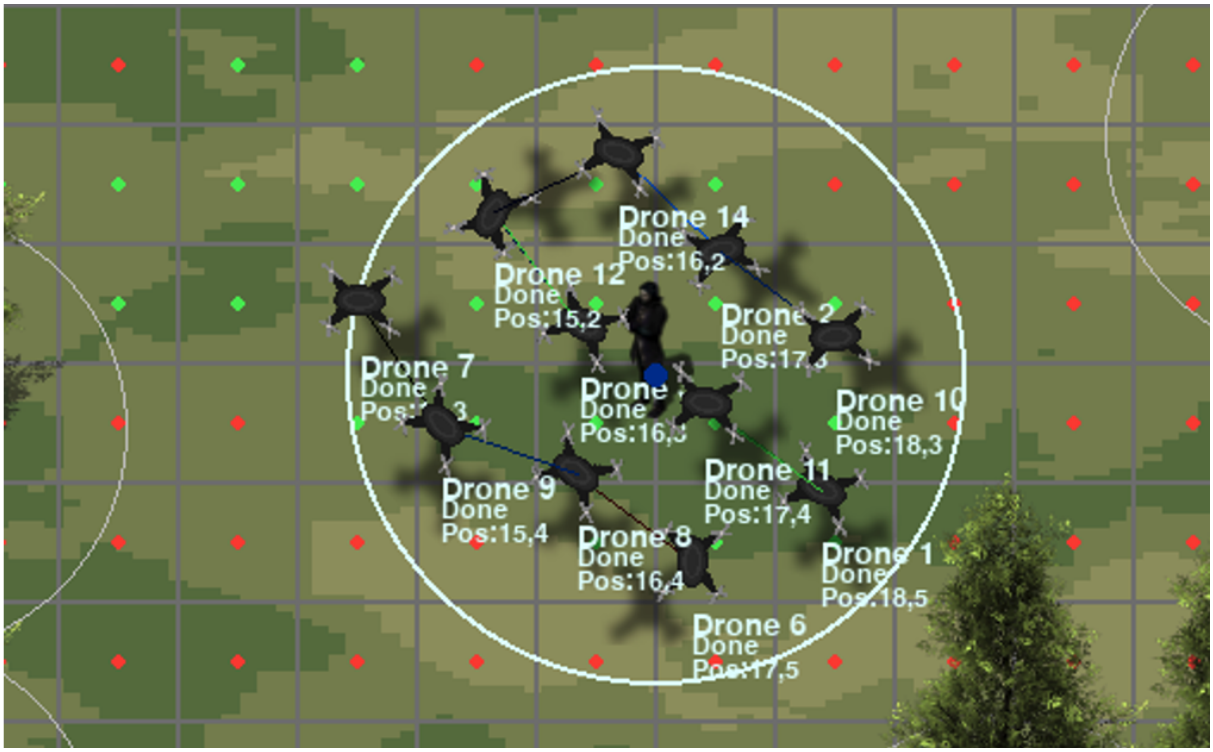

FIGURE 4.10 – Target is located and a swarm is formed around the missing person.

4.3.2 Bayesian Search for Search and Rescue

The estimation of the probabilities of each cell during the initial stages of a search mission relies on assumptions made before any data is collected. These assumptions are based on expert knowledge. The search process involves several steps.

First, the search area is divided into a grid map, and each cell is assigned a priori probabilities. These probabilities represent the likelihood of the target being present in each cell. Those probabilities are divided into three groups: low (0% - 33%), medium (34% - 66%), and high (34% - 99%).

The second step involves searching the cell with the highest probability. If the target is found in that cell, the search mission is completed successfully. However, if the target is not found in the selected cell, the search proceeds to the next step.

In the third step, the probability value of the cell where the target was not found is reduced, while the probabilities of the remaining cells are increased proportionally. This adjustment is made to reflect the updated information obtained from the search.

The process is then repeated iteratively until the target is located. The underlying principle guiding this iterative search process is the application of the Bayes formula. The Bayes formula provides a framework for updating the probabilities of the cells based on the observed data. It is defined as follows:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{P(B \cap A)}{P(B)}, \tag{4.2}$$

where A and B represent events or conditions. $P(A)$ and $P(B)$ are the probability of the events A and B, respectively. Also, $P(A|B)$ represents the probability of event A given event B, $P(B|A)$ is the probability of event B given event A, and $P(B \cap A)$ represents the joint probability of events B and A occurring together (the intersection of events B and A). Specifically, A denotes the event that the target is in the search area, while B represents the event of the target not being found. To calculate the probability of the target being in the search area given that it was not found, we can use adapt the formula to:

$$P(\text{Is in the search area}|\text{Not found}) = \frac{P(\text{Is in the search area AND Not found})}{P(\text{Not found})}. \tag{4.3}$$

To update the probabilities of the cells, let us introduce the variables: p, the probability of the target being in a cell, and f, the probability of finding the target if it is present in the cell. With these variables, we can define the probabilities $P(B)$ and $P(B \cap A)$ as

$$P(B) = P(\text{Not found}) = p \cdot (1 - f) + (1 - p), \tag{4.4}$$

$$P(B \cap A) = P(\text{Not found} \cap \text{Is in the search area}) = p \cdot (1 - f), \tag{4.5}$$

The posterior probability of a searched cell, denoted as p', can be updated using the formula:

$$p' = \frac{p \cdot (1 - f)}{1 - pf}. \tag{4.6}$$

Similarly, the posterior probability of the other cells, denoted as r', can be updated using:

$$r' = \frac{r}{1 - pf}, \tag{4.7}$$

where r represents the individual probability of all other cells. By iteratively updating the probabilities of the cells using these equations, the search process dynamically adjusts the probabilities based on the observed data, leading to an improved estimation of the target's location.

4.3.3 Search Process using a Probability Map

At the start of the search process, the drones communicate with one another to select different cells on the map with high probabilities and navigate towards it to begin the search process, as shown in Figure 4.11. Once the drone reaches the intended location, it actively initiates the search process using Hill Climbing (HC) search and Bayesian Search (BS) algorithms.

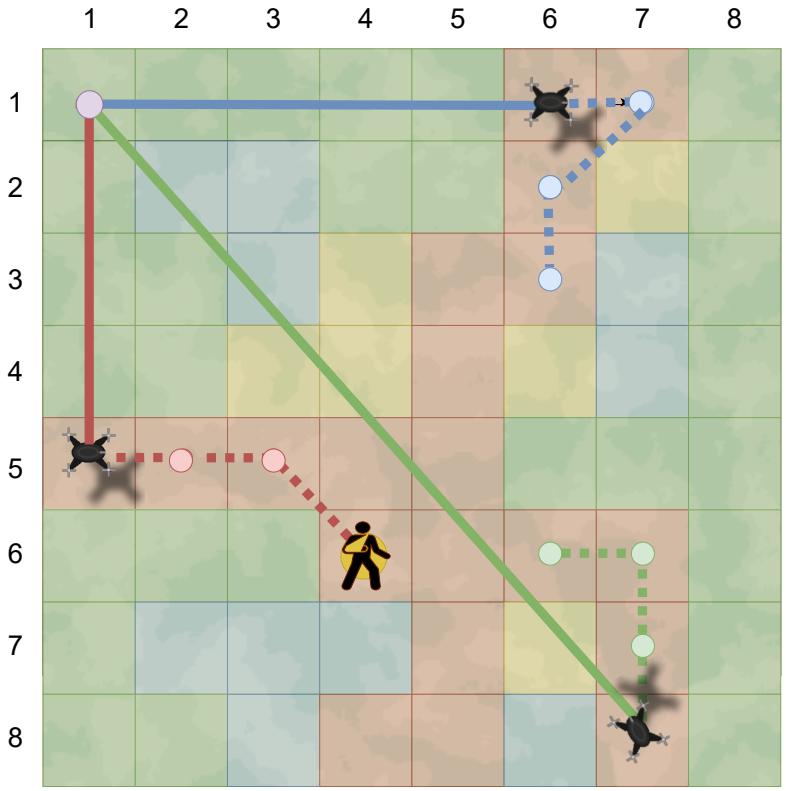

FIGURE 4.11 – Example of search process using three drones. Green, yellow, and red cells represent low, medium, and high probability regions, respectively. No flight zones are illustrated as blue cells. The yellow circle with a person represents the object of interest. Each line with a different color represents the path taken by a different drone. Gray circle represents the start point.

In Figure 4.12, we can observe the search process being performed by the drone. The drone follows a strategy where it visits a cell and, in case the object is not identified, it examines the probabilities of the 8 neighboring cells. Then, using the HC algorithm, the drone moves to the cell with the highest probability to continue the search.

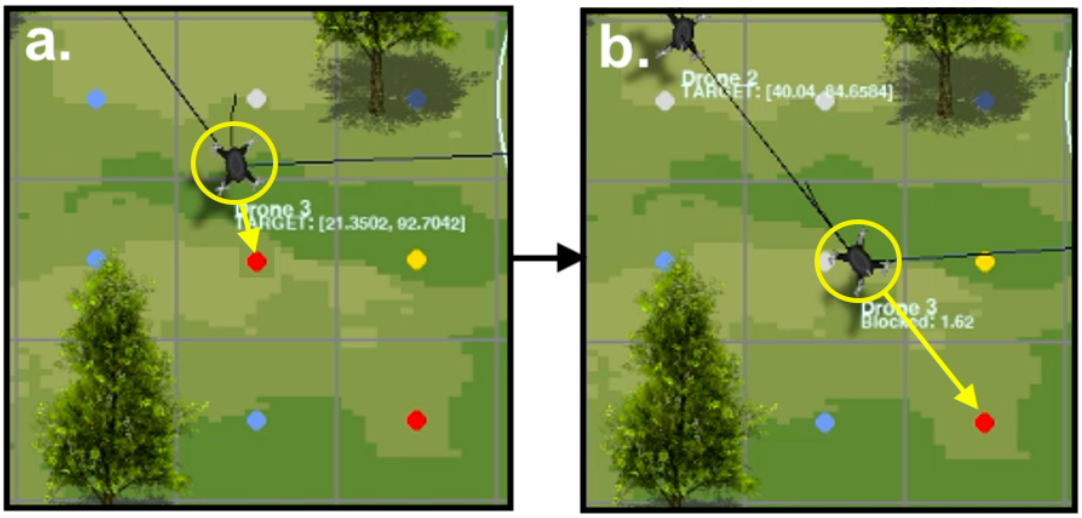

FIGURE 4.12 – Drone performing the search in the 8-connected cells. Red, Yellow and Blue cells represent the high, medium and low probabilities respectively. In Gray, cells that were already visited.

In Figure 4.13 we illustrate the Bayes algorithm updating the cell probabilities in a simulation. The probabilities of all map cells are updated at each step using the Equations (4.6) and (4.7). As seen in frames (a) and (b), the probabilities increase as the drone visits new cells without finding the object. During each iteration of the simulation, a new probability map is created randomly. The probability for each cell in the map is also randomly assigned. The location of the individual is also updated to a new high probability cell in the newly generated map. Also, new obstacles are randomly positioned in the map, which helps to evaluate the algorithm's performance under different conditions. The random assignment of probabilities to each cell ensures that the simulation results are unbiased towards any particular scenario.

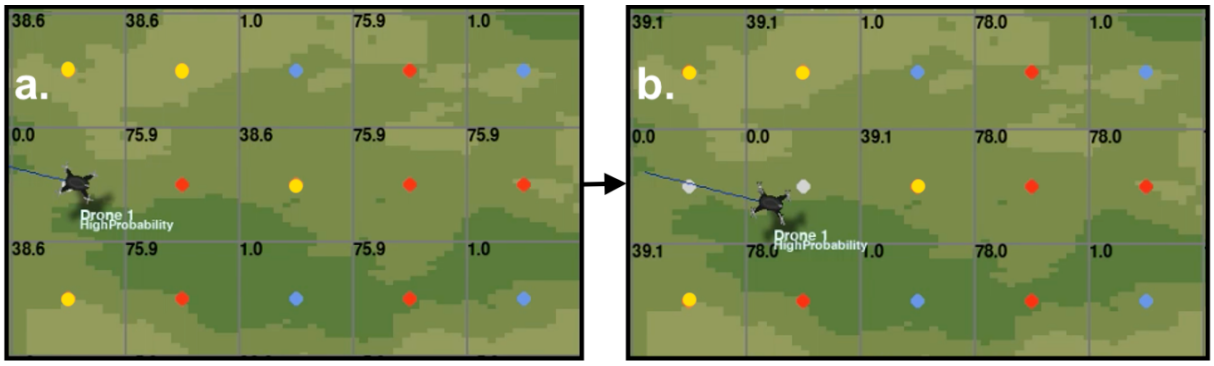

FIGURE 4.13 – Drone performs the search using the Bayes algorithm and updates the probabilities associated with each cell as it visits them. These probabilities correspond to the numbers assigned to each cell and are updated at every new cell the drone explores. Note that these probabilities are multiplied by 100.

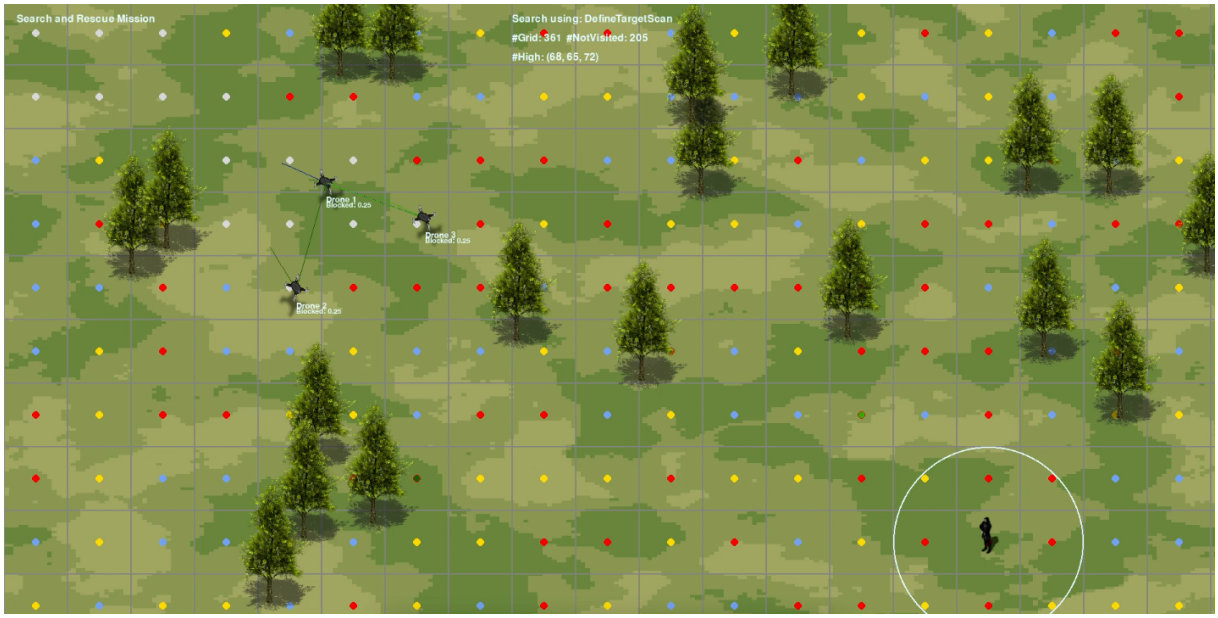

FIGURE 4.14 – 2D Simulation view of the search mission in using Bayesian probabilities and HC search.

4.4 UAV Search Swarm Simulation

As the first step in the development process, it is necessary to define the simulation environment in which the search scenario is implemented. Similarly to the 2D simulator of the loyal wingman scenario, the simulator was developed using the same framework in Python, leveraging the *Pygame* library for visual representation, as depicted in Figure 4.1. Our simulator aims to facilitate future search and rescue simulations and accommodate various scenarios through an easily accessible and modifiable open-source code. Interested readers can view a presentation video of our simulator[1], which provides an overview of its functionalities. Additionally, more implementation details can be found in our GitHub repository[2].

Our simulations aim to evaluate collaborative online mission planning algorithms. To compare the efficiency of different algorithms, we limit the scenario to 5 autonomous drones, which is sufficient as a proof of concept. We developed a 2D UAV search simulator, as shown in Figure 4.1 and Figure 4.14. The simulation focuses on search missions, where various search algorithms are tested to identify the location of the missing person in the patrolling area and complete the mission.

The simulation environment can be summarized with the diagram in Figure 4.15. We implemented a communication model for the swarm to ensure that all UAVs have access to information on which cells have been searched and a basic strategy for searching in a specific direction once all surrounding cells have been visited. The UAVs communicate

[1]https://www.youtube.com/watch?v=l07YPjrnLNw
[2]https://github.com/luizgiacomossi/Search_Drone_Swarms/tree/grid_search

through a shared map, using the whiteboard model of communication (MCCANNE, 1992) and a centralized ground control system, which enables each drone to read and update map probabilities simultaneously. Additionally, the simulation is connected to the 2D visualization engine that displays the current state of the simulation. By connecting the simulation to a visualization engine, we can monitor and adjust the simulation in run time. This is useful for testing different scenarios or optimizing the behavior of the drones within the simulation. Additionally, the potential field technique is implemented for collision avoidance and path planning, described in Section 2.6.1.

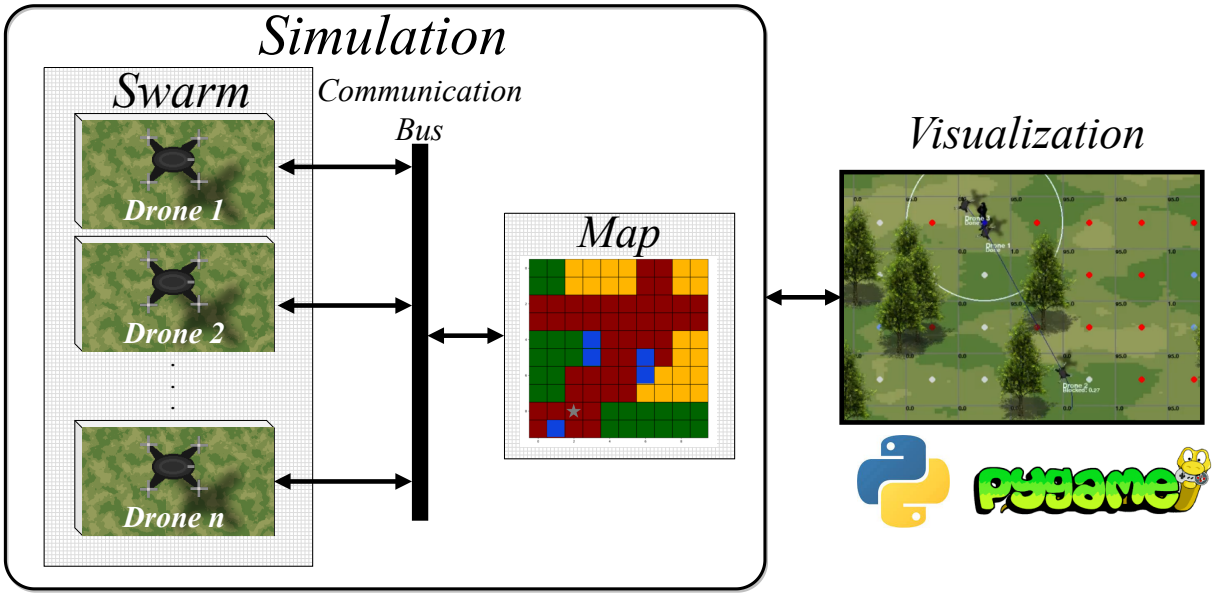

FIGURE 4.15 – High-level overview of the components and interactions within the simulation environment.

4.5 Results and Discussions

In this section, we present the details of our experiments utilizing the solution developed in the 2D simulator, along with their corresponding results. We have divided our experiments into two main approaches. Firstly, we examine the decision-making process using a discrete map, where the cells represent a visited or unvisited state. In this analysis, we focus on the intelligence for the swarm and for the autonomous drones, and the heuristics developed to address the issue of local minima that arise when employing the potential fields technique. Subsequently, we progress to further enhance our solution by introducing a novel heuristics approach, which incorporates a probability map, Bayesian search, and hill climbing techniques. This new approach aims to overcome the limitations associated with local minima and improve the overall performance of the system. The cells of the probability map represent the likelihood of finding the person in the cell. By adopting this two-step experimental methodology, we can thoroughly investigate and

evaluate the effectiveness of our developed heuristics for the autonomous and collective intelligence, while also introducing a more advanced and refined approach for the search strategy using the UAV swarm decision-making.

4.5.1 Experiments of the Decision-Making

In this experiments, the autonomous and collective decision-making developed are evaluated, using the discrete map of the region, containing only the states of the visited and non-visited cells. The evaluation is based on the computational performance evaluation methodology, proposed by Jain (1991). The Table 4.2 presents the factors and levels considered for the evaluation. We applied a complete factorial design for the experiments, with 27 possible combinations for 10 repetitions each, totaling 270 executions.

TABLE 4.2 – Experiments description.

Factors	**Drones**	**Obstacles**	**Search Strategy**
Level 1	5, 10, 15	10, 15, 30	Target location previously known and without heuristics – *TNH.*
Level 2	5, 10, 15	10, 15, 30	No knowledge of target location and without blockage heuristics – *NTNH.*
Level 3	5, 10, 15	10, 15, 30	No knowledge of target location and using blockage heuristics – *NTH.*

In this evaluation, the metrics observed are: mission completion time and the number of observed local minima. The mission is considered completed when all drones form a swarm around the located target, as shown in Figure 4.10. At level 1, we run the experiments with the coordinates of the target previously known, using this result as a baseline reference, as it can be considered as the best possible case, with the UAVs heading directly to the target.

4.5.1.1 Comparison of Mission Completion Time

In Figure 4.16 we have the bar chart of the average time with a confidence interval of 95%. For all cases, the strategies in level 2 and 3 presented a longer mission completion time compared to the level 1 reference. This result is expected, since there is no guarantee of optimal time with the use of heuristics for search.

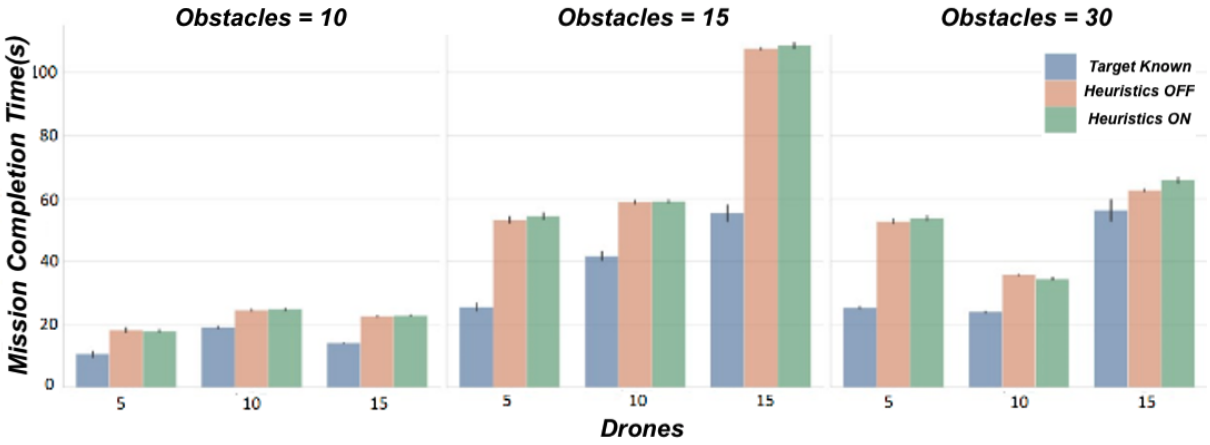

FIGURE 4.16 – Mission completion time by the number of drones used.

Therefore, we have observed that when the swarm employs our search strategy, there is a tendency for the UAVs to disperse throughout the area. This leads to some UAVs conducting searches in regions that are distant from the target location or encountering complex situations with multiple obstacles along the path to the target. As a result, the mission time increases. Additionally, our statistical analysis reveals no significant differences in mission times between level 2 and level 3 strategies. The level 3 strategy incorporates our blockage prevention heuristic into the agents' operations.

Note the case of the experiment with 15 obstacles, which, compared to the one with 30 obstacles, had higher completion times, especially in the experiments with 10 and 15 drones. This can be explained by the complexity of the simulated scenario. For example, counter-intuitively, when compared to the scenario with 30 obstacles, you can encounter more challenging search scenarios with 15 obstacles, where the target is found in areas with very closely spaced obstacles, creating significant barriers. And therefore, causing an overall increase in completion time, even with the usage of the heuristics. But overall, we observed that it was a good performance in almost all experiments when compared to the baseline reference values obtained when the simulation was run with the known target.

4.5.1.2 Comparison of local minima quantities

In Figure 4.17, we present a chart depicting the average number of local minima encountered. We can observe that as the number of obstacles and UAVs increases, there is a tendency for the average number of local minima to increase as well. However, when our swarm utilizes our search heuristics, the number of local minima is significantly reduced, as evidenced by the results from level 2 and 3 strategies.

It is worth noting that in the reference level 1 case, the UAVs were more prone to being blocked. This can be attributed to the fact that the UAVs were directly heading

towards the target in a swarm formation, leaving little room for maneuvering. Furthermore, the force field exerted by other UAVs in the vicinity could have a detrimental effect, exacerbating the number of blocked drones.

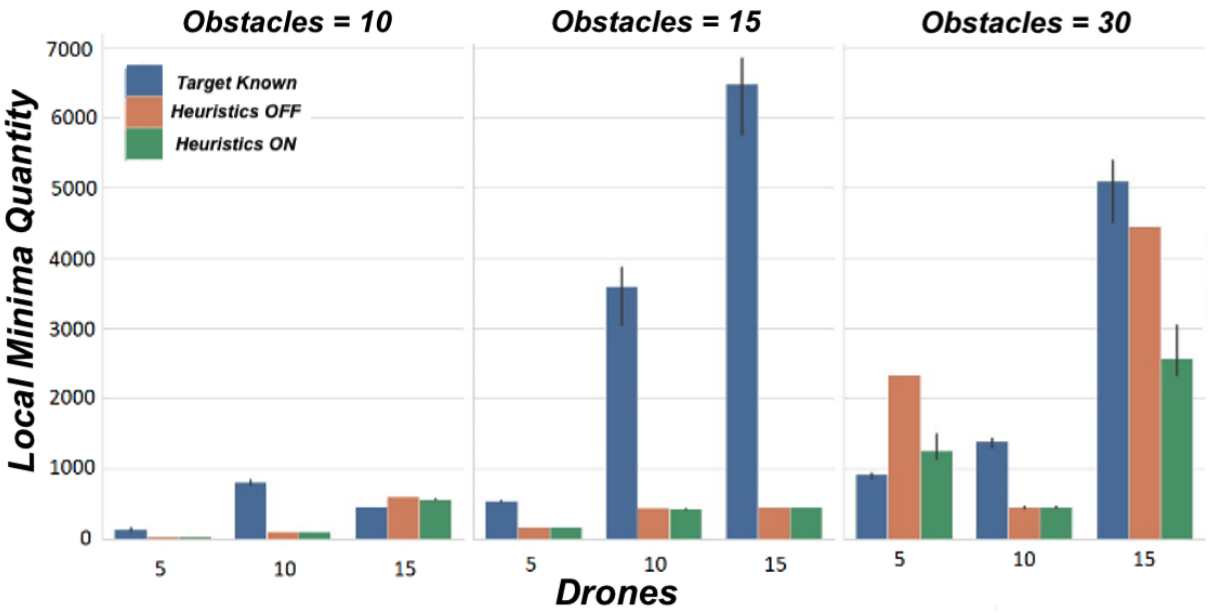

FIGURE 4.17 – Local minima observed in each tested scenarios.

This metric clearly illustrates the effectiveness of our heuristics in reducing the number of blocked UAVs, particularly in scenarios with a substantial presence of local minima. By implementing our blockage prevention heuristics, we observed that as more UAVs were added to the swarm, the likelihood of a neighboring drone assisting a blocked UAV increased. Consequently, fewer drones experienced blockages, as evidenced by the results obtained in the experiments involving 10 and 15 UAVs encountering 30 obstacles. Once we have the heuristic enabled, there is a greater effort required to unlock the drones during iterations, which can lead to an increase in completion time. However, it's important to note that the increase in time occurs because more agents are successfully completing the mission, which is a desirable outcome in practice.

4.5.2 Experiments with the Map of Probabilities

In the upcoming experiments, the evaluation of the new search strategy incorporating the probability map is conducted. The simulator has been enhanced to generate a new probability map during each iteration, which includes updated obstacles and a new location for the missing person. This dynamic probability map enables the drones to adapt their search patterns based on the latest information, enhancing their efficiency. For the experiments, we evaluate the average time taken to complete the mission in 270 iterations and compare it to the results obtained in our previous section, which employed a strategy

based on the lawn mower search and no additional information.

During the experiments, note that the probability maps used for the swarm search are randomly generated. This ensures that the evaluation is conducted in a realistic and unbiased manner. By employing random generation of the probability maps, we simulate a variety of scenarios that reflect different terrain characteristics and target locations. This approach allows us to test the robustness and adaptability of the swarm search and intelligence heuristic in diverse and unpredictable environments. Also, note that the target is always placed in a randomly assigned high probability cell of the map.

4.5.2.1 Results with the Map of Probabilities

In Table 1 we present the results obtained from the experiments. Notably, Experiment 2 demonstrated a shorter mean time of 24.2 seconds compared to Experiment 1. This indicates that the new strategy utilizing the probability map with the addition of the HC and BS algorithms resulted in an approximately 72% improvement in time efficiency for identifying the missing person. These results suggest that the HC and BS algorithms significantly reduced the time required to complete the task, in comparison to the approach adopted in Experiment 1.

TABLE 4.3 – Results of the experiments.

Exp.	Strategy	Mean Time [s]
1	Lawn Mower - Giacomossi *et al.* (2021b)	57.6
2	Probability Map + HC + BS	33.4

It is important to consider that the drones are being simulated in a 2D environment with reduced degrees of freedom, simplified dynamics, and with idealized identification and communication capabilities. So notice that these results can deviate from real world performance. Furthermore, it demonstrates the importance of improving the search method when the goal is to optimize the time efficiency. Note that the Experiment 2 may be more useful as a model for achieving that goal, but the Experiment 1 is still a safe approach when performing a search when there is no information about the region, i.e, no previous map of probabilities is provided.

The results of the comparison between the mission with and without the aid of a probability map provide evidence that using such a map can significantly reduce search time. An interesting observation from the experiments is that when the target is located in a high probability area with most of its cells in the direct path of the drones, as seen in Figure 8, the time taken to locate the target is significantly reduced. This is because the drones can fly almost directly to the target, bypassing areas with low probabilities of detection. However, if the target is not in a high probability area, the algorithm may lead

the drones to wrong areas, resulting in decreased time efficiency. Therefore, while the use of a probability map can improve search time in optimal scenarios, its effectiveness may vary in situations where the map has incorrect assumptions. Thus, the accuracy of the probability map is a critical factor that has a significant impact on the search.

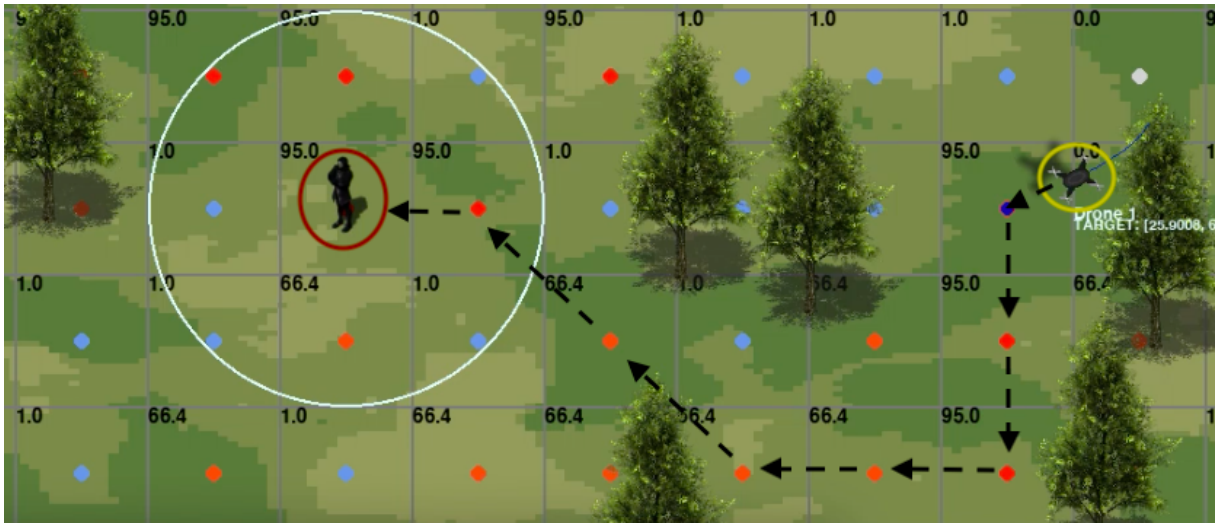

FIGURE 4.18 – Drone performing the search in the 8-connected cells. Red, Yellow and Blue cells represent the high, medium and low probabilities respectively. In Gray, cells that were already visited. The numbers represent the probability of each cell.

In Experiment 2, even though the probability map is generated randomly, it features areas with varying probabilities, with the target consistently positioned in a high-probability region. It's important to acknowledge that this setup introduces a search bias, as the drones primarily focus their search efforts in a comparatively "smaller" area, always gravitating towards regions with high probability. In other words, the fact that the target is consistently placed in the highest probability region shouldn't be interpreted as a realistic scenario. Hence, it's crucial to explore scenarios where the target is located in low probability regions as well to ensure a more comprehensive evaluation. The primary objective of these experiments was to assess the effectiveness of the decision-making techniques in such context. Another perspective is that consistently placing the target in the highest probability region effectively normalizes it to 100%, assuming that the target always resides in those regions, which may not hold true in practice.

To maintain methodological rigor in the context, two alternative approaches can be contemplated to handle the bias in the experimentation. One viable approach involves the stochastic placement of the target across the map while adhering to the prescribed probability distribution. This entails the random selection of a map cell, with subsequent acceptance or rejection of the target's placement based on the probability value associated with the selected cell. This methodology effectively eliminates any inherent bias towards high probability regions, thus ensuring a more equitable evaluation.

An alternative strategy entails the initialization of the probability map with a uniform distribution, specifically 50%, followed by continuous updates as the exploration progresses. This dynamic adaptation of probability values in real-time accurately reflects the evolving nature of the search scenario, contributing to a more realistic and unbiased assessment within the context of the dissertation.

By leveraging the power of AI and probability mapping, search and rescue operations can become faster, more efficient, and ultimately more successful. Time can be translated into reduction of fuel and energy costs, as well as reduced number of UAVs involved to meet the success criteria of the mission. These results can yield large performance increases by choosing the right mission planning algorithms. When performing searches with a map of probabilities and Bayesian updates it is important to make precise assumptions about the probabilities when preparing the map.

5 Conclusion

This dissertation delves into the utilization of decision-making techniques in the context of swarms of unmanned aerial vehicles (UAVs), commonly referred to as drones. The research undertakes an evaluation of two distinct applications: the first involving the so-called loyal wingman drones within defense scenarios, while the second concentrates on a civil application, specifically targeting search and rescue missions aimed at locating missing individuals. For both applications, AI-based decision-making techniques were employed. Additionally, a tailored set of rules and strategies was developed to address the unique challenges presented by each specific problem.

5.1 General Conclusions

The first objective of our work has been successfully completed, during which we delineated two key scenarios: the loyal wingman and search and rescue (SAR) applications. Furthermore, we established a comprehensive framework for the essential capabilities essential for a cooperative air engagement system. This included the definition of engagement rules and techniques for orchestrating collaboration among a team of autonomous agents in both contexts. Moving on to the second objective, we have accomplished the development of an open-source 2D prototype for a decision-making module, which implements advanced distributed autonomous decision-making techniques.

Furthermore, we successfully achieved the objective of collaborating with Embraer and other ITA researchers to develop a high-fidelity simulation system for a swarm of loyal wingmen drones. In this endeavor, we ensured seamless consistency between high-level decision-making processes and low-level operations within the simulation system. This encompassed cooperative path planning, guidance, and the establishment of distributed interfaces between software layers. Within this system, we implemented the necessary behaviors for the agents, strategically employing finite state machine and behavior tree techniques to optimize decision-making. Our primary focus was on implementing this system within simulation environments, integrating it effectively with the participating agents.

Additionally, we successfully achieved the objective of crafting intelligent search strategies tailored for search and rescue missions. Our approach began with the creation of a high-level decision-making simulation featuring simplified dynamics, primarily designed for prototyping in a 2D environment. Within this environment, we devised a strategy that harnessed collective intelligence, utilizing a shared probability map and an autonomous module governed by finite state machines. These strategies were not only successfully developed but also subjected to rigorous analysis through experiments. We meticulously assessed the system's performance via simulations, employing metrics that gauged its effectiveness. These metrics included measures such as completion time and the number of drones encountering obstructions during the mission.

5.2 Loyal Wingman Application

In this study, we have evaluated the application of the Cooperative Engagement Capability (CEC) concept to a Manned-Unmanned team (MUM-T) operating in a gamified defense scenario. The team consists of a manned-controlled UAV serving as the leader, and a group of heterogeneous loyal wingman UAVs with combat capabilities. Our main focus has been to enhance the defense of the leader UAV and protect a critical infrastructure from a swarm of Kamikaze UAVs equipped with explosives. To achieve this, effective cooperation among the team members was essential in order to maximize their mission efficiency. The primary objective of the loyal wingman UAVs is to defend the manned-controlled leader UAV and safeguard the critical infrastructure. To counter the Kamikaze UAV threat, the loyal wingmen are equipped with two types of idealized weapons: a vaporizer gun and a freezing gun. The vaporizer gun serves as a short-range energy weapon capable of neutralizing a threat with a single shot, while the freezing gun acts as a medium-range non-lethal weapon, which can slow down the incoming threats and improve the safety of the approach towards the kamikaze UAVs.

By leveraging the capabilities of the loyal wingman UAVs with AI-based decision-making and promoting collaboration within the MUM-T, we aimed to achieve a higher degree of efficiency in accomplishing the main mission. The loyal wingmen possess the necessary engagement and neutralization capabilities to effectively counter the threats. Their cooperation with the leader UAV enables them to work together towards the common goal, effectively defending the critical infrastructure and ensuring the safety of the manned-controlled leader UAV.

To initiate the project, we commenced by developing a 2D prototype of the loyal wingman scenario, aiming to carry out an initial investigation on decision-making within this specific context. The simulator was entirely constructed using Python and relied on

the Pygame library to provide visual representation. Our main objective in developing this simulator was: firstly, to formulate a decision framework for the agents involved, and secondly, to establish the initial architecture of the solution while enabling fast prototyping. This was achievable owing to the simplified dynamics of the simulator, which facilitated expedited simulations and development processes. Furthermore, for the second stage of this research with the loyal wingman scenario, a more complex 3D simulation of the high fidelity scenario was entirely developed for this project, composed of robust control, dynamics, path planning, and collision avoidance capabilities for the drones.

This study explored the application of decision-making methods in utilizing autonomous drones as Loyal Wingman (LW) Unmanned Aerial Vehicles (UAVs) in a Manned-Unmanned Teaming (MUM-T) context. We have developed intelligence frameworks for both LW agents and kamikaze UAVs, focusing on the implementation of Finite State Machines (FSMs) and Behavior Trees (BTs). Drawing upon these techniques, we have established a decision-making architecture by decomposing the expected behaviors of LW agents and kamikaze drones into a set of behaviors.

For the LW agents, the decision-making process is realized through the adoption of behavior trees. We have categorized the agent's behaviors into two main groups, the first is the movement behaviors: *Chase Threat*, *Go To Formation*, *Approach Formation*, and, the offensive behaviors: *Vaporize Threat*, *Freeze Threat*, and *Sacrifice Attack*. Furthermore, an optimization algorithm has been employed to find optimal parameters for the radius of the formation and engagement range, thereby enhancing LW efficiency in neutralizing the kamikaze threats. In contrast, kamikaze drones employ a simplified yet effective intelligence approach, utilizing a Finite State Machine (FSM) for decision-making. The behaviors implemented for kamikaze drones encompass *Attack Leader State*, *Attack Protected Area State*, *Attack Loyal Wingman State*, and *IdleState*.

In order to evaluate the cooperative capabilities within the Manned-Unmanned Teaming (MUM-T) framework, specifically the collaboration among loyal wingman (LW) agents for threat neutralization, we employed the widely recognized Munkres algorithm. This task allocation method was implemented with the aim of optimizing task distribution among LW agents, thereby enhancing overall coordination and cohesion within the team. Additionally, we proposed the adoption of a uniform-circular formation for the MUM-T, which although simple, proved to be remarkably effective in practice.

The analysis of the 2D prototype yielded encouraging results, demonstrating the efficacy of the developed decision-making framework for the LW agents. Following optimization utilizing the Particle Swarm Optimization (PSO) algorithm, approximately 93% of the threats were successfully neutralized. Additionally, the significance of the formation configuration, wherein each drone possesses distinct defense specifications, was observed. Notably, the utilization of decentralized decision-making and straightforward engagement

rules led to the emergence of cooperative teamwork in neutralizing threats. Notably, LW drones exhibited specialized defense behaviors, such as a particular LW assuming the role of defending the leader, highlighting the adaptive nature of the system.

Regarding the 3D simulation, to reduce the complexity of this problem, all the vehicles are modeled as fully actuated UAVs, and the problem is broken down into two layers, one involving the Low-Level (LL) control layer, and another involving an High-Level (HL) layer of intelligent software capable of making autonomous and decentralized decisions. This dissertation focused only on the High-Level aspects of the problem. The LL control layer designs a joint attitude-position control law using a first-order sliding mode control that guarantees the closed-loop system robustness with respect to model uncertainties and disturbances. On the other hand, The HL was composed of the decision-making modules for loyal wingman agents, following the solution developed using the behavior tree technique.

In future research, it is recommended to enhance the complexity of kamikazes' behaviors to improve the difficulty of the proposed scenario, as well as alternative weaponry for them. Additionally, it would be beneficial to define more intricate scenarios that involve various types of threats and alternative drone models. It is essential for further studies to explore optimization and specialization of formations based on different types of attacks and defense maneuvers. Consensus algorithms should be investigated to improve the utilization of LW and delegate responsibilities in alternative manners. Furthermore, it is crucial to conduct and analyze future work on evolving more complex behaviors, potentially utilizing reinforcement learning for both types of agents, to learn specific subbehaviors and new strategies. It is also advisable to perform tests incorporating real drones for a more comprehensive evaluation. Future research could delve into the impact of formations on these specializations to gain a deeper understanding of their usage in real warfare scenarios. Finally, we recommended to apply reinforcement learning to the scenario and utilize the LW's decision-making process developed in this project as the adversarial team to enable the agents to learn new behaviors and complex strategies to handle and compare with the intelligence developed by a human.

Future studies could also focus on conflict resolution in airspace, which is also crucial when it comes to unmanned drones. As the use of drones continues to grow, the need to ensure the safety and seamless integration of these devices into traditional airspace becomes even more critical. To avoid conflicts between drones and manned aircraft, advanced technologies along with specific regulations for operating drones at designated altitudes and areas. Additionally, communication between drone operators and air traffic controllers plays an essential role in preventing hazardous situations, specially when dealing with military applications such as the loyal wingman, ensuring that drones operate safely and efficiently in shared airspace.

5.3 Search and Rescue Application

As the civil application in this dissertation, we conducted an investigation into the utilization of UAV swarms for target search in regions with obstacles, particularly when dealing with the challenge of locating an individual of unknown whereabouts. To facilitate our study, we developed a UAV swarm simulator that allowed us to evaluate various aspects of this research. Specifically, our focus was on addressing two primary concerns: the decision-making process of the drones within the scenario and the coordination required among the group to successfully carry out the search for the missing person.

We have developed intelligent search strategies for the UAVs by incorporating autonomous and collective decision-making capabilities to the agents. To achieve individual autonomous intelligence, we employed a Finite State Machine (FSM) that effectively controls the behaviors of each drone in a decentralized manner. Through this, the UAVs were able to operate autonomously, making independent decisions based on their surroundings.

In addition to individual autonomy, we also focused on achieving collective intelligence among the UAVs. To enable this, we designed a discrete map of the search region and established a shared network among the drones. By utilizing this common network, the UAVs functioned as a cohesive team, exchanging information crucial for search coordination and distributing the search task effectively. In this map, the drones updated the state of the cells that they were visiting. To address the issue of UAVs getting stuck in local minima, a common problem when using the potential field technique for collision avoidance, we developed a blockage prevention heuristic. This heuristic was implemented to minimize the occurrence of UAVs being blocked in local minima, allowing for smoother navigation and enhanced search efficiency.

By combining individual autonomous intelligence, collective decision-making, and the blockage prevention heuristic, our research significantly improves the overall search capabilities of UAVs. This cohesive approach ensures efficient search coordination, task distribution, and effective collision avoidance, ultimately enhancing the overall performance and effectiveness of the UAV system.

Based on our evaluation, where we varied the number of obstacles and UAVs in the swarm, we have found that our swarm intelligence approach is highly effective. Notably, it has successfully reduced the occurrence of UAVs getting stuck in local minima, which is a common problem encountered when using the potential fields technique. This finding indicates that our approach holds promise as a solution to this frequent issue. Furthermore, our results demonstrated that the mission completion times achieved by our swarm intelligence approach were comparable to the reference time set by the level 1 strategy. This outcome is particularly encouraging, as it suggests that our approach can achieve mission completion within a reasonable timeframe. It is important to note, however, that

when employing heuristics for search, there is no guarantee of achieving optimal time in finding the individual.

Moreover, building on the foundations of the decision making developed in this first stage of experiments, a new and more complex search strategy was developed. In this particular step, the discretized map of the region now incorporates probabilities assigned to each cell. These probabilities serve as indicators of the likelihood of an individual being situated within a specific cell. It is essential to note that this probability map is prepared in advance for the purpose of the search and should be formulated by experts in the field.

In terms of our high-level strategy and tactics for searching the area of interest, we have developed a methodology that incorporates several effective search techniques. Our approach combines the Lawn Mower search, the Hill Climbing search, Bayesian search, and potential fields. These search patterns were integrated with simulation techniques to create a search framework, and adapted to the simulator we had previously developed for evaluating the concept. The experiments in this stage reduced by approximately 72% the time taken in searching for the missing person using the strategies developed in comparison with our previous strategy, which employed a strategy with no previous information of the probabilities of where the missing person might be located. The findings highlight the importance of continually exploring new approaches and technologies in order to optimize search and rescue missions and increase the chances of saving lives. Also, we noticed the importance of making precise assumptions when preparing the probability map, otherwise it may lead to incorrect and not effective searches.

Consequently, our findings highlight the importance of evaluating and exploring various forms of collective swarm intelligence to further enhance target search missions. The utilization and refinement of such approaches hold significant potential for delivering notable advantages in similar contexts. By harnessing the power of collective swarm intelligence, we can improve the efficiency and effectiveness of target search missions, paving the way for enhanced outcomes in real-world scenarios.

Future research should consider applying optimization algorithms to the decision-making parameters to improve the obtained results. Another suggestion is the evaluation of additional effectiveness metrics, such as redundant paths and revisited cells, that UAVs may occasionally revisit. Alternative ideas to avoid getting stuck at local minima can also be further evolved. Furthermore, we intend to study the performance of the algorithms in a 3D environment with robust physics and realistic drone dynamics, as seen in Figure 5.1 and Figure 5.2 and in video [1]. Also, we intend in the future is to apply this technique in real drones. Further experimentation with different kinds of maps to evaluate the algorithms and tweaking of the probability update function can be also studied.

[1] https://drive.google.com/file/d/1j9C8yJjPHHJ7Q3XsSqkrSfmtfhKhTc-e/view?usp=sharing

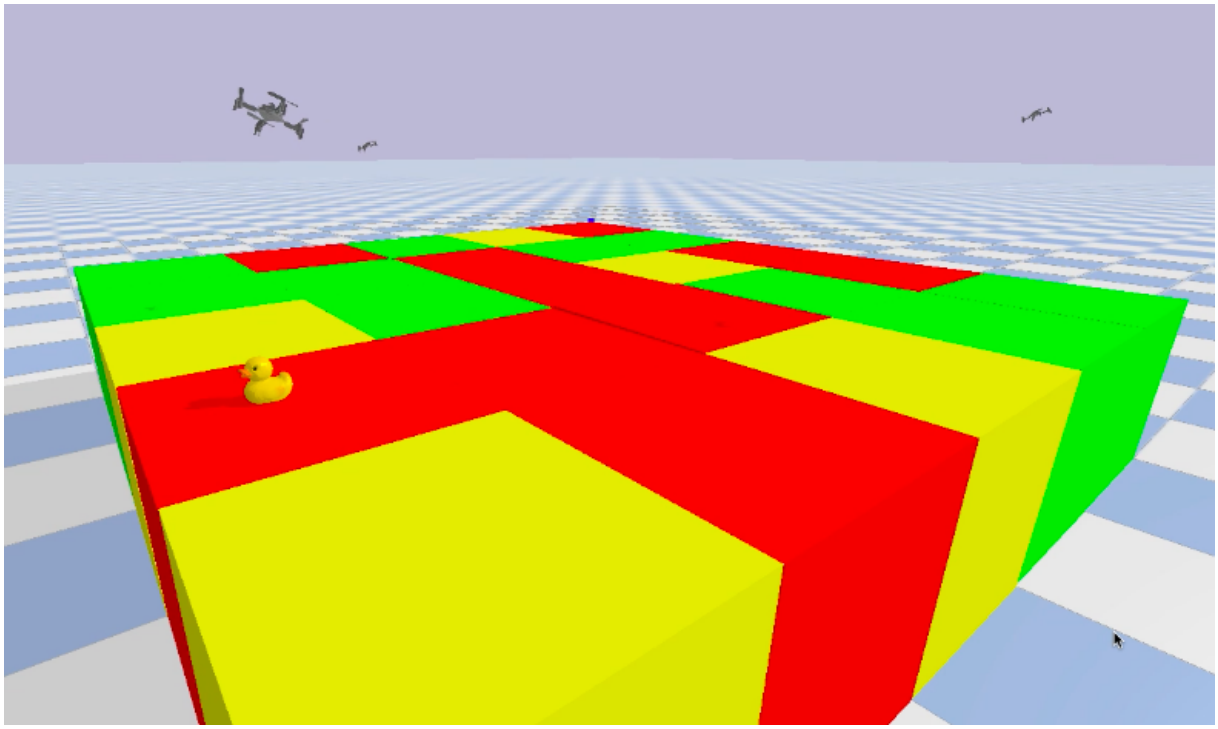

FIGURE 5.1 – Search and rescue scenario adapted and performed in the py-bullet 3D environment. The cells' colors represent the probability of a subregion of the map. Green refers to low probability cells, Yellow to medium probability, and Red cells to high probability cells. The duck represents the missing individual.

5.4 Similarities Between Contexts

Analyzing the two studied contexts, we can identify several similarities. It is evident that both applications developed autonomous decision-making modules for drones using state machines, and these state machines exhibited modularized behaviors to address identified subtasks within each context. Additionally, individual intelligence modules, specific to each agent, were designed, and group-level strategies for the swarm of drones were formulated. In the first context, a study on defense and attack formations was conducted, involving task allocation and centralizing this allocation within a leading drone. In the search and rescue context, this centralization was aimed at fostering cooperation in creating a shared map for the swarm. In both contexts, communication among the drones was essential for cooperative decision-making, emphasizing the necessity of efficient strategies for communication aiming at an effective decision-making.

In the case of search and rescue, communication was aimed at sharing information to form an overview of the search area through discretized mapping. Information exchange involved sharing data about cell visits, collaborative probability updates, and coordinate sharing when the target was located. On the other hand, communication within the loyal wingman swarm primarily focused on leader defense and safeguarding the protected area. This involved exchanging formation coordinates for protection, requests for entry and

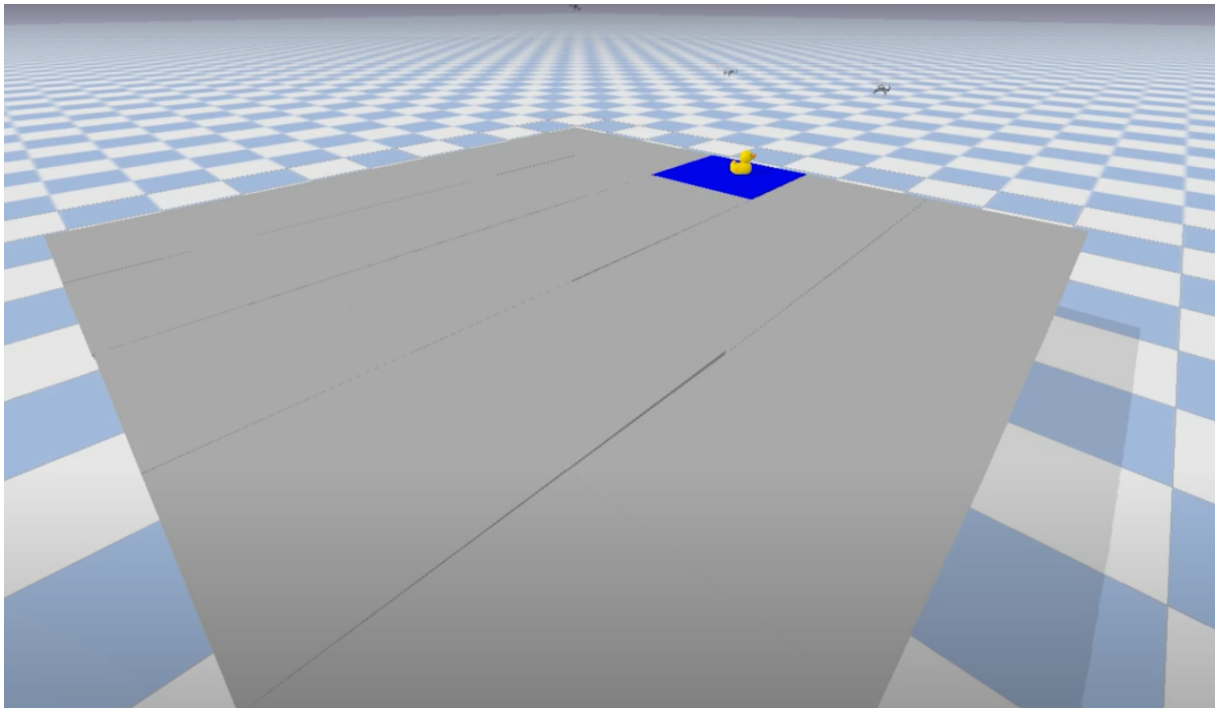

FIGURE 5.2 – End of the search, with the individual being found and all drones near their location. The gray cells represent the discarded areas where the search was conducted and the individual was not identified. Individual represented by the rubber duck on the blue cell.

exit from the formation, and requests for specific threat attacks. It is evident, therefore, that communication plays an incredibly vital role within the context of intelligent drone swarms, and the effectiveness of decision-making strategies relies heavily on this factor and its efficiency.

Both cases were initially studied using the framework of tools developed within the 2D simulators, where, in both contexts, algorithms for potential fields for collision avoidance and state machines for autonomous decision-making were primarily shared. In the case of the loyal wingman application, there was a need to scale the collision avoidance due to the high fidelity of the simulation, leading to the application of the AOV algorithm. In contrast, for cooperative behavior, the Munkres algorithm was employed for optimal task allocation in threat neutralization, although it was not used in the search and rescue context. Additionally, a more scalable decision-making technique, the behavior trees, was required due to the increased complexity of the loyal wingman agent compared to the search drones. Furthermore, as the fidelity of simulations increased in the 3D case, the drone's flight control system was modified to implement controllers using sliding mode control (SMC), a scenario that was not explored in the search and rescue application.

In the search scenario, Bayesian search theory and hill climbing (HC) search were employed to update and conduct the search on the probability map, enhancing the efficiency and time of locating the missing person. This was a requirement that did not apply to the loyal wingman drones. However, it could potentially be adapted if the assumption

of idealized situational awareness were removed. In such a scenario, the loyal wingman drones would also need to search for kamikaze threats, making both contexts even more similar.

Hence, it is evident that there are several similarities in the application contexts of drone swarms, even when one is military and the other is civilian, underscoring the need for the continuous advancement of these techniques.

Bibliography

AKIYAMA, H.; ARAMAKI, S.; NAKASHIMA, T. Online cooperative behavior planning using a tree search method in the robocup soccer simulation. *In*: **2012 Fourth International Conference on Intelligent Networking and Collaborative Systems**. **Proceedings** [...]. [*S.l.: s.n.*], 2012. p. 170–177.

ALSAMHI, S. H.; MA, O.; ANSARI, M. S. Convergence of machine learning and robotics communication in collaborative assembly: mobility, connectivity and future perspectives. **Journal of Intelligent & Robotic Systems**, Springer, v. 98, p. 541–566, 2020.

ALTSHULER, Y.; YANOVSKY, V.; WAGNER, I. A.; BRUCKSTEIN, A. M. Efficient cooperative search of smart targets using uav swarms. **Robotica**, Cambridge University Press, v. 26, n. 4, p. 551–557, 2008.

APL-TEAM. The cooperative engagement capability. **JOHNS HOPKINS APL TECHNICAL DIGEST**, v. 16, n. 4, 1995. ISSN 0019-0578.

ARKIN, R. C. **Behavior–Based Robotics**. Cambridge, Massachusetts, USA: The MIT Press, 1998.

ARKIN, R. C.; ARKIN, R. C. *et al.* **Behavior-based robotics**. [*S.l.*]: MIT press, 1998.

ARORA, R. K. **Optimization: algorithms and applications**. [*S.l.*]: Chapman and Hall/CRC, 2019.

AUNON P. CRUZ, A. **Comparison of Heuristic Algorithms in Discrete Search and Surveillance Tasks Using Aerial Swarms.** [*S.l.*]: Swarm Robotics: MDPI, 2018. 130 - 160 p.

BALDAO, T. P.; MAXIMO, M. R. O. A.; CESAR, C. de A. C. Decision-making for 5x5 very small size soccer teams. *In*: **2020 Latin American Robotics Symposium (LARS), 2020 Brazilian Symposium on Robotics (SBR) and 2020 Workshop on Robotics in Education (WRE)**. **Proceedings** [...]. [*S.l.: s.n.*], 2020. p. 1–6.

BALDWIN, E.; SOH, H.; ZHOU, X.; CHUAH, C.-N. Heterogeneous uav swarm control using deep reinforcement learning. *In*: IEEE. **2018 15th International Symposium on Wireless Communication Systems (ISWCS)**. **Proceedings** [...]. [*S.l.: s.n.*], 2018. p. 1–6.

BARNES, L. **A Potential Field Based Formation Control Methodology for Robot Swarms**. [*S.l.*]: University of South Florida, 2018.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software architecture in practice**. [*S.l.*]: Addison-Wesley Professional, 2003.

BENI, G.; WANG, J. Swarm intelligence in robotics. **Robotics and Automation Magazine, IEEE**, IEEE, v. 11, n. 3, p. 71–80, 2004.

BERG, J. V. D.; SNAPE, J.; GUY, S. J.; MANOCHA, D. Reciprocal collision avoidance with acceleration-velocity obstacles. **Proceedings - IEEE International Conference on Robotics and Automation**, p. 3475–3482, 2011. ISSN 10504729.

BERG, J. van den; SNAPE, J.; GUY, S. J.; MANOCHA, D. Reciprocal collision avoidance with acceleration-velocity obstacles. *In*: **2011 IEEE International Conference on Robotics and Automation**. **Proceedings** [...]. [*S.l.: s.n.*], 2011. p. 3475–3482.

BERLINGER, F.; GAUCI, M.; NAGPAL, R. Implicit coordination for 3d underwater collective behaviors in a fish-inspired robot swarm. **Science Robotics**, American Association for the Advancement of Science, v. 6, n. 50, p. eabd8668, 2021.

BEZERRA, J. A.; SANTOS, D. A. Optimal exact control allocation for under-actuated multirotor aerial vehicles. **IEEE Control Systems Letters**, v. 6, p. 1448–1453, 2022.

BISIO, I.; GARIBOTTO, C.; HALEEM, H.; LAVAGETTO, F.; SCIARRONE, A. A systematic review of drone based road traffic monitoring system. **IEEE Access**, v. 10, p. 101537–101555, 2022.

BOEING. **MQ-28**. Accessed: 2023. `https://www.boeing.com/defense/MQ-28/`.

BOURGEOIS, F.; LASSALLE, J.-C. An extension of the munkres algorithm for the assignment problem to rectangular matrices. **Communications of the ACM**, ACM New York, NY, USA, v. 14, n. 12, p. 802–804, 1971.

BOX, G. E.; TIAO, G. C. **Bayesian inference in statistical analysis**. [*S.l.*]: John Wiley & Sons, 2011.

BOYLE, M. J. The legal and ethical implications of drone warfare. *In*: **Legal and Ethical Implications of Drone Warfare**. [*S.l.*]: Routledge, 2018. p. 1–22.

BRAMBILLA, M.; FERRANTE, E.; BIRATTARI, M.; DORIGO, M. Swarm robotics: a review from the swarm engineering perspective. **Swarm Intelligence**, Springer, v. 7, p. 1–41, 2013.

Brescianini, D.; D'Andrea, R. Design, modeling and control of an omni-directional aerial vehicle. **IEEE International Conference on Robotics and Automation (ICRA)**, p. 3261–3266, 2016.

BURKARD, R. E.; ÇELA, E. Linear assignment problems and extensions. *In*: _____. **Handbook of Combinatorial Optimization: Supplement Volume A**. Boston, MA: Springer US, 1999. p. 75–149. ISBN 978-1-4757-3023-4. Available at: https://doi.org/10.1007/978-1-4757-3023-4_2.

CAMPBELL, D. T. The two distinct routes beyond kin selection to ultrasociality: Implications for the humanities and social sciences. **The nature of prosocial development: Theories and strategies**, p. 11–41, 1983.

CAMPION, M.; RANGANATHAN, P.; FARUQUE, S. Uav swarm communication and control architectures: a review. **Journal of Unmanned Vehicle Systems**, NRC Research Press, v. 7, n. 2, p. 93–106, 2018.

CAMPION, M.; RANGANATHAN, P.; FARUQUE, S. Uav swarm communication and control architectures: a review. **Journal of Unmanned Vehicle Systems**, v. 7, n. 2, p. 93–106, 2019. Available at: https://doi.org/10.1139/juvs-2018-0009.

CAO, Y. U.; FUKUNAGA, A. S.; KAHNG, A. B.; MENG, F. Cooperative mobile robotics: Antecedents and directions. *In*: IEEE. **Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots**. **Proceedings** [...]. [*S.l.: s.n.*], 1995. v. 1, p. 226–234.

CAUDLE, K. Searching algorithm using bayesian updates. **Journal of Computers in Mathematics and Science Teaching**, Association for the Advancement of Computing in Education (AACE), v. 29, n. 1, p. 19–29, 2010.

CHALUP, S.; MAIRE, F. A study on hill climbing algorithms for neural network training. *In*: IEEE. **Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406)**. **Proceedings** [...]. [*S.l.: s.n.*], 1999. v. 3, p. 2014–2021.

CHANG, H. J.; LEE, C. S. G.; LU, Y.-H.; HU, Y. C. P-slam: Simultaneous localization and mapping with environmental-structure prediction. **IEEE Transactions on Robotics**, v. 23, n. 2, p. 281–293, 2007.

CHEN, P.; ZHOU, X.; ZHAO, J.; SHEN, F.; SUN, S. Energy-efficient resource allocation for secure d2d communications underlaying uav-enabled networks. **IEEE Transactions on Vehicular Technology**, IEEE, v. 71, n. 7, p. 7519–7531, 2022.

CHEN, W.; LIU, J.; GUO, H.; KATO, N. Toward robust and intelligent drone swarm: Challenges and future directions. **IEEE Network**, IEEE, v. 34, n. 4, p. 278–283, 2020.

CHOSET, H.; LYNCH, K.; HUTCHINSON, S.; KANTOR, G.; BURGARD, W. **Principles of Robot Motion: Theory, Algorithms, and Implementations**. MIT Press, 2005. (Intelligent Robotics and Autonomous Agents series). ISBN 9780262033275. Available at: https://books.google.com.br/books?id=S3biKR21i-QC.

CHUNG, T. H.; BURDICK, J. W. Analysis of search decision making using probabilistic search strategies. **IEEE Transactions on Robotics**, v. 28, n. 1, p. 132–144, 2012.

COLLEDANCHISE, M.; OGREN, P. **Behavior Trees in Robotics and AI: An Introduction**. [*S.l.*]: Chapman & Hall/CRC Press, 2018. ISBN 9781138593732.

CRUZ, J.; CHEN, G.; LI, D.; WANG, X. Particle swarm optimization for resource allocation in uav cooperative control. *In*: **AIAA Guidance, Navigation, and Control Conference and Exhibit**. **Proceedings** [...]. [*S.l.: s.n.*], 2012.

DANTAS, A.; DINIZ, L.; ALMEIDA, M.; OLSSON, E.; FUNK, P.; SOHLBERG, R.; RAMOS, A. Intelligent system for detection and identification of ground anomalies for rescue. *In*: SPRINGER. **ITNG 2022 19th International Conference on Information Technology-New Generations**. **Proceedings** [...]. [*S.l.: s.n.*], 2022. p. 277–282.

DINI, P.; SAPONARA, S. Processor-in-the-loop validation of a gradient descent-based model predictive control for assisted driving and obstacles avoidance applications. **IEEE Access**, v. 10, p. 67958–67975, 2022.

DOUTHWAITE, J. A.; ZHAO, S.; MIHAYLOVA, L. S. A comparative study of velocity obstacle approaches for multi-agent systems. *In*: IEEE. **2018 UKACC 12th International Conference on Control (CONTROL)**. **Proceedings** [...]. [*S.l.: s.n.*], 2018. p. 289–294.

DRAKE, Y.; SURESH, G. **An introduction to Python and computer programming**. [*S.l.*]: World Scientific, 2016.

ENDSLEY, M. R.; CONNORS, E. S. Situation awareness: State of the art. *In*: **2008 IEEE Power and Energy Society General Meeting - Conversion and Delivery of Electrical Energy in the 21st Century**. **Proceedings** [...]. [*S.l.: s.n.*], 2008. p. 1–4.

EXCELLENCE, U. A. M. C. of. **Ideas to Action: Airborne "Loyal Wingman" autonomous drone technology**. 2021. Available at: https://www.army.mil/article-/242920/ideas_to_action_airborne_loyal_wingman_autonomous_drone_technology.

FINN, R. L.; WRIGHT, D. Unmanned aircraft systems: Surveillance, ethics and privacy in civil applications. **Computer Law Security Review**, v. 28, n. 2, p. 184–194, 2012. ISSN 0267-3649. Available at: https://www.sciencedirect.com/science/article/pii/S0267364912000234.

FIORINI, P.; SHILLER, Z. Motion planning in dynamic environments using velocity obstacles. **The International Journal of Robotics Research**, v. 17, n. 7, p. 760–772, 1998. Available at: https://doi.org/10.1177/027836499801700706.

FORCIER, J.; BISSEX, P.; CHUN, W. J. **Python web development with Django**. [*S.l.*]: "O'Reilly Media, Inc.", 2009.

FU, L.; XIE, F.; WANG, D.; MENG, G. The overview for uav air-combat decision method. *In*: **The 26th Chinese Control and Decision Conference (2014 CCDC)**. **Proceedings** [...]. [*S.l.: s.n.*], 2014. p. 3380–3384.

FUNKE, J.; BROWN, M.; ERLIEN, S. M.; GERDES, J. C. Collision avoidance and stabilization for autonomous vehicles in emergency scenarios. **IEEE Transactions on Control Systems Technology**, IEEE, v. 25, n. 4, p. 1204–1216, 2016.

GARCIA-AUNON, P.; CERRO, J. del; BARRIENTOS, A. Behavior-based control for an aerial robotic swarm in surveillance missions. **Sensors**, v. 19, n. 20, 2019. ISSN 1424-8220. Available at: https://www.mdpi.com/1424-8220/19/20/4584.

GARCIA-AUNON, P.; CRUZ, A. B. Comparison of heuristic algorithms in discrete search and surveillance tasks using aerial swarms. **Applied Sciences**, v. 8, n. 5, 2018. ISSN 2076-3417. Available at: https://www.mdpi.com/2076-3417/8/5/711.

GARULLI, A.; GIANNITRAPANI, A.; ROSSI, A.; VICINO, A. Mobile robot slam for line-based environment representation. *In*: **Proceedings of the 44th IEEE Conference on Decision and Control**. **Proceedings** [...]. [*S.l.: s.n.*], 2005. p. 2041–2046.

GASPARETTO, A.; BOSCARIOL, P.; LANZUTTI, A.; VIDONI, R. Path planning and trajectory planning algorithms: A general overview. *In*: _____. **Motion and Operation Planning of Robotic Systems: Background and Practical Approaches**. Cham: Springer International Publishing, 2015. p. 3–27. ISBN 978-3-319-14705-5. Available at: https://doi.org/10.1007/978-3-319-14705-5_1.

GERKEY, B. P.; MATARIC, M. J. A formal analysis and taxonomy of task allocation in multi-robot systems. *In*: IEEE. **Proceedings 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)(IEEE Cat. No. 04CH37566)**. **Proceedings** [...]. [*S.l.: s.n.*], 2004. v. 3, p. 1515–1520.

GIACOMOSSI, L.; CAREGNATO-NETO, A.; MAXIMO, M. R. O. A. Optimization of force and torque bounds for the flight control system of a quadcopter using pso. *In*: **2022 Latin American Robotics Symposium (LARS), 2022 Brazilian Symposium on Robotics (SBR), and 2022 Workshop on Robotics in Education (WRE)**. **Proceedings** [...]. [*S.l.: s.n.*], 2022a. p. 1–6.

GIACOMOSSI, L.; MAXIMO, M. R.; SUNDELIUS, N.; FUNK, P.; BRANCALION, J. F.; SOHLBERG, R. Cooperative search and rescue with drone swarm. 2023.

GIACOMOSSI, L.; RICARDO, J. A.; BRANCALION, J. F.; MAXIMO, M. R.; SANTOS, D. A. Cooperative threat engagement with heterogeneous drone swarms. **Congress of the International Council of the Aeronautical Sciences**, 2022b.

GIACOMOSSI, L.; Schwanz Dias, S.; BRANCALION, J.; MAXIMO, M. Cooperative and decentralized decision-making for loyal wingman UAVs. *In*: IEEE. **IEEE Latin American Robotics Symposium (LARS)**. **Proceedings** [...]. [*S.l.: s.n.*], 2021. p. 78–83.

GIACOMOSSI, L.; SOUZA, F.; CORTES, R. G.; CORTEZ, H. M. M.; FERREIRA, C.; MARCONDES, C. A. C.; LOUBACH, D. S.; SBRUZZI, E. F.; VERRI, F. A. N.; MARQUES, J. C.; PEREIRA, L. A.; MAXIMO, M. R. O. A.; CURTIS, V. V. Autonomous and collective intelligence for uav swarm in target search scenario. *In*: IEEE. **2021 Latin American Robotics Symposium (LARS)**. **Proceedings** [...]. [*S.l.: s.n.*], 2021. p. 72–77.

GORELICK, J.; OZSVALD, I. **Data science at the command line: Facing the future with time-tested tools**. [*S.l.*]: O'Reilly Media, 2014.

GOVERS, F. X. **Artificial intelligence for robotics: Build intelligent robots that perform human tasks using AI techniques**. [*S.l.*]: Packt Publishing Ltd, 2018.

GROCHOLSKY, B.; CHELLAPPA, R.; VIDAL, R.; SWAMI, A.; HAGER, G. D. Cooperative air and ground surveillance. *In*: IEEE. **2006 IEEE International Conference on Robotics and Automation**. **Proceedings** [...]. [*S.l.: s.n.*], 2006. p. 2778–2785.

HANSEN, N. The CMA evolution strategy: A tutorial. **arXiv preprint arXiv:1604.00772**, 2016.

HANSEN, N.; MÜLLER, S. D.; KOUMOUTSAKOS, P. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es). **Evolutionary computation**, MIT Press, v. 11, n. 1, p. 1–18, 2003.

HARALDSEN, A. **Collision Avoidance using the Velocity Obstacle Algorithm for Underactuated Surface Vehicles**. Dissertation (Mestrado) — NTNU, 2020.

HARRISON, R. L. Introduction to monte carlo simulation. **AIP Conf Proc**, v. 1204, p. 17–21, 2010.

HERNÁNDEZ, D.; CECÍLIA, J. M.; CALAFATE, C. T.; CANO, J.-C.; MANZONI, P. The kuhn-munkres algorithm for efficient vertical takeoff of uav swarms. *In*: IEEE. **2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)**. **Proceedings** [...]. [*S.l.: s.n.*], 2021. p. 1–5.

HEYLIGHEN, F. Evolution, selfishness and cooperation. **Evolution**, Citeseer, v. 77, p. 84, 2009.

HONG, S.-E.; KIM, H. J. Quadrotor helicopter attitude control using dual quaternion. *In*: IEEE. **2012 International Conference on Control, Automation and Systems**. **Proceedings** [...]. [*S.l.: s.n.*], 2012. p. 1023–1028.

HOWARD, M. Autonomous loyal wingman drones to become weapons in australia's military arsenal. **ABC News**, 2020. Available at: https://www.abc.net.au/news/2020-02-28/australian-air-force-to-use-autonomous-loyal-wingman-drones/11998398.

HUANG, H.-M.; MESSINA, E. **Autonomy Levels for Unmanned Systems (ALFUS) FrameworkVolume II: Framework Models Initial Version**. [*S.l.*]: Special Publication (NIST SP), National Institute of Standards and Technology, Gaithersburg, MD, 2007.

HUMPHREYS, C.; COBB, R.; JACQUES, D.; REEGER, J. Optimal mission path for the uninhabited loyal wingman. *In*: **16th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference**. **Proceedings** [...]. [*S.l.: s.n.*], 2015. p. 2792.

HUSSAIN, R.; ZEADALLY, S. Autonomous cars: Research results, issues, and future challenges. **IEEE Communications Surveys Tutorials**, v. 21, n. 2, p. 1275–1313, 2019.

HUTCHINGS, A.; KOLLING, A.; HAMMARSTRAND, L. Optimization of uav swarm configuration for urban search and rescue. **Journal of Field Robotics**, Wiley Online Library, v. 36, n. 1, p. 233–251, 2019.

HWANG, Y. K.; AHUJA, N. *et al.* A potential field approach to path planning. **IEEE Transactions on Robotics and Automation**, Citeseer, v. 8, n. 1, p. 23–32, 1992.

IOVINO, M.; SCUKINS, E.; STYRUD, J.; ÖGREN, P.; SMITH, C. A survey of behavior trees in robotics and ai. **Robotics and Autonomous Systems**, Elsevier, p. 104096, 2022.

IOVINO, M.; SCUKINS, E.; STYRUD, J.; ÖGREN, P.; SMITH, C. A survey of behavior trees in robotics and ai. **Robotics and Autonomous Systems**, Elsevier, v. 154, p. 104096, 2022.

JAIN, R. **The Art of Computer System Performance Analysis: Techniques for Experimental Design, Measurement, Simulation and Modeling**. [*S.l.*]: John Wiley & Sons, 1991. 130 - 160 p.

JOYCE, J. Bayes' Theorem. *In*: ZALTA, E. N. (Ed.). **The Stanford Encyclopedia of Philosophy**. Fall 2021. [*S.l.*]: Metaphysics Research Lab, Stanford University, 2021.

KARACA, Y.; CICEK, M.; TATLI, O.; SAHIN, A.; PASLI, S.; BESER, M. F.; TUREDI, S. The potential use of unmanned aircraft systems (drones) in mountain search and rescue operations. **The American journal of emergency medicine**, Elsevier, v. 36, n. 4, p. 583–588, 2018.

KENNEDY, J. The particle swarm: social adaptation of knowledge. *In*: **Proceedings of 1997 IEEE International Conference on Evolutionary Computation (ICEC '97)**. **Proceedings** [...]. [*S.l.: s.n.*], 1997. p. 303–308.

KENNEDY, J.; EBERHART, R. Particle swarm optimization. *In*: **Proceedings of ICNN'95 - International Conference on Neural Networks**. **Proceedings** [...]. [*S.l.: s.n.*], 1995. v. 4, p. 1942–1948 vol.4.

KOOPMAN, B. **Search and Screening: General Principles with Historical Applications**. Military Operations Research Society, 1999. (MORS heritage series). ISBN 9780930473082. Available at: https://books.google.com.br/books?id=8kFHtwAACAAJ.

KORF, R. E. **Artificial intelligence search algorithms**. [*S.l.*]: Citeseer, 1999.

KÖRNER, F.; SPECK, R.; GÖKTOGAN, A. H.; SUKKARIEH, S. Autonomous airborne wildlife tracking using radio signal strength. *In*: IEEE. **2010 IEEE/RSJ International Conference on Intelligent Robots and Systems**. **Proceedings** [...]. [*S.l.: s.n.*], 2010. p. 107–112.

KUHN, H. W. The hungarian method for the assignment problem. **Naval research logistics quarterly**, Wiley Online Library, v. 2, n. 1-2, p. 83–97, 1955.

KUMAR, V.; MICHAEL, N. Opportunities and challenges with autonomous micro aerial vehicles. **The International Journal of Robotics Research**, SAGE Publications Sage UK: London, England, v. 31, n. 11, p. 1279–1291, 2012.

LAVALLE, S. M. Randomized path planning for robot navigation. *In*: IEEE. **Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164)**. **Proceedings** [...]. [*S.l.: s.n.*], 2001. v. 1, p. 468–475.

LIN, S.; KERNIGHAN, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations research**, Informs, v. 21, n. 2, p. 498–516, 1973.

LIU, S.; QU, C.; XU, M.; WAN, J.; SHENG, H.; ZENG, Z.; CUI, J. An improved kuhn munkres algorithm for ship matching in optical satellite images. **IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing**, IEEE, 2023.

LUTZ, M. **Learning Python**. [*S.l.*]: O'Reilly Media, Inc., 2013.

MAC, T. T.; COPOT, C.; DUC, T. T.; KEYSER, R. D. Ar.drone uav control parameters tuning based on particle swarm optimization algorithm. *In*: **2016 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR)**. **Proceedings** [...]. [*S.l.: s.n.*], 2016. p. 1–6.

MACALPINE, P.; STONE, P. Overlapping layered learning. **Artificial Intelligence**, Elsevier, v. 254, p. 21–43, January 2018. ISSN 0004-3702. Available at: https://www.sciencedirect.com/science/article/pii/S0004370217301066.

MACENSKI, S.; FOOTE, T.; GERKEY, B.; LALANCETTE, C.; WOODALL, W. Robot operating system 2: Design, architecture, and uses in the wild. **Science Robotics**, v. 7, n. 66, p. eabm6074, 2022. Available at: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

MAHADEVAN, S.; CONNELL, J. Automatic programming of behavior-based robots using reinforcement learning. **Artificial Intelligence**, v. 55, n. 2, p. 311–365, 1992. ISSN 0004-3702. Available at: https://www.sciencedirect.com/science/article/pii/0004370292900586.

MARCOTTE, R.; HAMILTON, H. J. Behavior trees for modelling artificial intelligence in games: A tutorial. **The Computer Games Journal**, Springer, v. 6, n. 3, p. 171–184, 2017.

MARZINOTTO, A.; COLLEDANCHISE, M.; SMITH, C.; ÖGREN, P. Towards a unified behavior trees framework for robot control. *In*: **2014 IEEE International Conference on Robotics and Automation (ICRA)**. **Proceedings** [...]. [*S.l.: s.n.*], 2014. p. 5420–5427.

MCCANNE, S. **A distributed whiteboard for network conferencing**. [*S.l.*]: Citeseer, 1992.

MCKINNEY, W. **Python for Data Analysis**. [*S.l.*]: O'Reilly Media, 2018.

MEDEIROS, T. F. de; MAXIMO, M. R. O. A.; YONEYAMA, T. 2020 latin american robotics symposium (lars), 2020 brazilian symposium on robotics (sbr) and 2020 workshop on robotics in education (wre). *In*: . **Proceedings** [...]. [*S.l.: s.n.*], 2020.

MELO, D. C.; FORSTER, C. H. Q.; MáXIMO, M. R. O. A. Learning when to kick through deep neural networks. *In*: **2019 Latin American Robotics Symposium (LARS), 2019 Brazilian Symposium on Robotics (SBR) and 2019 Workshop on Robotics in Education (WRE)**. **Proceedings** [...]. [*S.l.: s.n.*], 2019. p. 43–48.

MELO, L. C.; MELO, D. C.; MAXIMO, M. R. Learning humanoid robot running motions with symmetry incentive through proximal policy optimization. **Journal of Intelligent & Robotic Systems**, Springer, v. 102, n. 3, p. 1–15, 2021.

MERKERT, R.; BUSHELL, J. Managing the drone revolution: A systematic literature review into the current use of airborne drones and future strategic directions for their effective control. **Journal of air transport management**, Elsevier, v. 89, p. 101929, 2020.

MICHAEL, N.; FINK, J.; JUDD, S.; WILCOX, D.; HUANG, A.; GUPTA, A.; IANDOLA, F.; KEUTZER, K. Mastering the game of go with deep neural networks and tree search. **Nature**, Nature Publishing Group, v. 529, n. 7587, p. 484–489, 2016.

MILLINGTON, I. **AI for Games**. CRC Press, Taylor & Francis Group, 2019. ISBN 9781138483972. Available at: https://books.google.com.br/books?id=HMKjuwEACAAJ.

MUNKRES, J. Algorithms for the assignment and transportation problems. **Journal of the society for industrial and applied mathematics**, SIAM, v. 5, n. 1, p. 32–38, 1957.

MUZIO, A. F.; MAXIMO, M. R.; YONEYAMA, T. Deep reinforcement learning for humanoid robot behaviors. **Journal of Intelligent & Robotic Systems**, Springer, v. 105, n. 1, p. 12, 2022.

NEIRA, J.; TARDOS, J. D. Data association in stochastic mapping using the joint compatibility test. **IEEE Transactions on Robotics and Automation**, IEEE, v. 17, n. 6, p. 890–897, 2001.

O'CONNOR, R. Pygame. **ACM SIGSOFT Software Engineering Notes**, ACM, v. 44, n. 1, p. 45–45, 2019.

OGREN, P. Increasing modularity of uav control systems using computer game behavior trees. **AIAA Guidance, Navigation, and Control Conference**, p. AIAA 2012–4458, August 2012.

OUSINGSAWAT, J.; EARL, M. G. Modified lawn-mower search pattern for areas comprised of weighted regions. *In*: **2007 American Control Conference**. **Proceedings** [...]. [*S.l.: s.n.*], 2007. p. 918–923.

PADEN, B.; ČÁP, M.; YONG, S. Z.; YERSHOV, D.; FRAZZOLI, E. A survey of motion planning and control techniques for self-driving urban vehicles. **IEEE Transactions on intelligent vehicles**, IEEE, v. 1, n. 1, p. 33–55, 2016.

PILGRIM, M. **Dive into Python 3**. [*S.l.*]: APRESS, 2009.

PINHEIRO, F. C. R.; MAXIMO, M. R. O. A.; YONEYAMA, T. 2020 latin american robotics symposium (lars), 2020 brazilian symposium on robotics (sbr) and 2020 workshop on robotics in education (wre). *In*: . **Proceedings** [...]. [*S.l.: s.n.*], 2020.

POPE, A. P.; IDE, J. S.; MIĆOVIĆ, D.; DIAZ, H.; ROSENBLUTH, D.; RITHOLTZ, L.; TWEDT, J. C.; WALKER, T. T.; ALCEDO, K.; JAVORSEK, D. Hierarchical reinforcement learning for air-to-air combat. *In*: IEEE. **2021 international conference on unmanned aircraft systems (ICUAS)**. **Proceedings** [...]. [*S.l.: s.n.*], 2021. p. 275–284.

PURSEL, B.; RONEN, A.; JAIN, A.; KUSHLEYEV, A.; TSIOTRAS, P. Autonomy and decision making in multi-uav missions: A survey. *In*: AMERICAN INSTITUTE OF AERONAUTICS AND ASTRONAUTICS. **2018 AIAA Guidance, Navigation, and Control Conference**. **Proceedings** [...]. [*S.l.: s.n.*], 2018.

QUIGLEY, M.; CONLEY, K.; GERKEY, B.; FAUST, J.; FOOTE, T.; LEIBS, J.; WHEELER, R.; NG, A. Y. *et al.* Ros: an open-source robot operating system. *In*: KOBE, JAPAN. **ICRA workshop on open source software**. **Proceedings** [...]. [*S.l.: s.n.*], 2009. v. 3, n. 3.2, p. 5.

RAMALHO, L. **Fluent Python: Clear, Concise, and Effective Programming**. [*S.l.*]: O'Reilly Media, 2017.

RICARDO, J. A.; GIACOMOSSI, L.; TRENTIN, J. F. S.; BRANCALION, J. F. B.; MAXIMO, M. R. O. A.; SANTOS, D. A. Cooperative threat engagement using drone swarms. **IEEE Access**, v. 11, p. 9529–9546, 2023.

RICARDO, J. A.; SANTOS, D. A. Attitude tracking control for a quadrotor aerial robot using adaptive sliding modes. **Proceedings of the XLI Ibero-Latin-American Congress on Computational Methods in Engineering**, 2020.

RICARDO, J. A.; SANTOS, D. A. Position guidance and control for fully actuated multirotor aerial vehicles in dynamic environments. **XLIII Ibero-Latin-American Congress on Computational Methods in Engineering**, 2022.

RICARDO, J. A.; SANTOS, D. A. Position guidance and control for fully actuated multirotor aerial vehicles in dynamic environments. **XLIII Ibero-Latin-American Congress on Computational Methods in Engineering**, 2022.

RICARDO, J. A.; SANTOS, D. A. Robot guidance and control using global sliding modes and acceleration velocity obstacles. **International Workshop on Variable Structure Systems and Sliding Mode Control**, 2022.

RICARDO, J. A.; SANTOS, D. A. Smooth second-order sliding mode control for fully actuated multirotor aerial vehicles. **ISA Transactions**, 2022. ISSN 0019-0578.

RICARDO, J. A.; SANTOS, D. A. Robust collision avoidance for mobile robots in the presence of moving obstacles. **IEEE Control Systems Letters**, v. 7, p. 1584–1589, 2023.

RICARDO, J. A.; SANTOS, D. A. dos. Robust collision-free guidance and control for fully actuated multirotor aerial vehicles. PREPRINT (Version 1) available at Research Square, May 2023. Available at: https://doi.org/10.21203/rs.3.rs-2786638/v1.

RIZK, Y.; AWAD, M.; TUNSTEL, E. W. Cooperative heterogeneous multi-robot systems: A survey. **ACM Computing Surveys (CSUR)**, ACM New York, NY, USA, v. 52, n. 2, p. 1–31, 2019.

RM, K. C.; MM, R.; JANARDHANAN, M. N. Review on state-of-the-art dynamic task allocation strategies for multiple-robot systems. **Industrial Robot: the international journal of robotics research and application**, Emerald Publishing Limited, v. 47, n. 6, p. 929–942, 2020.

ROBINSON, J. A.; KLOFAS, J. M.; ZUFFEREY, J.-C.; BEYELER, A.; MERMOUD, G.; SIEGWART, R. Autonomous battery swapping for extended operation of small unmanned aerial vehicles. *In*: IEEE. **2006 IEEE/RSJ International Conference on Intelligent Robots and Systems**. **Proceedings** [...]. [*S.l.: s.n.*], 2006. p. 4717–4722.

ROESNER, F.; CASTRILLIÓN, C. A. S.; HARTANTO, R.; STRUCK, A. Optimal search strategies for rescue drones based on swarm behaviour of different ethics. *In*: RODRIGUES, H.; HERSKOVITS, J.; SOARES, C. M.; ARAÚJO, A.; GUEDES, J.; FOLGADO, J.; MOLEIRO, F.; MADEIRA, J. F. A. (Ed.). **EngOpt 2018 Proceedings of the 6th International Conference on Engineering Optimization**. **Proceedings** [...]. Cham: Springer International Publishing, 2019. p. 122–131. ISBN 978-3-319-97773-7.

ROSSUM, G. V.; DRAKE, F. L. **Python tutorial**. [*S.l.*]: Centrum voor Wiskunde en Informatica Amsterdam, 1999.

RUSSELL, S.; NORVIG, P.; DAVIS, E. **Artificial Intelligence: A Modern Approach**. Prentice Hall, 2010. (Prentice Hall series in artificial intelligence). ISBN 9780136042594. Available at: https://books.google.com.br/books?id=8jZBksh-bUMC.

SANTOS, D. A.; BEZERRA, J. A. On the control allocation of fully actuated multirotor aerial vehicles. **Aerospace Science and Technology**, v. 122, p. 107424, 2022. ISSN 1270-9638.

SANTOSO, F.; GARRATT, M. A.; ANAVATTI, S. G. Visual–inertial navigation systems for aerial robotics: Sensor fusion and technology. **IEEE Transactions on Automation Science and Engineering**, IEEE, v. 14, n. 1, p. 260–275, 2016.

SCHWAB, D.; ZHU, Y.; VELOSO, M. Learning Skills for Small Size League RoboCup. *In*: . **Proceedings** [...]. Montreal, Canada: Springer, 2018. Nominated for Best Paper Award.

SELMAN, B.; GOMES, C. P. Hill-climbing search. **Encyclopedia of cognitive science**, Wiley, v. 81, p. 82, 2006.

SIEGWART, R.; NOURBAKHSH, I.; SCARAMUZZA, D. **Introduction to Autonomous Mobile Robots, second edition**. MIT Press, 2011. (Intelligent Robotics and Autonomous Agents series). ISBN 9780262015356. Available at: https://books.google.com.br/books?id=4of6AQAAQBAJ.

SIEGWART, R.; NOURBAKHSH, I. R.; SCARAMUZZA, D. **Introduction to Autonomous Mobile Robots**. Cambridge, Massachusetts, USA: The MIT Press, 2011.

SILVA, G. L.; MAXIMO, M. R. O. A.; PEREIRA, L. A. A minimalist open source behavior tree framework in c++. *In*: **2021 Latin American Robotics Symposium (LARS), 2021 Brazilian Symposium on Robotics (SBR), and 2021 Workshop on Robotics in Education (WRE)**. **Proceedings** [...]. [*S.l.: s.n.*], 2021. p. 306–311.

SILVA, M. Campos potenciais modificados aplicados ao controle de robôs em ambiente tridimensionais. **Universidade de São Paulo**, 2019.

SKOROBOGATOV, G.; BARRADO, C.; SALAMí, E. Multiple uav systems: A survey. **Unmanned Systems**, v. 08, n. 02, p. 149–169, 2020. Available at: https://doi.org/10.1142/S2301385020500090.

SOMMERVILLE, I. **Software Engineering**. Addison-Wesley, 1992. (International computer science series). ISBN 9780201565294. Available at: https://books.google.com.br/books?id=DoJQAAAAMAAJ.

STONE, L. D. Theory of optimal search. *In*: . **Proceedings** [...]. [*S.l.: s.n.*], 1975.

STONE, L. D.; ROYSET, J. O.; WASHBURN, A. R. *et al.* Optimal search for moving targets. Springer, 2016.

SUN, H.; QI, J.; WU, C.; WANG, M. Path planning for dense drone formation based on modified artificial potential fields. *In*: **2020 39th Chinese Control Conference (CCC)**. **Proceedings** [...]. [*S.l.: s.n.*], 2020. p. 4658–4664.

SUNDELIUS N., F. P. . S. R. Simulation environment evaluating ai algorithms for search missions using drone swarms. *In*: **IAI2023 0 International Congress and Workshop on Industrial AI**. **Proceedings** [...]. [*S.l.: s.n.*], 2023.

SWEIGART, A. **Invent your own computer games with Python**. [*S.l.*]: No Starch Press, 2012.

TOUSSAINT, M. The bayesian search game. *In*: **Theory and Principled Methods for the Design of Metaheuristics**. [*S.l.*]: Springer, 2013. p. 129–144.

TUCKER, P. **Air Force to Deploy 'Loyal Wingman' Drone Swarms to Accompany Fighter Planes**. 2020. Available at: https://www.defenseone.com/technology/2020/02-/air-force-deploy-loyal-wingman-drone-swarms-accompany-fighter-planes/163599/.

TURNER, R.; ERIKSSON, D.; MCCOURT, M.; KIILI, J.; LAAKSONEN, E.; XU, Z.; GUYON, I. Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020. *In*: ESCALANTE, H. J.; HOFMANN, K. (Ed.). **Proceedings of the NeurIPS 2020 Competition and Demonstration Track**. **Proceedings** [...]. PMLR, 2021. (Proceedings of Machine Learning Research, v. 133), p. 3–26. Available at: https://proceedings.mlr.press/v133/turner21a.html.

UTKIN, V. I. **Sliding modes in control and optimization**. [*S.l.*]: Springer Science & Business Media, 2013.

WAGNER, F.; SCHMUKI, R.; WAGNER, T.; WOLSTENHOLME, P. **Modeling Software with Finite State Machines: A Practical Approach**. CRC Press, 2006. ISBN 9781420013641. Available at: https://books.google.com.br/books?id=YmfLBQAAQBAJ.

WANG, Y.; CHENXIE, Y.; TAN, J.; WANG, C.; WANG, Y.; ZHANG, Y. Fuzzy radial basis function neural network pid control system for a quadrotor uav based on particle swarm optimization. *In*: **2015 IEEE International Conference on Information and Automation**. **Proceedings** [...]. [*S.l.: s.n.*], 2015. p. 2580–2585.

WONG, E.-M.; BOURGAULT, F.; FURUKAWA, T. Multi-vehicle bayesian search for multiple lost targets. *In*: IEEE. **Proceedings of the 2005 ieee international conference on robotics and automation**. **Proceedings** [...]. [*S.l.: s.n.*], 2005. p. 3169–3174.

XING, D.; ZHEN, Z.; GONG, H. Offense–defense confrontation decision making for dynamic UAV swarm versus UAV swarm. **Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering**, v. 233, n. 15, p. 5689–5702, 2019.

YANG, F.; JI, X.; YANG, C.; LI, J.; LI, B. Cooperative search of uav swarm based on improved ant colony algorithm in uncertain environment. *In*: IEEE. **2017 IEEE International Conference on Unmanned Systems (ICUS)**. **Proceedings** [...]. [*S.l.: s.n.*], 2017. p. 231–236.

YANG, Q.; ZHANG, J.; SHI, G.; HU, J.; WU, Y. Maneuver decision of uav in short-range air combat based on deep reinforcement learning. **IEEE Access**, v. 8, p. 363–378, 2020.

ZENG, L.; BONE, G. M. Mobile robot collision avoidance in human environments. **International Journal of Advanced Robotic Systems**, SAGE Publications Sage UK: London, England, v. 10, n. 1, p. 41, 2013.

ZHANG, M.; SHEN, Y.; WANG, Q.; WANG, Y. Dynamic artificial potential field based multi-robot formation control. *In*: **2010 IEEE Instrumentation Measurement Technology Conference Proceedings**. **Proceedings** [...]. [*S.l.: s.n.*], 2010. p. 1530–1534.

ZHANG, X.; WANG, C.; JIANG, L.; AN, L.; YANG, R. Collision-avoidance navigation systems for maritime autonomous surface ships: A state of the art survey. **Ocean Engineering**, Elsevier, v. 235, p. 109380, 2021.

ZHAO, Z.; ZHANG, H.; DAI, B. A review on the applications of unmanned aerial vehicle (uav) swarm in search and rescue. **International Journal of Disaster Risk Reduction**, Elsevier, v. 63, p. 102402, 2021.

ZHU, H.; ZHOU, M.; ALKINS, R. Group role assignment via a kuhn–munkres algorithm-based solution. **IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans**, IEEE, v. 42, n. 3, p. 739–750, 2011.

FOLHA DE REGISTRO DO DOCUMENTO

1. CLASSIFICAÇÃO/TIPO	2. DATA	3. REGISTRO N°	4. N° DE PÁGINAS
DM	28 de agosto de 2023	DCTA/ITA/DM-085/2023	169

5. TÍTULO E SUBTÍTULO:

Cooperative decision-making for drone swarms in simulated environments

6. AUTOR(ES):

Luiz Carlos Giacomossi Junior

7. INSTITUIÇÃO(ÕES)/ÓRGÃO(S) INTERNO(S)/DIVISÃO(ÕES):

Instituto Tecnológico de Aeronáutica – ITA

8. PALAVRAS-CHAVE SUGERIDAS PELO AUTOR:

Decision-Making; Drone; Loyal Wingman; Search and Rescue

9.PALAVRAS-CHAVE RESULTANTES DE INDEXAÇÃO:

Veículos não-tripulados; Tomada de decisões, Aprendizagem (inteligência artificial); Operações de resgate; Aeronaves teleguiadas; Monitoramento; Computação.

10. APRESENTAÇÃO: **(X) Nacional ( ) Internacional**

ITA, São José dos Campos. Curso de Mestrado. Programa de Pós-Graduação em Engenharia Eletrônica e Computação. Área de Informatica. Orientador: Prof. Dr. Marcos Ricardo Omena de Albuquerque Máximo; coorientador: Dr. José Fernando Basso Brancalion. Defesa em 28/07/2023. Publicada em 2023.

11. RESUMO:

The swarm of Unmanned Aerial Vehicles (UAV), also called swarm of drones, has been the subject of several research studies due to its potential to improve applications such as monitoring, surveillance, search and rescue missions and defense missions. With several drones flying simultaneously, the challenge is to increase their level of automation and intelligence, avoiding collisions, reducing the level of communication with these entities and improving the strategic organization to carry out a specific task. With the application of decision-making techniques, it is possible to automate many of the intelligent decisions of UAVs, leaving human commanders to focus on high-level decisions in modern scenarios, providing a crucial tactical advantage. In this context, two applications are studied, the first being a defense scenario and the second a civilian search and rescue scenario. The ability of multiple manned and unmanned aircraft systems to cooperatively engage and disable an aerial threat plays a decisive role in modern warfare scenarios. In this context, we apply key methods to enable the so-called cooperative threat engagement capability between a team of agents, e.g., a swarm of drones, with combat and communication capabilities. In particular, this research combines AI-based decision-making and control techniques for a swarm of loyal drones to coordinate efficient defense actions cooperatively and autonomously. In this sense, we apply these concepts in a modeled defense scenario to analyze the concept of loyal wingman, which is an interesting test for cooperative decision making and low-level control techniques. The investigated methods were initially implemented in a 2D simulator and later in a high fidelity 3D simulator to provide an application and evaluation of behavior strategies and control methods. This work also studies the application of tests in a scenario of search and rescue of people, where a solution is proposed to coordinate a swarm using an autonomous and distributed intelligence between drones for a cooperative application of search of people based on a discrete map shared. The results showed a swarm effectiveness using distributed decision-making methods, indicating to be an effective approach to solve the problem.

12. GRAU DE SIGILO:

(X) OSTENSIVO **( ) RESERVADO** **( ) SECRETO**